PCB Design Application

Tutorial-depth Implementation Reference

Prompt-driven schematic capture, manual & assisted PCB layout, manufacturing export

Document version: 1.0 — May 2026

Target stack: Python 3.11+ with PySide6 (Qt 6.11). All code in this document assumes that stack.

How to use this document: This is a reference handed to a code-generation model at the start of every session. The model reads the relevant chapter for the phase it is working on, plus Chapter 1 (this one) and Chapter 4 (architecture). Code in this document is illustrative and production-shaped, not always copy-paste runnable; it shows the structure you should implement and the patterns to follow.

1. How to Read This Document

1.1 What this is

This is a long-running engineering specification for a desktop PCB design application. It assumes you, the reader, are an LLM-based code-generation model operating in a code editor or a chat interface, and that a human is asking you to implement features from this spec one chunk at a time. It tells you how to build the application, in what order, what libraries to use, and which patterns to follow.

The human you are working for is a master's student with a Python and machine-learning background, learning PCB design and desktop application architecture as they go. Explain your decisions; do not just emit code. When this document and the human's request conflict, ask before you act.

1.2 The hard rules

These rules are non-negotiable. If you find yourself wanting to violate one, stop and ask the human first.

- **R1. Schematic and PCB are separate domains.** They share a netlist. They do not share data structures. A symbol is on a schematic; a footprint is on a board; a pin is on a symbol; a pad is on a footprint. Do not conflate them in code or in user-facing copy.
- **R2. The internal data model is structured.** It is JSON or pydantic on disk and in memory. SVG, Gerber, and PDF are only ever produced by exporters; they are never the primary representation, never the source of truth, never round-tripped.
- **R3. The LLM never emits finished output.** Any feature that uses an LLM emits a structured intermediate representation (the IR, Chapter 8). The IR is validated against a schema before any project state is mutated. If validation fails, surface the error to the user; never silently “correct” it.
- **R4. Use industry-correct terminology.** Trace, not wire, on the PCB. Pad, not pin, on a footprint. Solder mask, not UV resin and not copper colour. Silkscreen, not white text. Footprint, not symbol, when on the PCB. Net, not connection. The glossary in Chapter 22 is canonical.
- **R5. Reuse existing libraries.** KiCad's symbol and footprint libraries are CC-BY-SA 4.0 and ship thousands of production-grade parts. Do not build a parts library from scratch. FreeRouting is the auto-router; do not write your own. Gerbonara is the Gerber/Excellon library. Chapter 5 lists every dependency.
- **R6. Manual routing first.** Auto-routing is NP-hard. Provide good interactive routing tools, then add FreeRouting as an out-of-process integration via DSN/SES. A custom router is not a goal of this project.
- **R7. Core has no UI dependencies.** The core domain model imports nothing from PySide6 or any GUI library. Services sit above core. UI sits above services. Imports flow strictly downward. Violating this produces a tool that cannot be tested headless and cannot be scripted.

- **R8. Integer nanometre coordinates internally.** All positions in the data model are 64-bit signed integers expressed in nanometres. Floating-point creeps in only at the UI boundary and during transient geometry. Chapter 4 explains why.
- **R9. Tests for every domain function.** Pytest. Property-based with Hypothesis where the invariants warrant it. The data model is too central to test by hand.
- **R10. Never silently fabricate.** If a part is unknown, a pin number cannot be resolved, or a value is missing, raise an error and surface it. Do not invent values. Do not substitute a similar part. Do not guess pin numbers from a datasheet you have not read.

1.3 Build order

Six phases. Each ends with a demo-able milestone. Do not start phase N+1 until phase N runs end-to-end. The temptation to build the LLM pipeline first is strong; resist it. Without the data model and at least a working schematic editor, the LLM has nothing to produce that you can validate.

- **Phase 0 — Foundation.** Data model, project I/O, netlist generation, CLI. No UI. Chapter 6.
- **Phase 1 — Schematic editor MVP.** Canvas, place, wire, label, ERC, save/load. Chapter 7.
- **Phase 2 — PCB editor MVP.** Canvas, footprints, manual routing, ratsnest, DRC, Gerber export. Chapter 9.
- **Phase 3 — LLM pipeline.** IR schema, validator, prompt panel, diff-confirm, structured outputs. Chapter 8 (and 16).
- **Phase 4 — Library import and polish.** KiCad library reader, custom editors, BOM, PnP, SVG/PDF docs export. Chapter 14.
- **Phase 5 — Routing and DRC depth.** FreeRouting integration, push-and-shove, live DRC, net classes. Chapter 15.

1.4 The persistent prompt

Use the following text at the start of any new code session. It tells the model how to behave when working on this project. Edit only the placeholder at the bottom (the task description) per session; the rest stays constant.

```
You are helping me build a desktop PCB design application called “PCBSmith”
```

```
(rename as desired). Read “PCB_Application_Specification.docx” before
responding; the relevant chapter for today's task is named in the task line.

Hard rules (do not violate):
1. Schematic and PCB are separate domains, linked by a netlist. Never conflate.
2. Internal data model is structured (pydantic / JSON / S-expr). SVG and Gerber
   are export-only.
3. LLM features emit a structured JSON intermediate representation, validated
   against a schema before mutating state. No raw SVG, no finished schematics.
4. Industry terminology: trace not wire, pad not pin, solder mask not UV resin,
   silkscreen not white text, footprint not symbol on PCB, net not connection.
5. Reuse existing libraries: KiCad libraries (CC-BY-SA), FreeRouting, Gerbonara,
   pyclipper. Do not reinvent these.
6. Manual routing first; FreeRouting integration second; never a custom router.
7. Core domain has no UI imports. UI → services → core. No upward imports.
8. Coordinates are 64-bit signed integer nanometres in the data model.
9. Pytest for every domain function. Hypothesis for invariants.
10. Never silently fabricate parts, pins, or values. Raise and surface errors.

Build order (do not skip ahead):
  Phase 0: data model + netlist + CLI  →  Chapter 6
  Phase 1: schematic editor            →  Chapter 7
  Phase 2: PCB editor + Gerber export  →  Chapter 9
  Phase 3: LLM pipeline + IR validator →  Chapter 8 + 16
  Phase 4: library import, BOM, polish →  Chapter 14
  Phase 5: FreeRouting + advanced      →  Chapter 15

Stack:
  Python 3.11+, PySide6 6.11+, pydantic 2.x, pytest, hypothesis, gerbonara,
  pyclipper, shapely, networkx, anthropic SDK (>=0.40 with structured outputs).
  See Chapter 5 for the full pinned list.

When you produce code:
  - Show me the file tree first, then the file you are working on.
  - Build incrementally. One module at a time, smallest unit first.
  - Tests live next to the code, in tests/ mirroring the package layout.
  - Explain why, not just what. Cite the chapter section by number.
  - Flag anything in the spec you think is wrong before implementing it.
  - Use type hints everywhere. mypy --strict should pass.

If I ask for something that violates a hard rule, push back and explain the
conflict before doing it. If two parts of the spec contradict each other,
ask which to follow.
```

```
Today's task: <FILL IN PHASE, CHAPTER, AND TASK>
```

Replace the last line per session, e.g. “Today's task: Phase 0, Chapter 6 §6.3, implement the Schematic and Net pydantic models with full unit tests.”

2. Project Identity and Naming

2.1 What you are building

PCBSmith (working title; rename freely) is a desktop application that lets a single user design a printed circuit board through three complementary input methods. They are complementary, not exclusive: a single project can use any combination.

- **Natural-language interface.** The user types or pastes a description — “an STM32F411 with USB-C and a power LED” — and a large language model emits a structured intermediate representation that the application converts into real schematic objects. The model never emits a finished schematic. It emits an IR.
- **Graphical schematic editor.** The user places symbols from a library and wires them together. This is the same workflow as KiCad's Eeschema, EasyEDA's schematic editor, or Altium's schematic editor.
- **Graphical PCB layout editor.** Footprints derived from the schematic netlist are placed on a board outline; the user routes traces between pads, places vias, and pours zones. Optional auto-routing via FreeRouting.

2.2 What you are not building

Saying no early is how this project ships. The following are explicitly out of scope for the first six phases. They may move to scope in a future iteration; until then, do not implement them.

- Full SPICE simulation (use ngspice as an external tool if needed in V2).
- 3D rendering of the populated board (defer; view in KiCad's 3D viewer if needed).
- Real-time multi-user collaboration.
- Mixed-signal analog routing helpers beyond basic length matching.
- RF / microwave-specific features (controlled impedance calculators, Smith charts).
- Replacing KiCad. The goal is a focused, prompt-driven companion tool.
- Cloud sync, accounts, paid tiers. This is a single-user desktop application.
- Mobile or tablet support.

2.3 Reference applications

Study these tools while building. Each demonstrates patterns you should follow or explicitly diverge from.

Tool	What to learn from it	What to avoid
KiCad 9	Data model, file formats, library structure, ratsnest, DRC. The	Plugin architecture is heavy; we don't need it. Eeschema's older UI

	reference open-source EDA.	patterns.
Altium Designer	Component library workflow, push-and-shove routing, design rules.	Closed-source, expensive, complex. We are not cloning it.
EasyEDA	Web-first UX, library browser, JLCPCB integration.	Web-only architecture. We are desktop.
Horizon EDA	Modern open-source EDA in C++/GTK; clean data model.	GTK; we are using Qt.
Fritzing	Beginner-friendly UI, breadboard-first onboarding.	Limited PCB capability; not a reference for serious boards.
tscircuit	Code-as-circuit (React-based JSX). Demonstrates structured-input PCB design.	Different abstraction model; we read it for the IR-style approach.

2.4 Audience

Primary audience: hobbyists and students who can describe a circuit in plain English but do not yet have the patience for full KiCad. The LLM lowers the bar for the first draft; the editor lets them iterate. Secondary audience: experienced users who want a fast “generate the boilerplate” step before dropping into KiCad for serious work; for them, KiCad-format export is the killer feature.

3. Corrections to Common Misconceptions

This chapter codifies corrections from the original project description and from common ML-engineer misconceptions about EDA tools. Read it once. Internalise it. Code reviews should reject any change that reintroduces these errors.

3.1 SVG is not a PCB format

SVG is a 2D vector rendering format. It has no concept of a net, a pin, a footprint, a copper layer, a drill, an aperture, a clearance, or any of the other things a PCB tool needs to model. If you store a board as SVG, you cannot route it, cannot run DRC on it, cannot export it to manufacturing, and cannot edit it without re-parsing the SVG every time.

SVG is one of several output formats. The application uses SVG to render a board for documentation — a blog post, a build guide, a homework submission. It is not how the application thinks about a board internally.

> **Practical consequence.** When the LLM is asked to generate a circuit, it must not emit SVG. It emits the IR (Chapter 8). When the user clicks “Export SVG,” that is an exporter pulling from the structured data model and writing SVG — a one-way operation, never read back.

3.2 Schematic and PCB are distinct stages

A schematic is the logical circuit. It contains symbols (R1, U1, J3) and nets (VCC, GND, SDA, SCL). It is intentionally abstract: a 10 kΩ resistor on a schematic does not yet have a package, only a value and connections.

A PCB is the physical realisation. It contains footprints (the land patterns of specific packages: 0603, SOIC-8), traces (copper paths on a layer), vias (plated holes between layers), zones (filled copper areas), and a board outline. Every footprint on the PCB is bound to a symbol instance on the schematic by the reference designator (R1 on the schematic = R1's footprint on the board).

The professional workflow has clear stages:

```
schematic capture
```

```
   → footprint assignment (per component)
   → netlist generation
   → board outline
   → footprint placement
   → routing
   → design rule check (DRC)
```

```
   → manufacturing export (Gerber + Excellon + BOM + PnP)
```

Your code mirrors these stages. Do not introduce features that require collapsing them. If the user wants to “draw a PCB” without a schematic, they are asking for something the data model does not support; the right answer is to first generate a schematic (possibly via the LLM) that produces the netlist.

3.3 Auto-routing is hard

Routing a multi-net PCB on multiple layers with design rules is NP-hard in the general case. Commercial routers (Altium, Cadence, Mentor) are the product of decades of specialist work. FreeRouting, the leading open-source router, has a small core team and still produces sub-optimal routes on dense boards.

Plan accordingly. Manual routing is the default and the launch feature. Auto-routing comes from FreeRouting integration in Phase 5, run as an out-of-process tool over the DSN/SES interface (Chapter 15). Do not write a custom router; you will not produce anything competitive in less than several years of focused work.

3.4 Component libraries are a massive undertaking

A serviceable PCB tool needs thousands of components, each with: a symbol, one or more candidate footprints, accurate pin numbers, accurate pad sizes, courtyard, silkscreen, fabrication notes, and ideally a 3D model. KiCad's official libraries on GitLab (gitlab.com/kicad/libraries) contain on the order of 20,000 symbols and 15,000 footprints, all CC-BY-SA 4.0 licensed.

Use them. Do not build your own. Chapter 14 covers how to read KiCad library files. Phase 1 ships with a hard-coded library of about 25 parts so that the schematic editor is testable without the full KiCad import; that is a development crutch, not a production library.

3.5 Terminology, made strict

Wrong / vague term	Correct term	Notes
wire (on PCB)	trace	Wire is on the schematic. Trace is

		the copper path on the board.
pin (on a PCB footprint)	pad	Pin is on a symbol. Pad is the copper that solders to a pin.
circuit (the file)	schematic / board	Be specific. A project contains both.
hole	drill, via, or mounting hole	PTH (plated through hole), NPTH (non-plated). Vias connect layers.
copper colour	solder mask colour	Copper is always copper. The mask sits on top.
UV resin colour	solder mask colour	UV resin is for SLA 3D printing; solder mask is the PCB coating.
white text	silkscreen	The white legend layer (also called silk or legend).
connection	net	A net is a named electrical connection set.
component (on PCB)	footprint instance	Component is a logical concept; footprint is its physical placement.
wire colour	net class colour	Schematic UI may colour wires by net class for clarity.
track / trail	trace	Track is acceptable in some communities. Pick trace and stay consistent.

3.6 Units and precision

PCB tools store coordinates as integers, not floats. Floating-point arithmetic produces rounding errors that compound across operations: a trace endpoint moved through a rotation, a translation, and a snap-to-grid will not return exactly to its starting position with floats. Errors of a few nanometres will manifest as DRC failures and drill misalignment.

Use 64-bit signed integers in nanometres. KiCad uses this internally; so does Eagle. The maximum representable distance is about $\pm 9.2 \times 10^{18}$ mm (astronomically larger than any board). The smallest representable distance is 1 nm (roughly 1000× finer than any fab can produce). Convert to and from millimetres / mils only at the UI boundary and during display.

```
# Accepted unit conventions in this codebase
```

```
# Internal storage: nanometres, signed int64
position_nm: int = 12_700_000   # 12.7 mm

# UI display: millimetres, float, 3 decimal places
def nm_to_mm(nm: int) -> float:
    return nm / 1_000_000

def mm_to_nm(mm: float) -> int:
    return int(round(mm * 1_000_000))

# Schematic grid: 50 mil = 1.27 mm = 1_270_000 nm
SCHEMATIC_GRID_NM = 1_270_000

# Common PCB grids: 0.05 mm = 50_000 nm; 0.025 mm = 25_000 nm
PCB_GRID_FINE_NM = 25_000
```

```
PCB_GRID_COARSE_NM = 50_000
```

4. Architecture and Data Model

4.1 Layered architecture

Strict separation. Imports flow downward only. The core domain model imports nothing outside the standard library and pydantic. Services may import core. UI may import services and core. Reverse imports are forbidden and should be enforced by an import-linter rule in CI.

```
┌──────────────────────────────────────────────────────────┐
│ ui/                    PySide6, QGraphicsScene/View      │
│   schematic_canvas.py  pcb_canvas.py   prompt_panel.py   │
│   library_palette.py   project_tree.py inspector.py      │
│   main_window.py                                         │
├──────────────────────────────────────────────────────────┤
│ services/              application services, side-effects│
│   project_io.py         load/save *.pcbsmith files       │
│   library_loader.py     KiCad and built-in libraries     │
│   netlist.py            schematic → netlist computation  │
│   erc.py drc.py         validation engines               │
│   exporters/            gerber.py excellon.py svg.py pdf.py bom.py │
│   llm/                  ir_schema.py validator.py prompt.py client.py │
│   freerouting.py        DSN export, FreeRouting subprocess, SES import │
├──────────────────────────────────────────────────────────┤
│ core/                  pure data + pure functions, NO side effects │
│   geom.py               Point, Vec, Box, Polyline, geometric primitives │
│   ids.py                typed ids: ProjectId, SymbolId, NetId, etc. │
│   schematic.py          Schematic, SymbolInstance, Wire, Net, NetLabel │
│   board.py              Board, FootprintInstance, Trace, Via, Zone │
│   library.py            Symbol, Pin, Footprint, Pad (template entities) │
│   project.py            Project root entity, design rules │
│   netops.py             pure functions: derive_netlist, merge_nets, ... │
└──────────────────────────────────────────────────────────┘
```

A useful test: can you run pytest without ever importing PySide6? You should be able to. If a core test fails because Qt did not initialise, you have a layering bug.

4.2 Project layout on disk

A project is a folder, not a single file. This is the same model KiCad uses (with its .kicad_pro / .kicad_sch / .kicad_pcb triple). A folder makes it easier to version-control, easier to diff, and easier to use with multiple schematic sheets.

```
my_project/
```

```
├── project.pcbsmith              # JSON; project metadata, design rules, library
refs
├── schematics/
│   ├── main.sch.json             # primary schematic sheet
│   └── power.sch.json            # secondary sheets (optional)
├── boards/
│   └── main.brd.json             # primary board
├── library/
│   ├── symbols/                  # custom symbols (.sym.json)
│   └── footprints/               # custom footprints (.fp.json)
├── prompts/
│   └── history.jsonl             # append-only log of LLM prompts and IRs
└── output/                       # generated; safe to delete
    ├── gerbers/
    ├── svg/
    ├── pdf/
```

```
    └── bom.csv
```

Why JSON, not S-expressions? JSON is well-supported in every language, parses faster than S-expressions in pure Python, and works directly with pydantic. KiCad uses S-expressions, but interoperability is a one-shot import-export problem (Chapter 14), not a daily concern. Keep your native format simple.

4.3 Core entities, in detail

These are the entities the data model must carry. Names are deliberately matched to PCB-industry terminology so future contributors recognise them and so external tools (KiCad importers, BOM generators) line up cleanly.

Entity	Lives in	Purpose
Project	project.pcbsmith	Top-level container. Holds schematics, boards, library refs, design rules, settings.
DesignRules	Project	Min trace width, min clearance, drill sizes, courtyard rules. Validated by DRC.
Schematic	schematics/*.sch.json	Logical circuit. Contains symbol instances, wires, net labels,

		junctions.
Symbol	library	Reusable symbol template: body shape, pins. Has a unique library:name id.
Pin	Symbol	Electrical connection point on a symbol. Number, name, position, electrical type.
SymbolInstance	Schematic	Placement of a Symbol on a sheet. Reference designator (R1), value (10k), position, rotation.
Wire	Schematic	Drawn polyline visually representing part of a Net. Snap to grid.
Junction	Schematic	Explicit T-intersection between two wires. Drawn as a dot.
NetLabel	Schematic	Named wire endpoint: “VCC”, “GND”, “SDA”. Same-name labels merge nets.
Net	Schematic (computed)	Named electrical connection set. Computed from wires, junctions, labels, pins. Not edited directly.
Footprint	library	Physical land pattern: pads, drills, courtyard, silkscreen graphics.
Pad	Footprint	One piece of copper on a footprint. Shape, size, drill, layers, pin number it maps to.
Board	boards/*.brd.json	Physical PCB. Outline, layer stackup, footprint instances, traces, vias, zones, design rules.
Layer	Board	Stackup entry: ordinal, name, type (signal/plane/silkscreen/mask/paste/fab/edge).
FootprintInstance	Board	Placement of a Footprint, linked to a SymbolInstance via reference designator.
Trace	Board	Copper segment on a signal layer connecting two points along a Net.

Via	Board	Plated through-hole or blind/buried hole connecting two layers along a Net.
Zone	Board	Filled copper region on a layer, tied to a Net (typically GND).
DimensionedShape	Board	Drawn graphics: dimensions, fab notes, anything not copper.

4.4 The pydantic model, in full

Below is the full pydantic 2.x model. This is what Chapter 6 (Phase 0) asks you to implement. Read it carefully. Field types are deliberate; do not loosen them. Frozen / mutable status is deliberate; do not flip it.

4.4.1 Identifiers and primitives

```
# core/ids.py
```

```
from typing import NewType
import uuid

# Strongly-typed UUIDs prevent passing a NetId where a SymbolId is wanted.
ProjectId        = NewType("ProjectId",        str)
SchematicId      = NewType("SchematicId",      str)
BoardId          = NewType("BoardId",          str)
SymbolId         = NewType("SymbolId",         str)   # library:name, e.g.
"stdlib:R"
SymbolInstanceId = NewType("SymbolInstanceId", str)
PinNumber        = NewType("PinNumber",        str)   # "1", "A6", "CLK"
NetId            = NewType("NetId",            str)
NetName          = NewType("NetName",          str)   # "VCC", "GND", "~"
WireId           = NewType("WireId",           str)
JunctionId       = NewType("JunctionId",       str)
NetLabelId       = NewType("NetLabelId",       str)
FootprintId      = NewType("FootprintId",      str)
FootprintInstanceId = NewType("FootprintInstanceId", str)
PadNumber        = NewType("PadNumber",        str)
TraceId          = NewType("TraceId",          str)
ViaId            = NewType("ViaId",            str)
ZoneId           = NewType("ZoneId",           str)
LayerName        = NewType("LayerName",        str)   # "F.Cu", "B.Cu", "In1.Cu"

def new_id() -> str:
    return str(uuid.uuid4())
```

```

```

```
# core/geom.py
```

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Tuple

# All distances in nanometres (int64). See §3.6.

@dataclass(frozen=True, slots=True)
class Point:
    x: int   # nm
    y: int   # nm

    def __add__(self, other: "Vec") -> "Point":
        return Point(self.x + other.dx, self.y + other.dy)

    def __sub__(self, other: "Point") -> "Vec":
        return Vec(self.x - other.x, self.y - other.y)

    def to_mm(self) -> Tuple[float, float]:
        return (self.x / 1_000_000, self.y / 1_000_000)

    @classmethod
    def from_mm(cls, x_mm: float, y_mm: float) -> "Point":
        return cls(int(round(x_mm * 1_000_000)), int(round(y_mm * 1_000_000)))


@dataclass(frozen=True, slots=True)
class Vec:
    dx: int
    dy: int


@dataclass(frozen=True, slots=True)
class Box:
    """Axis-aligned bounding box. Used for hit testing, layout queries."""
    x0: int
    y0: int
    x1: int
    y1: int

    @property
    def width(self) -> int:  return self.x1 - self.x0
    @property
    def height(self) -> int: return self.y1 - self.y0

    def contains(self, p: Point) -> bool:
        return self.x0 <= p.x <= self.x1 and self.y0 <= p.y <= self.y1

    def intersects(self, other: "Box") -> bool:
```

```
        return not (self.x1 < other.x0 or other.x1 < self.x0
                 or self.y1 < other.y0 or other.y1 < self.y0)


# Snap a point to a grid spacing in nm.
def snap(p: Point, grid_nm: int) -> Point:
    return Point(
        round(p.x / grid_nm) * grid_nm,
        round(p.y / grid_nm) * grid_nm,
    )
```

```
```

4.4.2 Library entities (Symbol, Pin, Footprint, Pad)

```
# core/library.py
```

```
from __future__ import annotations
from enum import Enum
from typing import Annotated
from pydantic import BaseModel, Field, ConfigDict
from .geom import Point
from .ids import SymbolId, PinNumber, FootprintId, PadNumber, LayerName


class PinElectricalType(str, Enum):
    INPUT       = "input"
    OUTPUT      = "output"
    BIDI        = "bidi"
    TRISTATE    = "tristate"
    PASSIVE     = "passive"
    POWER_IN    = "power_in"
    POWER_OUT   = "power_out"
    OPEN_COLL   = "open_collector"
    OPEN_EMIT   = "open_emitter"
    NC          = "no_connect"
    UNSPECIFIED = "unspecified"


class PinDirection(str, Enum):
    """Which side of the symbol the pin sticks out from."""
    UP    = "up"
    DOWN  = "down"
    LEFT  = "left"
    RIGHT = "right"


class Pin(BaseModel):
    model_config = ConfigDict(frozen=True)
    number: PinNumber                 # "1", "A6"
    name: str                          # "VCC", "SDA", "~" if anonymous
    position: Point                    # in symbol-local coords (nm)
    direction: PinDirection
    length_nm: int = 2_540_000         # default 100 mil
    electrical_type: PinElectricalType = PinElectricalType.PASSIVE
    visible: bool = True


class Symbol(BaseModel):
    """A reusable symbol template. Lives in a library."""
    model_config = ConfigDict(frozen=True)
    id: SymbolId                       # "stdlib:R", "kicad:Device:R"
    name: str                          # human-readable: "Resistor"
```

```
    description: str = ""
    keywords: list[str] = Field(default_factory=list)
    reference_prefix: str = "U"        # the letter for ref designators: R, C, U,
J, ...
    pins: list[Pin] = Field(default_factory=list)
    body_polylines: list[list[Point]] = Field(default_factory=list)
    body_circles: list[tuple[Point, int]] = Field(default_factory=list)  #
center, radius_nm
    default_footprint: FootprintId | None = None
    fields: dict[str, str] = Field(default_factory=dict)  # Manufacturer, MPN,
Datasheet, ...


class PadShape(str, Enum):
    CIRCLE   = "circle"
    RECT     = "rect"
    OVAL     = "oval"
    RRECT    = "rounded_rect"
    CUSTOM   = "custom"


class PadType(str, Enum):
    SMT_TOP  = "smt_top"
    SMT_BOT  = "smt_bottom"
    THT      = "through_hole"   # PTH
    NPTH     = "non_plated"     # mounting hole, slot etc.
    CONNECT  = "connect"        # connector edge contact


class Pad(BaseModel):
    """One pad on a footprint."""
    model_config = ConfigDict(frozen=True)
    number: PadNumber              # matches the symbol pin number it solders to
    type: PadType
    shape: PadShape
    position: Point                # footprint-local
    rotation_deg: float = 0.0
    size_x_nm: int                 # bounding pad size
    size_y_nm: int
    drill_diameter_nm: int = 0     # 0 for SMT
    drill_slot_y_nm: int = 0       # for slotted PTH; 0 = round
    layers: list[LayerName]        # which copper layers this pad reaches
    paste_margin_nm: int | None = None
    mask_margin_nm: int | None = None


class Footprint(BaseModel):
    model_config = ConfigDict(frozen=True)
    id: FootprintId                # "stdlib:R_0603",
"kicad:Resistor_SMD:R_0603_1608Metric"
```

```
    name: str
    description: str = ""
    keywords: list[str] = Field(default_factory=list)
    pads: list[Pad]
    silkscreen_polylines: list[list[Point]] = Field(default_factory=list)
    courtyard_polylines: list[list[Point]] = Field(default_factory=list)
    fab_polylines:       list[list[Point]] = Field(default_factory=list)
    reference_anchor: Point = Point(0, 0)         # where the ref designator goes
by default
    value_anchor:     Point = Point(0, 0)
    height_nm: int = 0                            # for collision and 3D
    model_3d: str | None = None                   # path or URL
```

```
```

4.4.3 Schematic entities

```
# core/schematic.py
```

```
from __future__ import annotations
from typing import Annotated, Literal
from pydantic import BaseModel, Field, ConfigDict
from .geom import Point
from .ids import (SchematicId, SymbolId, SymbolInstanceId, PinNumber,
                  WireId, JunctionId, NetLabelId, NetName)


class SymbolInstance(BaseModel):
    """A placed symbol on a schematic sheet."""
    id: SymbolInstanceId
    symbol_id: SymbolId
    reference: str                  # "R1", "U3". Unique within the schematic.
    value: str                       # "10k", "STM32F411CEU6"
    position: Point
    rotation_deg: Literal[0, 90, 180, 270] = 0
    mirror_x: bool = False           # vertical mirror
    mirror_y: bool = False           # horizontal mirror
    fields: dict[str, str] = Field(default_factory=dict)
    footprint_id: str | None = None  # filled at footprint-assignment time


class Wire(BaseModel):
    """A drawn polyline on the schematic. Visual; nets are derived from it."""
    id: WireId
    points: list[Point]             # >= 2 points, ortho preferred but free-
angle allowed


class Junction(BaseModel):
    """Explicit dot at a wire intersection that should be electrically
connected."""
    id: JunctionId
    position: Point


class NetLabel(BaseModel):
    """A name attached to a wire endpoint. Same name elsewhere = same net."""
    id: NetLabelId
    position: Point
    text: NetName
    is_power: bool = False          # power ports (GND, +3V3, +5V) drawn
specially
    rotation_deg: Literal[0, 90, 180, 270] = 0
```

```
class NoConnect(BaseModel):
    """Marker on a pin that is intentionally left disconnected. Suppresses ERC
warning."""
    position: Point


class Schematic(BaseModel):
    id: SchematicId
    name: str = "main"
    title_block: dict[str, str] = Field(default_factory=dict)  # title, rev,
date, comp
    sheet_size: Literal["A4", "A3", "A2", "US-A", "US-B"] = "A3"
    symbols:    list[SymbolInstance] = Field(default_factory=list)
    wires:      list[Wire]           = Field(default_factory=list)
    junctions:  list[Junction]       = Field(default_factory=list)
    labels:     list[NetLabel]       = Field(default_factory=list)
    no_connects: list[NoConnect]     = Field(default_factory=list)


class PinConnection(BaseModel):
    """A (symbol_instance, pin_number) tuple. The atomic unit of a netlist."""
    model_config = ConfigDict(frozen=True)
    symbol_instance_id: SymbolInstanceId
    pin_number: PinNumber

    def designator_dot(self, schematic: Schematic) -> str:
        """"R1.1" style human-readable. Resolves the instance ref from the
schematic."""
        for inst in schematic.symbols:
            if inst.id == self.symbol_instance_id:
                return f"{inst.reference}.{self.pin_number}"
        return f"<unknown>.{self.pin_number}"


class Net(BaseModel):
    """Computed; not edited directly. See netops.derive_netlist."""
    id: str
    name: NetName                    # explicit if labelled, else auto: "Net-(R1-
1)"
    pins: list[PinConnection] = Field(default_factory=list)
    is_power: bool = False
    net_class: str = "default"
```

4.4.4 Board entities

```
# core/board.py
```

```
from __future__ import annotations
from enum import Enum
from typing import Literal
from pydantic import BaseModel, Field
from .geom import Point
from .ids import (BoardId, FootprintId, FootprintInstanceId, PadNumber,
                  TraceId, ViaId, ZoneId, NetId, LayerName)


class LayerType(str, Enum):
    SIGNAL    = "signal"      # routable copper
    POWER     = "power"        # plane copper
    SILK      = "silkscreen"
    MASK      = "solder_mask"
    PASTE     = "solder_paste"
    FAB       = "fabrication"
    EDGE      = "edge_cuts"
    ADHES     = "adhesive"
    USER      = "user"


class Layer(BaseModel):
    ordinal: int                    # 0 = front copper, last copper = bottom
    name: LayerName                 # canonical: F.Cu, In1.Cu, B.Cu, F.SilkS,
etc.
    type: LayerType
    visible: bool = True
    color: str | None = None        # "#RRGGBB"; None uses theme default


class FootprintInstance(BaseModel):
    id: FootprintInstanceId
    footprint_id: FootprintId       # library reference
    reference: str                  # matches a SymbolInstance.reference (R1,
U1, ...)
    value: str
    position: Point
    rotation_deg: float = 0.0       # arbitrary; not just 0/90/180/270
    side: Literal["top", "bottom"] = "top"
    locked: bool = False


class Trace(BaseModel):
    id: TraceId
    layer: LayerName
    width_nm: int
```

```
    points: list[Point]              # 2 → segment; >2 → polyline (45°/90°
corners typical)
    net_id: NetId | None = None      # may be None during draw, must be set on
commit


class ViaType(str, Enum):
    THROUGH = "through"
    BLIND   = "blind"
    BURIED  = "buried"
    MICRO   = "micro"


class Via(BaseModel):
    id: ViaId
    type: ViaType = ViaType.THROUGH
    position: Point
    drill_nm: int
    diameter_nm: int                 # outer pad diameter
    layer_from: LayerName
    layer_to: LayerName
    net_id: NetId | None = None


class Zone(BaseModel):
    id: ZoneId
    layer: LayerName
    net_id: NetId | None = None      # GND or VCC; None = isolated copper
    polygon: list[Point]             # outline; the fill is computed
    clearance_nm: int = 200_000      # 0.2 mm default
    min_thickness_nm: int = 250_000  # 0.25 mm
    thermal_relief_gap_nm: int = 200_000
    thermal_spoke_width_nm: int = 250_000


class DesignRules(BaseModel):
    min_trace_width_nm:    int = 150_000   # 0.15 mm
    min_clearance_nm:      int = 150_000
    min_drill_nm:          int = 300_000   # 0.30 mm
    min_via_diameter_nm:   int = 600_000
    min_annular_ring_nm:   int = 100_000
    min_silk_to_pad_nm:    int =  50_000
    courtyard_clearance_nm: int = 0


class Board(BaseModel):
    id: BoardId
    name: str = "main"
    layers: list[Layer]              # canonical ordering, 2/4/6/8 copper
```

```
    outline: list[Point] = Field(default_factory=list)  # closed polygon on
Edge.Cuts
    footprints: list[FootprintInstance] = Field(default_factory=list)
    traces: list[Trace]              = Field(default_factory=list)
    vias:   list[Via]                = Field(default_factory=list)
    zones:  list[Zone]               = Field(default_factory=list)
    design_rules: DesignRules = Field(default_factory=DesignRules)
    thickness_nm: int = 1_600_000    # 1.6 mm standard
    solder_mask_color: Literal["green", "red", "blue", "black", "white",
"yellow", "purple"] = "green"
    silkscreen_color:  Literal["white", "black"] = "white"
    surface_finish:    Literal["HASL", "ENIG", "OSP", "hard_gold"] = "HASL"
```

```
```

4.4.5 Project root

```
# core/project.py
```

```
from __future__ import annotations
from pydantic import BaseModel, Field
from .ids import ProjectId
from .schematic import Schematic
from .board import Board, DesignRules


class LibraryRef(BaseModel):
    """Reference to an external library directory or file."""
    name: str                        # "kicad-symbols", "stdlib"
    path: str                        # filesystem path or builtin://stdlib
    enabled: bool = True


class Project(BaseModel):
    id: ProjectId
    name: str
    description: str = ""
    author: str = ""
    schematics: list[Schematic] = Field(default_factory=list)
    boards:     list[Board]     = Field(default_factory=list)
    libraries:  list[LibraryRef] = Field(default_factory=list)
    design_rules: DesignRules = Field(default_factory=DesignRules)
    created_iso: str = ""
    modified_iso: str = ""
    schema_version: int = 1
```

```
```

4.5 The netlist is the contract

The netlist is the single source of truth that ties the schematic to the board. Every electrical decision flows from it. Understand this section before writing any DRC code.

- When the user wires two pins on the schematic, the netlist gains a connection.
- When the user opens the PCB editor, the netlist tells the ratsnest which pads must be connected.
- When a trace is routed between two pads on the same Net, DRC marks that connection satisfied.
- When a trace touches pads on different Nets, DRC raises a short.
- When a Net contains pins of incompatible electrical types (two outputs), ERC raises a conflict.

The netlist is computed from the schematic, not stored. Treat it as a derived view. If you store it independently, sooner or later it will drift from the schematic, and diagnosing the resulting bugs is hellish. Recompute it after every schematic edit; the computation is cheap.

4.6 Netlist derivation algorithm

Given a Schematic, derive its list of Nets. The algorithm is union-find over pin connection points, with three sources of equivalence: shared wire endpoints, junctions, and shared net labels.

```
# core/netops.py
```

```
from __future__ import annotations
from collections import defaultdict
from .schematic import Schematic, Net, PinConnection
from .library import Symbol
from .geom import Point

def derive_netlist(
    sch: Schematic,
    symbol_lookup: dict[str, Symbol],   # symbol_id → Symbol
) -> list[Net]:
    """Compute Nets from the visual schematic.

    Steps:
      1. Collect every electrical anchor: each pin's tip, each wire endpoint,
         each junction, each net label.
      2. Build an equivalence relation (union-find) joining anchors that:
           - share the same Point (after snap-to-grid)
           - lie along the same wire's polyline
           - share a label name (label-to-label cross-sheet)
      3. Each equivalence class becomes a Net.
      4. The Net's name is the label name if any anchor in the class is a label,
         otherwise auto: "Net-(R1-2)" using the first pin alphabetically.
    """
    uf = UnionFind()

    # 1. Pin tips. World position of each pin = symbol position +
rotated/mirrored pin pos.
    pin_anchors: dict[Point, list[PinConnection]] = defaultdict(list)
    for inst in sch.symbols:
        sym = symbol_lookup.get(inst.symbol_id)
        if sym is None:
            raise ValueError(f"Unresolved symbol {inst.symbol_id}")
        for pin in sym.pins:
            tip = transform_pin(pin, inst)
            uf.add(tip)
            pin_anchors[tip].append(PinConnection(
                symbol_instance_id=inst.id,
                pin_number=pin.number,
            ))

    # 2. Wire endpoints and segments.
    for w in sch.wires:
        for pt in w.points:
            uf.add(pt)
        for a, c in zip(w.points, w.points[1:]):
            uf.union(a, c)

    # 3. Junctions force a union of all wires touching them.
```

```
    for j in sch.junctions:
        uf.add(j.position)
        # Any wire vertex coincident with this junction joins.

    # 4. Net labels: every label position joins the wire it touches; same-name
    #    labels also join each other.
    label_groups: dict[str, list[Point]] = defaultdict(list)
    for lbl in sch.labels:
        uf.add(lbl.position)
        label_groups[lbl.text].append(lbl.position)
    for points in label_groups.values():
        for p in points[1:]:
            uf.union(points[0], p)

    # 5. Assemble nets.
    classes: dict = uf.classes()      # rep → set of points
    nets: list[Net] = []
    for rep, pts in classes.items():
        connected_pins: list[PinConnection] = []
        net_name = None
        is_power = False
        for p in pts:
            connected_pins.extend(pin_anchors.get(p, []))
            for lbl in sch.labels:
                if lbl.position == p:
                    net_name = lbl.text
                    if lbl.is_power:
                        is_power = True
        if not connected_pins:
            continue   # not a real net (e.g. a stray wire)
        if net_name is None:
            # Auto-generate a stable name from the first pin sorted by ref.
            first = min(connected_pins, key=lambda c: c.designator_dot(sch))
            net_name = f"Net-({first.designator_dot(sch).replace('.', '-')})"
        nets.append(Net(
            id=rep,                                  # rep is a stable point hash
            name=net_name,
            pins=sorted(set(connected_pins),
                        key=lambda c: c.designator_dot(sch)),
            is_power=is_power,
        ))
    return nets


# Minimal union-find over hashable elements. §6.5 has the implementation.
class UnionFind:
    def __init__(self): self.parent = {}
    def add(self, x):
        if x not in self.parent: self.parent[x] = x
    def find(self, x):
```

```
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra != rb: self.parent[ra] = rb
    def classes(self):
        groups = defaultdict(set)
        for x in self.parent:
            groups[self.find(x)].add(x)
        return groups
```

```
```

Watch out for floating-point coordinates. The whole algorithm depends on Point equality, which is integer equality after snap-to-grid. If pins land on grid intersections (they must) and wires are snapped (they must be), this works. If you let any of these be floats, you get netlist instability where moving a wire by a sub-grid amount silently disconnects it. Hypothesis tests in Chapter 17 cover this.

4.7 The KiCad correspondence

When you export to KiCad (Chapter 14), entities map to KiCad as follows. Keep this table open while implementing the exporter.

PCBSmith entity	KiCad equivalent	KiCad file
Schematic	Schematic sheet	.kicad_sch
SymbolInstance	(symbol "Lib:Name" ...)	.kicad_sch
Wire	(wire (pts ...) ...)	.kicad_sch
NetLabel	(label "X" (at ...))	.kicad_sch
Net (computed)	Implicit; KiCad recomputes from wires/labels	—
Board	PCB board	.kicad_pcb
Layer	(layers (0 "F.Cu" signal) ...)	.kicad_pcb
FootprintInstance	(footprint "Lib:Name" ...)	.kicad_pcb
Trace	(segment (start) (end) (width) (layer) (net))	.kicad_pcb
Via	(via (at) (size) (drill) (layers) (net))	.kicad_pcb

Zone	(zone (net N) (layer L) (polygon …))	.kicad_pcb
Outline (Edge.Cuts)	(gr_line … (layer "Edge.Cuts"))	.kicad_pcb

5. Stack, Dependencies, and Environment

5.1 Pinned dependency list

These are the dependencies the application uses. Versions are current as of May 2026. Pin to majors only — minor and patch updates within a major should be transparent. Use the lowest pin compatible with the features you need; do not blindly take latest. Track CVEs.

Package	Version	Purpose	License
python	>= 3.11, < 3.14	Runtime. 3.11 for stability; 3.14 once PySide6 supports it.	PSF
pyside6	>= 6.11, < 7	Qt 6 bindings. Canvas, widgets, signals.	LGPL-3.0
pydantic	>= 2.7, < 3	Data model validation, JSON serialisation.	MIT
anthropic	>= 0.40	Claude API. Required for structured outputs (beta).	MIT
gerbonara	>= 1.6	Modern Gerber/Excellon read/write. Tested across CAD tools.	AGPL-3.0
pyclipper	>= 1.4	Polygon clipping/offsetting (zone fills, clearance).	MIT (Boost-1.0 underlying)
shapely	>= 2.0, < 3	High-level geometry (intersection, contains, distance).	BSD-3-Clause
networkx	>= 3.2	Graph algorithms (netlist, MST for ratsnest, routing aids).	BSD-3-Clause
numpy	>= 1.26	Coordinate maths in tight loops.	BSD-3-Clause
rtree	>= 1.2	R-tree spatial index for hit testing and DRC.	MIT
sexpdata	>= 1.0	S-expression parsing for KiCad library import.	BSD
pytest	>= 8.0	Test runner.	MIT

hypothesis	>= 6.100	Property-based testing for invariants.	MPL-2.0
mypy	>= 1.10	Static type checking. CI-enforced.	MIT
ruff	>= 0.5	Linting + formatting.	MIT
import-linter	>= 2.0	Enforces the layered architecture (§4.1).	BSD-3-Clause
xdg-base-dirs	>= 6.0	Find user config / cache dirs cross-platform.	MPL-2.0

5.2 License compatibility

AGPL note for gerbonara. Gerbonara is AGPL-3.0. If you ship PCBSmith as a closed-source binary or as a hosted service, AGPL means you must publish your source. Two ways forward: (a) publish PCBSmith under AGPL or another GPL-compatible licence, which is the simplest option for an open-source hobby project, or (b) interact with gerbonara only via subprocess CLI, treating it as a separate program (debatable; consult counsel before relying on this for anything commercial). Option (a) is the recommended path. Document your choice in LICENSE.

KiCad's symbol and footprint libraries are CC-BY-SA 4.0. This is content, not code; shipping the library data with the application requires attribution and means anyone who redistributes the library must do so under the same licence. Designs that use parts from the library are not derivatives. The ShareAlike does not propagate to user projects. Add a LIBRARY-LICENSE.md at the root of any bundled libraries.

5.3 Project skeleton

Phase 0 starts here. Generate this layout on day one and never restructure it without a deliberate refactor.

```
pcbsmith/
```

```
├── pyproject.toml
├── README.md
├── LICENSE
├── LIBRARY-LICENSE.md          # attribution for KiCad libs when bundled
├── src/
│   └── pcbsmith/
│       ├── __init__.py
│       ├── core/               # §4.1 layer 1
│       │   ├── __init__.py
│       │   ├── ids.py
│       │   ├── geom.py
│       │   ├── library.py
│       │   ├── schematic.py
│       │   ├── board.py
│       │   ├── project.py
│       │   └── netops.py
│       ├── services/           # §4.1 layer 2
│       │   ├── __init__.py
│       │   ├── project_io.py
│       │   ├── library_loader.py
│       │   ├── builtin_library.py
│       │   ├── kicad_import.py
│       │   ├── erc.py
│       │   ├── drc.py
│       │   ├── freerouting.py
│       │   ├── llm/
│       │   │   ├── ir_schema.py
│       │   │   ├── validator.py
│       │   │   ├── prompt.py
│       │   │   └── client.py
│       │   └── exporters/
│       │       ├── gerber.py
│       │       ├── excellon.py
│       │       ├── svg.py
│       │       ├── pdf.py
│       │       ├── bom.py
│       │       ├── pnp.py
│       │       ├── kicad.py
│       │       └── dsn.py
│       ├── ui/                 # §4.1 layer 3
│       │   ├── __init__.py
│       │   ├── main_window.py
│       │   ├── schematic_canvas.py
│       │   ├── pcb_canvas.py
│       │   ├── prompt_panel.py
│       │   ├── library_palette.py
│       │   ├── project_tree.py
│       │   ├── inspector.py
```

```
│       │   ├── layer_panel.py
│       │   ├── console.py
│       │   ├── dialogs/
│       │   └── graphics/        # custom QGraphicsItem subclasses
│       └── cli.py
├── tests/
│   ├── unit/
│   │   ├── core/
│   │   ├── services/
│   │   └── ui/              # only smoke tests; UI tested via integration
│   ├── integration/
│   ├── fixtures/            # sample projects, KiCad files
│   └── golden/              # snapshot outputs for export tests
├── scripts/
│   ├── download_kicad_libs.py
│   └── verify_gerber.sh     # round-trip a fixture, view in gerbv
└── docs/
    ├── architecture.md      # short version of §4
    ├── contributing.md
    └── ADR/                 # architecture decision records
```

5.4 pyproject.toml starter

Drop this in at project root. Adjust author and URL.

```
[build-system]
```

```
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "pcbsmith"
version = "0.0.1"
description = "Prompt-driven PCB design application"
authors = [{ name = "Your Name" }]
readme = "README.md"
license = { file = "LICENSE" }
requires-python = ">=3.11,<3.14"
dependencies = [
    "PySide6 >=6.11,<7",
    "pydantic >=2.7,<3",
    "anthropic >=0.40",
    "gerbonara >=1.6",
    "pyclipper >=1.4",
    "shapely >=2.0,<3",
    "networkx >=3.2",
    "numpy >=1.26",
    "Rtree >=1.2",
    "sexpdata >=1.0",
    "xdg-base-dirs >=6.0",
]

[project.optional-dependencies]
dev = [
    "pytest >=8.0",
    "pytest-qt >=4.4",
    "hypothesis >=6.100",
    "mypy >=1.10",
    "ruff >=0.5",
    "import-linter >=2.0",
]

[project.scripts]
pcbsmith     = "pcbsmith.ui.main_window:main"
pcbsmith-cli = "pcbsmith.cli:main"

[tool.ruff]
line-length = 100
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "W", "I", "UP", "B", "SIM", "RET"]

[tool.mypy]
strict = true
```

```
files = ["src/pcbsmith"]
plugins = ["pydantic.mypy"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-ra --strict-markers"

[tool.importlinter]
root_packages = ["pcbsmith"]

[[tool.importlinter.contracts]]
name = "core has no external deps"
type = "forbidden"
source_modules = ["pcbsmith.core"]
forbidden_modules = [
    "pcbsmith.services",
    "pcbsmith.ui",
    "PySide6",
]

[[tool.importlinter.contracts]]
name = "services do not import ui"
type = "forbidden"
source_modules = ["pcbsmith.services"]
forbidden_modules = ["pcbsmith.ui"]
```

```
```

5.5 Bundling KiCad libraries

The KiCad libraries are large (≈ 800 MB unpacked for v9). Do not vendor them in the main repo. Provide a script that fetches them at install time. This script is scripts/download_kicad_libs.py. It clones the symbol and footprint repositories from GitLab into the user's data dir.

```
# scripts/download_kicad_libs.py
```

```
"""Fetch KiCad 9 symbol and footprint libraries into the user's data dir.

Usage:
    python -m scripts.download_kicad_libs
"""
from __future__ import annotations
import subprocess, sys
from pathlib import Path
from xdg_base_dirs import xdg_data_home

REPOS = [
    ("kicad-symbols",   "https://gitlab.com/kicad/libraries/kicad-symbols.git"),
    ("kicad-footprints","https://gitlab.com/kicad/libraries/kicad-
footprints.git"),
]
TAG = "9.0.0"  # match your KiCad version; update over time

def main() -> None:
    base = xdg_data_home() / "pcbsmith" / "kicad-libs"
    base.mkdir(parents=True, exist_ok=True)
    for name, url in REPOS:
        target = base / name
        if target.exists():
            print(f"{name} already present at {target}; pulling latest tag
{TAG}")
            subprocess.run(["git", "-C", str(target), "fetch", "--tags"],
check=True)
            subprocess.run(["git", "-C", str(target), "checkout", TAG],
check=True)
        else:
            print(f"Cloning {url} → {target}")
            subprocess.run(["git", "clone", "--depth", "1",
                            "--branch", TAG, url, str(target)], check=True)
    print(f"\nLibraries installed under {base}")
    print("Add this path to your project's libraries[] in project.pcbsmith")

if __name__ == "__main__":
    sys.exit(main())
```

```
```

5.6 FreeRouting

FreeRouting is a Java application. Two delivery modes are practical:

- **Bundled .jar.** Download freerouting-2.x.x.jar from github.com/freerouting/freerouting/releases. Ship it (or download it on first use), require the user to have a Java 17+ JRE, run it via subprocess. This is what KiCad does.
- **Docker.** ghcr.io/freerouting/freerouting publishes multi-arch images. Useful for headless / CI. Heavier dependency on the user side; not recommended as the default.

Default to the .jar approach. Provide a service in services/freerouting.py that discovers a JRE on PATH, locates the .jar in the user's data dir, and runs:

```
java -jar freerouting.jar -de input.dsn -do output.ses -mp 100 \
     -mt 4 -dr passes.json -dl
```

Chapter 15 covers the integration in detail.

5.7 Anthropic API

The LLM pipeline uses Anthropic's Messages API with the Structured Outputs beta header. Add the env var ANTHROPIC_API_KEY (do not commit secrets). The minimum SDK version is 0.40, which exposes client.beta.messages.parse() with native pydantic support.

```
# services/llm/client.py
```

```
from __future__ import annotations
from typing import Type, TypeVar
from anthropic import Anthropic
from pydantic import BaseModel

T = TypeVar("T", bound=BaseModel)

DEFAULT_MODEL = "claude-opus-4-7"
BETA_HEADER = "structured-outputs-2025-11-13"

class LLMClient:
    def __init__(self, api_key: str | None = None, model: str = DEFAULT_MODEL):
        self._client = Anthropic(api_key=api_key)
        self._model = model

    def parse(self, system: str, user: str, schema: Type[T], max_tokens: int =
4096) -> T:
        """One-shot structured call. Returns a validated instance of `schema` or
raises."""
        resp = self._client.beta.messages.parse(
            model=self._model,
            max_tokens=max_tokens,
            betas=[BETA_HEADER],
            system=system,
            messages=[{"role": "user", "content": user}],
            response_model=schema,
        )
        return resp.parsed_output
```

```
```

Why structured outputs over free-form JSON. With structured outputs the model literally cannot emit tokens that violate your pydantic schema; the API performs constrained decoding using a compiled grammar from your schema. You get the same JSON validation, but with no parse failures and no retry logic. Without structured outputs, expect to write retry/repair code for malformed JSON. Use structured outputs.

5.8 Local model fallback (optional)

For privacy or cost reasons, the user may prefer a local model. Plan for it from Phase 3 by abstracting the LLMClient behind a Protocol. A llama.cpp / Ollama backend trades constrained decoding for a JSON-mode prompt and a re-prompt-on-validation-failure loop, but the IR is the same.

```
# services/llm/client.py (continued)
```

```
from typing import Protocol

class LLMBackend(Protocol):
    def parse(self, system: str, user: str, schema: Type[T],
              max_tokens: int = ...) -> T: ...

class OllamaBackend:
    """Ollama with JSON mode + pydantic validation + retry.

    Trade-offs vs Anthropic structured outputs:
      - Validation can fail (model produces bad JSON). Retry up to N times.
      - Latency depends on the local hardware.
      - No cost.
    """
    def __init__(self, model: str = "qwen2.5-coder:14b", url: str =
"http://localhost:11434"):
        self.model, self.url = model, url
    def parse(self, system, user, schema, max_tokens=4096):
        ...   # see Chapter 8 §8.6
```

```
```

5.9 Continuous integration

GitHub Actions or equivalent. The workflow must pass on every PR. Failure to pass = no merge.

```
# .github/workflows/ci.yml
```

```
name: ci
on: [push, pull_request]

jobs:
  lint-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -e ".[dev]"
      - run: ruff check .
      - run: ruff format --check .
      - run: mypy
      - run: lint-imports
      - run: pytest -x --cov=pcbsmith --cov-report=term-missing
```

```
```

Add a separate workflow for export-roundtrip tests that downloads gerbv and verifies Gerbers from a fixture project parse without errors. Run nightly to catch regressions in gerbonara updates.

6. Phase 0 — Foundation

This is where the project starts. No GUI, no LLM, no PCB editor. The deliverable at the end of this phase is a CLI tool that can: create a new project, open a project, validate it, generate a netlist from a schematic, and save the project back to disk. If you cannot do this from the command line, the rest of the project will be built on sand.

Phase 0 milestone. A pytest test that constructs a small schematic in code (3 resistors, 1 LED, a battery, and a few wires), invokes derive_netlist(), and asserts the resulting Net list matches an expected dictionary. Plus a CLI invocation that reads a fixture project and prints the netlist.

6.1 Tasks for Phase 0, in order

1. Initialise the repository: pyproject.toml, ruff/mypy/pytest config, CI, Git hooks.
2. Implement core/ids.py and core/geom.py with full unit tests.
3. Implement core/library.py: Symbol, Pin, Footprint, Pad pydantic models.
4. Implement core/schematic.py: Schematic, SymbolInstance, Wire, Junction, NetLabel, NoConnect.
5. Implement core/board.py: Board, Layer, FootprintInstance, Trace, Via, Zone, DesignRules.
6. Implement core/project.py: Project root and LibraryRef.
7. Implement core/netops.py: derive_netlist() with union-find.
8. Implement services/builtin_library.py with 25 hard-coded parts (§6.6).
9. Implement services/project_io.py: load/save/new/validate.
10. Implement services/erc.py: minimal ERC (unconnected pins, output conflicts).
11. Implement ui/cli.py with argparse subcommands: new, info, netlist, erc, validate.
12. Write fixture projects under tests/fixtures and integration tests that round-trip them.

6.2 Initialising the repository

Most of this is in Chapter 5. Specific to Phase 0: ensure the import-linter contracts are in place from day one and the CI runs them. Catching a bad import early is much easier than ripping it out later.

```
# .pre-commit-config.yaml (recommended)
```

```
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.5.0
    hooks:
      - id: ruff
      - id: ruff-format
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.10.0
    hooks:
      - id: mypy
        additional_dependencies: [pydantic, types-PyYAML]
```

```
```

6.3 Tests live next to the code

Mirror the package layout in tests/. Co-locate fixtures with the tests that use them. Avoid one giant fixture file.

```
tests/
```

```
├── conftest.py                          # shared fixtures
├── fixtures/
│   ├── 555_blinker_project/             # full project fixture
│   └── simple_voltage_divider.json      # serialised schematic
├── unit/
│   ├── core/
│   │   ├── test_geom.py
│   │   ├── test_ids.py
│   │   ├── test_schematic_models.py
│   │   ├── test_board_models.py
│   │   └── test_netops.py              # netlist derivation
│   └── services/
│       ├── test_project_io.py           # save/load round-trip
│       ├── test_builtin_library.py
│       └── test_erc.py
└── integration/
    └── test_cli.py                      # subprocess against the CLI
```

```
```

6.4 Implementing the geometry primitives

Tests first. Always tests first in core/. The cycle is: write a property in plain English, write a Hypothesis test for it, then write the function. The result is geometry code you trust. The tests below are the contract; expect to write more than the function.

```
# tests/unit/core/test_geom.py
```

```
from hypothesis import given, strategies as st
from pcbsmith.core.geom import Point, Vec, Box, snap, mm_to_nm, nm_to_mm

def test_point_from_mm_round_trips():
    assert Point.from_mm(12.7, 0.0) == Point(12_700_000, 0)

def test_mm_nm_lossy_for_sub_nm():
    """Sub-nm fractions are rounded; that's by design."""
    assert mm_to_nm(0.0000005) == 1   # 0.5 nm rounded up
    assert mm_to_nm(0.0000004) == 0

@given(x=st.integers(-10**18, 10**18), y=st.integers(-10**18, 10**18),
       dx=st.integers(-10**18, 10**18), dy=st.integers(-10**18, 10**18))
def test_point_add_sub_inverse(x, y, dx, dy):
    p, v = Point(x, y), Vec(dx, dy)
    assert (p + v) - p == v

@given(x=st.integers(0, 10**12), y=st.integers(0, 10**12))
def test_snap_idempotent(x, y):
    GRID = 1_270_000
    p = Point(x, y)
    once  = snap(p, GRID)
    twice = snap(once, GRID)
    assert once == twice
    assert once.x % GRID == 0 and once.y % GRID == 0

def test_box_contains():
    b = Box(0, 0, 1000, 1000)
    assert b.contains(Point(500, 500))
    assert b.contains(Point(0, 0))
    assert not b.contains(Point(1001, 0))

def test_box_intersects_disjoint():
    a = Box(0, 0, 100, 100)
    b = Box(200, 0, 300, 100)
    assert not a.intersects(b)

def test_box_intersects_touching():
    """Touching at an edge counts as intersecting (closed boxes)."""
    a = Box(0, 0, 100, 100)
    b = Box(100, 0, 200, 100)
    assert a.intersects(b)
```

```
```

6.5 The union-find for netlist derivation

Plain integer-keyed union-find; nothing fancy. Pretty much every other part of the engine relies on it being correct, so test it directly even though derive_netlist also exercises it.

```
# core/netops.py (UnionFind)
```

```
class UnionFind:
    """Disjoint-set with path compression and union by rank.

    Hashable elements only. O(α(n)) per operation, effectively constant.
    """
    def __init__(self) -> None:
        self.parent: dict = {}
        self.rank:   dict = {}

    def add(self, x) -> None:
        if x not in self.parent:
            self.parent[x] = x
            self.rank[x] = 0

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]   # halving
            x = self.parent[x]
        return x

    def union(self, a, b) -> None:
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1

    def classes(self) -> dict:
        from collections import defaultdict
        groups: dict = defaultdict(set)
        for x in self.parent:
            groups[self.find(x)].add(x)
        return groups
```

```
```

```
# tests/unit/core/test_netops.py (UnionFind tests)
```

```
from pcbsmith.core.netops import UnionFind
import pytest

def test_singleton():
    uf = UnionFind(); uf.add(1)
    assert uf.find(1) == 1

def test_basic_union():
    uf = UnionFind()
    for i in range(5): uf.add(i)
    uf.union(0, 1); uf.union(2, 3); uf.union(1, 3)
    assert uf.find(0) == uf.find(2) == uf.find(3)
    assert uf.find(4) != uf.find(0)

def test_find_unknown_raises():
    uf = UnionFind()
    with pytest.raises(KeyError):
        uf.find("nope")

def test_union_idempotent():
    uf = UnionFind()
    for i in range(3): uf.add(i)
    uf.union(0, 1); rep = uf.find(0)
    uf.union(0, 1)   # again
    assert uf.find(0) == uf.find(1) == rep
```

```
```

6.6 The hard-coded built-in library (Phase 0 only)

Phase 0 needs a small library so that tests can construct schematics. About 25 parts is the right number: enough to build a non-trivial test circuit, small enough to maintain by hand. Do not grow this beyond 30 parts; once you can import KiCad libraries (Phase 4), this list shrinks back to a handful for testing.

Symbol	Ref prefix	Pins	Default footprint
R (resistor)	R	2	R_0603
C (capacitor)	C	2	C_0603
L (inductor)	L	2	L_0805
LED	D	2 (A, K)	LED_0603
Diode	D	2 (A, K)	D_SOD-123
Zener	D	2	D_SOD-123

Schottky	D	2	D_SOD-123
NPN BJT	Q	3 (B, C, E)	SOT-23
PNP BJT	Q	3 (B, C, E)	SOT-23
N-MOSFET	Q	3 (G, D, S)	SOT-23
P-MOSFET	Q	3 (G, D, S)	SOT-23
Op-amp (single)	U	5 (V+, V-, IN+, IN-, OUT)	SOIC-8
Voltage regulator (3-pin LDO)	U	3 (VIN, GND, VOUT)	SOT-223
Crystal	Y	2	Crystal_HC49
Push button (momentary)	SW	2	SW_TH_6mm
Slide switch (SPDT)	SW	3	SW_SPDT_PCB
Battery (cell)	BT	2	BatteryHolder_AA
Pin header 1x4	J	4	PinHeader_1x4_2.54mm
Pin header 1x10	J	10	PinHeader_1x10_2.54mm
USB-C receptacle (16-pin)	J	16	USB_C_Receptacle
3.5 mm jack	J	3	Jack_3.5mm_PJ307
Mounting hole M3	H	1 (NPTH)	MountingHole_M3
Test point	TP	1	TestPoint_1mm
GND symbol (power port)	#PWR	1	—
+3V3, +5V, +12V (power ports)	#PWR	1 each	—

6.6.1 Implementation pattern

Each part is a Symbol pydantic instance. Build them in code, not in JSON, so the static type-checker catches mistakes. Below is the resistor and an op-amp; copy the pattern for the rest.

```
# services/builtin_library.py
```

```
from __future__ import annotations
from pcbsmith.core.geom import Point
from pcbsmith.core.library import (
    Symbol, Pin, PinDirection, PinElectricalType,
    Footprint, Pad, PadType, PadShape,
)

def _MIL(n: float) -> int: return int(n * 25_400)
def _MM(n: float)  -> int: return int(n * 1_000_000)

# ---------------------------------------------------------------------------
# Resistor
# ---------------------------------------------------------------------------
SYM_RESISTOR = Symbol(
    id="stdlib:R",
    name="Resistor",
    description="Generic resistor (zigzag body)",
    keywords=["R", "resistor"],
    reference_prefix="R",
    pins=[
        Pin(number="1", name="~",
            position=Point(0, _MIL(100)), direction=PinDirection.UP,
            length_nm=_MIL(100), electrical_type=PinElectricalType.PASSIVE),
        Pin(number="2", name="~",
            position=Point(0, -_MIL(100)), direction=PinDirection.DOWN,
            length_nm=_MIL(100), electrical_type=PinElectricalType.PASSIVE),
    ],
    body_polylines=[[
        Point(-_MIL(40), _MIL(50)),
        Point( _MIL(40), _MIL(50)),
        Point( _MIL(40),-_MIL(50)),
        Point(-_MIL(40),-_MIL(50)),
        Point(-_MIL(40), _MIL(50)),
    ]],
    default_footprint="stdlib:R_0603",
)

# ---------------------------------------------------------------------------
# 0603 resistor footprint  (1.6 × 0.8 mm body, two SMT pads)
# ---------------------------------------------------------------------------
FP_R_0603 = Footprint(
    id="stdlib:R_0603",
    name="R_0603",
    description="0603 (1608 metric) two-pad SMT resistor",
    keywords=["resistor", "smt", "0603"],
    pads=[
        Pad(number="1", type=PadType.SMT_TOP, shape=PadShape.RECT,
            position=Point(-_MM(0.825), 0),
```

```
            size_x_nm=_MM(0.95), size_y_nm=_MM(1.00),
            layers=["F.Cu", "F.Mask", "F.Paste"]),
        Pad(number="2", type=PadType.SMT_TOP, shape=PadShape.RECT,
            position=Point(_MM(0.825), 0),
            size_x_nm=_MM(0.95), size_y_nm=_MM(1.00),
            layers=["F.Cu", "F.Mask", "F.Paste"]),
    ],
    silkscreen_polylines=[
        [Point(-_MM(1.55),  _MM(0.72)), Point( _MM(1.55),  _MM(0.72))],
        [Point(-_MM(1.55), -_MM(0.72)), Point( _MM(1.55), -_MM(0.72))],
    ],
    courtyard_polylines=[
        [Point(-_MM(1.65),  _MM(0.85)), Point( _MM(1.65),  _MM(0.85)),
         Point( _MM(1.65), -_MM(0.85)), Point(-_MM(1.65), -_MM(0.85)),
         Point(-_MM(1.65),  _MM(0.85))],
    ],
    height_nm=_MM(0.45),
)

# Registry pattern: dict[str, Symbol/Footprint] populated by all module-level
definitions.
_SYMBOLS: dict[str, Symbol] = {SYM_RESISTOR.id: SYM_RESISTOR, ...}
_FOOTPRINTS: dict[str, Footprint] = {FP_R_0603.id: FP_R_0603, ...}

def get_symbol(symbol_id: str) -> Symbol:
    if symbol_id not in _SYMBOLS:
        raise KeyError(f"Unknown built-in symbol: {symbol_id!r}")
    return _SYMBOLS[symbol_id]

def get_footprint(fp_id: str) -> Footprint:
    if fp_id not in _FOOTPRINTS:
        raise KeyError(f"Unknown built-in footprint: {fp_id!r}")
    return _FOOTPRINTS[fp_id]
```

```
```

6.7 Project I/O

A project is a folder. Loading reads project.pcbsmith plus every schematic and board the project file references. Saving writes them all back atomically (write to .tmp, fsync, rename). Atomic writes prevent half-written files when the user's editor crashes.

```
# services/project_io.py
```

```
from __future__ import annotations
import json, os, tempfile
from pathlib import Path
from datetime import datetime, timezone
from pcbsmith.core.project import Project, LibraryRef
from pcbsmith.core.schematic import Schematic
from pcbsmith.core.board import Board
from pcbsmith.core.ids import new_id


SUPPORTED_SCHEMA = {1}


def new_project(path: Path, name: str, author: str = "") -> Project:
    """Create a fresh project skeleton at `path` and return the model."""
    path = Path(path)
    if path.exists() and any(path.iterdir()):
        raise FileExistsError(f"{path} is not empty")
    path.mkdir(parents=True, exist_ok=True)
    (path / "schematics").mkdir()
    (path / "boards").mkdir()
    (path / "library/symbols").mkdir(parents=True)
    (path / "library/footprints").mkdir(parents=True)
    (path / "prompts").mkdir()
    (path / "output").mkdir()

    now = datetime.now(timezone.utc).isoformat()
    project = Project(
        id=new_id(),
        name=name,
        author=author,
        libraries=[LibraryRef(name="stdlib", path="builtin://stdlib")],
        created_iso=now,
        modified_iso=now,
    )
    save(project, path)
    return project


def save(project: Project, path: Path) -> None:
    """Atomically write project + child documents to `path`."""
    path = Path(path)
    project.modified_iso = datetime.now(timezone.utc).isoformat()
    project_minus_kids = project.model_dump(exclude={"schematics", "boards"})
    project_minus_kids["schematic_files"] = [s.name for s in project.schematics]
    project_minus_kids["board_files"]     = [bd.name for bd in project.boards]

    _atomic_write_json(path / "project.pcbsmith", project_minus_kids)
```

```
    for s in project.schematics:
        _atomic_write_json(path / "schematics" / f"{s.name}.sch.json",
                           s.model_dump())
    for bd in project.boards:
        _atomic_write_json(path / "boards" / f"{bd.name}.brd.json",
                           bd.model_dump())


def load(path: Path) -> Project:
    path = Path(path)
    raw = json.loads((path / "project.pcbsmith").read_text())
    sv = raw.get("schema_version", 1)
    if sv not in SUPPORTED_SCHEMA:
        raise ValueError(f"Unsupported schema_version {sv};
supported={SUPPORTED_SCHEMA}")
    schematic_files = raw.pop("schematic_files", [])
    board_files     = raw.pop("board_files", [])

    schematics: list[Schematic] = []
    for fname in schematic_files:
        sj = json.loads((path / "schematics" / f"{fname}.sch.json").read_text())
        schematics.append(Schematic.model_validate(sj))
    boards: list[Board] = []
    for fname in board_files:
        bj = json.loads((path / "boards" / f"{fname}.brd.json").read_text())
        boards.append(Board.model_validate(bj))

    raw["schematics"] = [s.model_dump() for s in schematics]
    raw["boards"] = [bd.model_dump() for bd in boards]
    return Project.model_validate(raw)


def _atomic_write_json(path: Path, payload) -> None:
    fd, tmp = tempfile.mkstemp(prefix=path.name, dir=path.parent)
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as fh:
            json.dump(payload, fh, indent=2, sort_keys=True, default=str)
            fh.flush()
            os.fsync(fh.fileno())
        os.replace(tmp, path)
    except Exception:
        try: os.unlink(tmp)
        except OSError: pass
        raise
```

6.8 Minimal ERC for Phase 0

The full ERC engine lives in Chapter 11. Phase 0 ships a small subset that can flag the obvious mistakes — enough to give the user feedback while you build the schematic editor in Phase 1.

```
# services/erc.py (Phase 0 subset)
```

```
from __future__ import annotations
from dataclasses import dataclass
from enum import Enum
from pcbsmith.core.schematic import Schematic
from pcbsmith.core.library import Symbol, PinElectricalType
from pcbsmith.core.netops import derive_netlist


class Severity(str, Enum):
    INFO = "info"; WARNING = "warning"; ERROR = "error"


@dataclass(frozen=True)
class ErcIssue:
    severity: Severity
    code: str           # "E001", machine-readable
    message: str
    where: str          # "R1.2", "Net VCC_3V3", etc.


def run_erc(sch: Schematic, symbols: dict[str, Symbol]) -> list[ErcIssue]:
    issues: list[ErcIssue] = []
    issues.extend(_check_duplicate_refs(sch))
    nets = derive_netlist(sch, symbols)
    issues.extend(_check_unconnected_pins(sch, symbols, nets))
    issues.extend(_check_output_conflicts(sch, symbols, nets))
    issues.extend(_check_power_in_unconnected(sch, symbols, nets))
    return issues
```

```

```

6.9 The CLI

argparse subcommands: new, info, netlist, erc, validate. Keep this small. The CLI is a test harness more than a product feature.

```
# ui/cli.py
```

```
from __future__ import annotations
import argparse, json, sys
from pathlib import Path
from pcbsmith.services.project_io import new_project, load
from pcbsmith.services.erc import run_erc, Severity
from pcbsmith.services.builtin_library import _SYMBOLS
from pcbsmith.core.netops import derive_netlist


def cmd_new(args):
    new_project(Path(args.path), args.name, author=args.author or "")
    print(f"Created project '{args.name}' at {args.path}")
    return 0

def cmd_info(args):
    p = load(Path(args.path))
    print(f"{p.name}  ({p.id})")
    print(f"  schematics: {len(p.schematics)}")
    print(f"  boards:     {len(p.boards)}")
    return 0

def cmd_netlist(args):
    p = load(Path(args.path))
    sch = p.schematics[0]
    nets = derive_netlist(sch, _SYMBOLS)
    out = [{"name": n.name, "pins":
            [pc.designator_dot(sch) for pc in n.pins]} for n in nets]
    print(json.dumps(out, indent=2))
    return 0

def cmd_erc(args):
    p = load(Path(args.path))
    sch = p.schematics[0]
    issues = run_erc(sch, _SYMBOLS)
    for i in issues:
        print(f"[{i.severity.value}] {i.code} {i.where}: {i.message}")
    return 1 if any(i.severity == Severity.ERROR for i in issues) else 0


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(prog="pcbsmith")
    sub = parser.add_subparsers(dest="cmd", required=True)

    n = sub.add_parser("new");      n.add_argument("path");
n.add_argument("--name", required=True); n.add_argument("--author")
    i = sub.add_parser("info");     i.add_argument("path")
    nl = sub.add_parser("netlist"); nl.add_argument("path")
    e = sub.add_parser("erc");      e.add_argument("path")
```

```
    n.set_defaults(func=cmd_new)
    i.set_defaults(func=cmd_info)
    nl.set_defaults(func=cmd_netlist)
    e.set_defaults(func=cmd_erc)

    args = parser.parse_args(argv)
    return args.func(args)
```

```
```

6.10 The Phase 0 acceptance test

This integration test exercises the whole stack from CLI down. It is the gate you must pass before declaring Phase 0 complete.

```
# tests/integration/test_phase0_acceptance.py
```

```
import json, subprocess, sys
from pathlib import Path
from pcbsmith.core.geom import Point
from pcbsmith.core.schematic import Schematic, SymbolInstance, Wire, NetLabel
from pcbsmith.services.project_io import save, load, new_project
from pcbsmith.core.ids import new_id


def test_voltage_divider_netlist(tmp_path):
    project_path = tmp_path / "divider"
    new_project(project_path, name="divider")
    p = load(project_path)

    r1 = SymbolInstance(id=new_id(), symbol_id="stdlib:R", reference="R1",
                        value="10k", position=Point.from_mm(0, 0))
    r2 = SymbolInstance(id=new_id(), symbol_id="stdlib:R", reference="R2",
                        value="10k", position=Point.from_mm(0, -10))
    sch = Schematic(id=new_id(), name="main",
                    symbols=[r1, r2],
                    wires=[
                        Wire(id=new_id(),
                             points=[Point.from_mm(0, -2.54),
                                     Point.from_mm(0, -7.46)]),
                    ],
                    labels=[
                        NetLabel(id=new_id(), position=Point.from_mm(0,  2.54),
                                 text="VCC", is_power=True),
                        NetLabel(id=new_id(), position=Point.from_mm(0, -12.7),
                                 text="GND", is_power=True),
                        NetLabel(id=new_id(), position=Point.from_mm(0, -5),
                                 text="OUT"),
                    ])
    p.schematics = [sch]
    save(p, project_path)

    out = subprocess.run(
        [sys.executable, "-m", "pcbsmith.ui.cli", "netlist", str(project_path)],
        capture_output=True, text=True, check=True,
    ).stdout
    nets = {n["name"]: set(n["pins"]) for n in json.loads(out)}
    assert nets["VCC"] == {"R1.1"}
    assert nets["OUT"] == {"R1.2", "R2.1"}
    assert nets["GND"] == {"R2.2"}
```

6.11 Common Phase 0 mistakes to avoid

- **Storing the netlist on disk.** Don't. The netlist is computed. Storing it produces drift bugs.
- **Floating-point coordinates anywhere in core.** Use int nm. Float comes in only at the UI/render boundary.
- **Mutating frozen pydantic models.** Library entities (Symbol, Footprint, Pin, Pad) are frozen. Instances are mutable. The library is a template.
- **Skipping atomic writes.** A power loss mid-save corrupts the project. fsync + rename is cheap insurance.
- **Inlining schematics into the project file.** They diff terribly when inlined. Keep them as separate files referenced by name.

7. Phase 1 — Schematic Editor MVP

Phase 1 builds the schematic editor: a PySide6 window with a canvas where the user places symbols, draws wires, adds labels, and runs ERC. By the end of this phase the user can construct the voltage divider from the Phase 0 acceptance test by hand, save it, close the application, reopen it, and continue editing.

Phase 1 milestone. Open the application from a freshly cloned repo, click File → New, place two resistors, draw a wire between them, save, quit, reopen, and see the schematic come back exactly. ERC button reports issues. Library palette shows the 25 built-in parts.

7.1 PySide6 architecture for the canvas

The Qt Graphics View framework is built for this kind of editor. There are three layers you must understand before writing any UI code:

- **QGraphicsScene.** A non-visual coordinate space holding QGraphicsItems. Like a model in MVC. Stores items in an internal BSP tree for fast lookup. Supports any number of views.
- **QGraphicsView.** A widget that renders some region of a scene with a particular transform (zoom, pan). You can have multiple views on one scene; we won't, but the option is free.
- **QGraphicsItem.** A renderable thing in the scene. We subclass it for symbols, wires, junctions, labels. Each item has its own paint() and shape() methods.

Coordinate dance. The scene uses pixel-ish floats. Our data model uses int nanometres. We pick a fixed scale (e.g. 1 scene-unit = 1 mil = 25,400 nm) and convert at the boundary between QGraphicsItem and the data model. Document the conversion in one place (ui/graphics/units.py) and never re-derive it inline.

7.2 Module breakdown

```
ui/
```

```
├── main_window.py                # QMainWindow, menu, toolbar, status bar
├── schematic_canvas.py           # SchematicScene, SchematicView, tool dispatch
├── library_palette.py            # left dock: searchable symbol list
├── project_tree.py               # left dock: schematics + boards in the
project
├── inspector.py                  # right dock: properties of selected item
├── console.py                    # bottom dock: ERC issues, LLM history, log
├── dialogs/
│   ├── new_project.py
│   ├── footprint_assignment.py
│   └── about.py
└── graphics/
    ├── units.py                  # scene<->nm conversion, GRID constants
    ├── styles.py                 # pens, brushes, colors per layer/role
    ├── symbol_item.py            # QGraphicsItem subclass for SymbolInstance
    ├── wire_item.py
    ├── junction_item.py
    ├── label_item.py
    └── grid_background.py
```

7.3 Tasks for Phase 1, in order

13. Set up main_window.py with the standard four-dock layout (left/right/bottom + central).
14. Implement units.py and grid_background.py; the canvas should show a 100-mil grid.
15. Implement SymbolItem so a single symbol can be placed and selected.
16. Implement the library palette — list of 25 parts with search and drag-and-drop.
17. Implement the place tool: drag from palette, hover preview, click to drop, ESC to cancel.
18. Implement WireItem with the wire-drawing tool: click to start, click for corners, double-click or ESC to finish.
19. Implement JunctionItem; auto-add at T-intersections of wires; allow manual placement.
20. Implement LabelItem; allow placement on wire endpoints; ENTER to commit text.
21. Implement select / move / rotate / delete with QUndoStack so every action is undoable.
22. Implement the inspector dock that shows the selected item's properties.
23. Wire ERC button → services/erc.py → console dock items with double-click-to-locate.
24. Wire File → Save / Open through services.project_io.
25. Acceptance test: scripted Qt test (pytest-qt) reproducing the voltage divider milestone.

7.4 The main window

Standard Qt main window. Menu bar, toolbar, central widget (the canvas), four dock widgets. Use a QStackedWidget in the centre so you can swap to the PCB canvas in Phase 2 without restructuring.

```
# ui/main_window.py
```

```
from __future__ import annotations
from pathlib import Path
from PySide6.QtWidgets import (
    QApplication, QMainWindow, QStackedWidget, QDockWidget, QMessageBox,
    QFileDialog, QToolBar,
)
from PySide6.QtGui import QAction, QKeySequence
from PySide6.QtCore import Qt
from pcbsmith.services.project_io import new_project, load, save
from pcbsmith.services.erc import run_erc
from pcbsmith.services.builtin_library import _SYMBOLS

from .schematic_canvas import SchematicCanvas
from .library_palette  import LibraryPalette
from .project_tree     import ProjectTree
from .inspector        import Inspector
from .console          import Console


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("PCBSmith")
        self.resize(1500, 950)
        self.project = None       # core.project.Project
        self.project_path = None  # Path

        self.canvas_stack = QStackedWidget()
        self.schematic_canvas = SchematicCanvas(self)
        self.canvas_stack.addWidget(self.schematic_canvas)
        # Phase 2 will addWidget(self.pcb_canvas)
        self.setCentralWidget(self.canvas_stack)

        self._setup_docks()
        self._setup_menu()
        self._setup_toolbar()

    def _setup_docks(self):
        self.lib_palette = LibraryPalette(self)
        self.project_tree = ProjectTree(self)
        self.inspector    = Inspector(self)
        self.console      = Console(self)

        for w in (self.lib_palette, self.project_tree):
            self.addDockWidget(Qt.LeftDockWidgetArea, w)
        self.tabifyDockWidget(self.project_tree, self.lib_palette)
        self.lib_palette.raise_()
```

```
        self.addDockWidget(Qt.RightDockWidgetArea, self.inspector)
        self.addDockWidget(Qt.BottomDockWidgetArea, self.console)

    def _setup_menu(self):
        bar = self.menuBar()
        # File
        m = bar.addMenu("&File")
        m.addAction(self._mk("&New project…", self.on_new, QKeySequence.New))
        m.addAction(self._mk("&Open project…", self.on_open, QKeySequence.Open))
        m.addAction(self._mk("&Save", self.on_save, QKeySequence.Save))
        m.addSeparator()
        m.addAction(self._mk("E&xit", self.close, QKeySequence.Quit))

        # Edit
        m = bar.addMenu("&Edit")
        undo = self.schematic_canvas.undo_stack.createUndoAction(self, "&Undo")
        undo.setShortcut(QKeySequence.Undo); m.addAction(undo)
        redo = self.schematic_canvas.undo_stack.createRedoAction(self, "&Redo")
        redo.setShortcut(QKeySequence.Redo); m.addAction(redo)

        # Tools
        m = bar.addMenu("&Tools")
        m.addAction(self._mk("Run &ERC", self.on_erc, "F8"))

    def _setup_toolbar(self):
        tb = QToolBar("Main"); self.addToolBar(tb)
        tb.addAction(self._mk("Select", lambda:
self.schematic_canvas.set_tool("select"), "S"))
        tb.addAction(self._mk("Place",  lambda:
self.schematic_canvas.set_tool("place"),  "P"))
        tb.addAction(self._mk("Wire",   lambda:
self.schematic_canvas.set_tool("wire"),   "W"))
        tb.addAction(self._mk("Label",  lambda:
self.schematic_canvas.set_tool("label"),  "L"))
        tb.addAction(self._mk("Junction",lambda:
self.schematic_canvas.set_tool("junction"), "J"))

    def _mk(self, label, slot, shortcut=None):
        a = QAction(label, self)
        a.triggered.connect(slot)
        if shortcut: a.setShortcut(shortcut if isinstance(shortcut, QKeySequence)
else QKeySequence(shortcut))
        return a

    # --------------- slots ---------------
    def on_new(self):
        path = QFileDialog.getExistingDirectory(self, "New project (empty
folder)")
        if not path: return
        self.project = new_project(Path(path), name=Path(path).name)
```

```
        self.project_path = Path(path)
        self._refresh()

    def on_open(self):
        path = QFileDialog.getExistingDirectory(self, "Open project folder")
        if not path: return
        try:
            self.project = load(Path(path))
            self.project_path = Path(path)
            self._refresh()
        except Exception as exc:
            QMessageBox.critical(self, "Open failed", str(exc))

    def on_save(self):
        if not self.project_path:
            return
        # Sync canvas back to model first.
        self.schematic_canvas.commit_to_model()
        save(self.project, self.project_path)
        self.statusBar().showMessage("Saved", 2000)

    def on_erc(self):
        if not (self.project and self.project.schematics): return
        sch = self.project.schematics[0]
        self.schematic_canvas.commit_to_model()
        issues = run_erc(sch, _SYMBOLS)
        self.console.show_erc(issues)

    def _refresh(self):
        if not self.project: return
        self.project_tree.set_project(self.project)
        if self.project.schematics:
            self.schematic_canvas.load_schematic(self.project.schematics[0])


def main():
    app = QApplication([])
    w = MainWindow()
    w.show()
    app.exec()
```

7.5 Units, grid, and styles

Pin all the magic numbers in one module. The codebase will refer to GRID_SCENE and GRID_NM thousands of times; it should be easy to find the source.

```
# ui/graphics/units.py
```

```
"""Single source of truth for scene <-> nanometre conversion."""
from pcbsmith.core.geom import Point

# 1 scene unit = 1 mil = 25_400 nm. Chosen so common schematic spacings
# (50, 100 mil) are integers in the scene. 1 mm = 39.3701 scene units (float ok
in scene).
NM_PER_SCENE = 25_400

def scene_to_nm(sx: float, sy: float) -> Point:
    return Point(int(round(sx * NM_PER_SCENE)),
                 int(round(sy * NM_PER_SCENE)))

def nm_to_scene(p: Point) -> tuple[float, float]:
    return (p.x / NM_PER_SCENE, p.y / NM_PER_SCENE)

# Grid constants in scene units.
GRID_SCHEMATIC_MAJOR = 100   # 100 mil = 2.54 mm
GRID_SCHEMATIC_MINOR = 50    # 50 mil  = 1.27 mm

# Y points DOWN in Qt scene by default. Schematic users expect Y up; we flip
# at the view level (setTransform with negative y-scale).
```

```

```

```
# ui/graphics/styles.py
```

```
from PySide6.QtGui import QPen, QBrush, QColor
from PySide6.QtCore import Qt

# Theme tokens (light theme default).
COLOR_BG          = QColor("#FFFFFF")
COLOR_GRID_MAJOR  = QColor("#D6D6D6")
COLOR_GRID_MINOR  = QColor("#EEEEEE")
COLOR_SYMBOL_BODY = QColor("#992A1A")     # KiCad-ish symbol red
COLOR_PIN         = QColor("#992A1A")
COLOR_WIRE        = QColor("#0F771A")     # KiCad-ish net green
COLOR_LABEL       = QColor("#000000")
COLOR_LABEL_POWER = QColor("#992A1A")
COLOR_JUNCTION    = QColor("#0F771A")
COLOR_SELECTION   = QColor("#FFA000")

PEN_SYMBOL = QPen(COLOR_SYMBOL_BODY, 6.0); PEN_SYMBOL.setCapStyle(Qt.RoundCap)
PEN_PIN    = QPen(COLOR_PIN, 6.0)
PEN_WIRE   = QPen(COLOR_WIRE, 6.0); PEN_WIRE.setCapStyle(Qt.RoundCap);
PEN_WIRE.setJoinStyle(Qt.RoundJoin)
PEN_SELECT = QPen(COLOR_SELECTION, 6.0)
BRUSH_JUNCTION = QBrush(COLOR_JUNCTION)
```

```
```

7.6 Grid background

Custom QGraphicsScene paints the grid in drawBackground. Two-tone (major + minor) lines so the user can place to the major grid easily. Adapt density when zoomed out to avoid moiré.

```
# ui/graphics/grid_background.py
```

```
from PySide6.QtCore import QRectF
from PySide6.QtGui import QPen, QPainter
from .styles import COLOR_GRID_MAJOR, COLOR_GRID_MINOR
from .units import GRID_SCHEMATIC_MAJOR, GRID_SCHEMATIC_MINOR


def draw_grid(painter: QPainter, rect: QRectF, view_scale: float) -> None:
    """Paint major + minor schematic grid in scene units.

    `view_scale` is the current QGraphicsView transform scale; we use it to
    drop the minor grid when zoomed out enough that it would alias.
    """
    show_minor = view_scale > 0.6
    if show_minor:
        pen_minor = QPen(COLOR_GRID_MINOR, 0)
        painter.setPen(pen_minor)
        x = (int(rect.left()) // GRID_SCHEMATIC_MINOR) * GRID_SCHEMATIC_MINOR
        while x < rect.right():
            painter.drawLine(x, rect.top(), x, rect.bottom())
            x += GRID_SCHEMATIC_MINOR
        y = (int(rect.top()) // GRID_SCHEMATIC_MINOR) * GRID_SCHEMATIC_MINOR
        while y < rect.bottom():
            painter.drawLine(rect.left(), y, rect.right(), y)
            y += GRID_SCHEMATIC_MINOR

    pen_major = QPen(COLOR_GRID_MAJOR, 0)
    painter.setPen(pen_major)
    x = (int(rect.left()) // GRID_SCHEMATIC_MAJOR) * GRID_SCHEMATIC_MAJOR
    while x < rect.right():
        painter.drawLine(x, rect.top(), x, rect.bottom())
        x += GRID_SCHEMATIC_MAJOR
    y = (int(rect.top()) // GRID_SCHEMATIC_MAJOR) * GRID_SCHEMATIC_MAJOR
    while y < rect.bottom():
        painter.drawLine(rect.left(), y, rect.right(), y)
        y += GRID_SCHEMATIC_MAJOR
```

```
```

7.7 SymbolItem

A QGraphicsItem subclass that knows how to draw a Symbol from the library at a given instance position and rotation. It owns a back-reference to the SymbolInstance so the model can be updated when the user moves it.

```
# ui/graphics/symbol_item.py
```

```
from __future__ import annotations
from PySide6.QtCore import QRectF, QPointF, Qt
from PySide6.QtGui import QPainter, QPainterPath, QFont, QFontMetricsF
from PySide6.QtWidgets import QGraphicsItem
from pcbsmith.core.library import Symbol, Pin
from pcbsmith.core.schematic import SymbolInstance
from .units import nm_to_scene, scene_to_nm
from .styles import PEN_SYMBOL, PEN_PIN, PEN_SELECT


class SymbolItem(QGraphicsItem):
    """Renders a Symbol at the position/rotation of a SymbolInstance."""
    def __init__(self, symbol: Symbol, instance: SymbolInstance, parent=None):
        super().__init__(parent)
        self.symbol = symbol
        self.instance = instance
        self.setFlag(QGraphicsItem.ItemIsSelectable, True)
        self.setFlag(QGraphicsItem.ItemIsMovable, True)
        self.setFlag(QGraphicsItem.ItemSendsGeometryChanges, True)
        sx, sy = nm_to_scene(instance.position)
        self.setPos(sx, sy)
        self.setRotation(float(instance.rotation_deg))
        self._bounding = self._compute_bounding()

    def _compute_bounding(self) -> QRectF:
        # Union of body polylines and pin tips.
        xs, ys = [0.0], [0.0]
        for poly in self.symbol.body_polylines:
            for pt in poly:
                sx, sy = nm_to_scene(pt); xs.append(sx); ys.append(sy)
        for pin in self.symbol.pins:
            sx, sy = nm_to_scene(pin.position); xs.append(sx); ys.append(sy)
        margin = 30
        return QRectF(min(xs) - margin, min(ys) - margin,
                      max(xs) - min(xs) + 2 * margin,
                      max(ys) - min(ys) + 2 * margin)

    def boundingRect(self) -> QRectF:
        return self._bounding

    def shape(self) -> QPainterPath:
        # A simple bounding rect path is fine for hit-testing. For tighter
        # selection regions we'd union the body polylines.
        path = QPainterPath()
        path.addRect(self._bounding)
        return path

    def paint(self, painter: QPainter, option, widget=None):
```

```
        painter.setRenderHint(QPainter.Antialiasing)

        painter.setPen(PEN_SYMBOL)
        for poly in self.symbol.body_polylines:
            pts = [QPointF(*nm_to_scene(p)) for p in poly]
            for a, c in zip(pts, pts[1:]):
                painter.drawLine(a, c)

        painter.setPen(PEN_PIN)
        for pin in self.symbol.pins:
            tip = QPointF(*nm_to_scene(pin.position))
            base = self._pin_base(pin)
            painter.drawLine(tip, base)
            if pin.visible and pin.name not in ("~", ""):
                self._draw_pin_label(painter, pin, base)

        # Reference and value.
        painter.setPen(Qt.black)
        f = QFont("Arial", 8); painter.setFont(f)
        fm = QFontMetricsF(f)
        painter.drawText(self._bounding.left() + 4,
                         self._bounding.top() + fm.ascent() + 2,
                         self.instance.reference)
        painter.drawText(self._bounding.left() + 4,
                         self._bounding.top() + fm.ascent() + fm.height() + 2,
                         self.instance.value)

        if self.isSelected():
            painter.setPen(PEN_SELECT)
            painter.drawRect(self._bounding)

    def _pin_base(self, pin: Pin) -> QPointF:
        from pcbsmith.core.library import PinDirection
        sx, sy = nm_to_scene(pin.position)
        L = pin.length_nm / 25_400
        if pin.direction == PinDirection.UP:    return QPointF(sx, sy - L)
        if pin.direction == PinDirection.DOWN:  return QPointF(sx, sy + L)
        if pin.direction == PinDirection.LEFT:  return QPointF(sx - L, sy)
        return QPointF(sx + L, sy)   # RIGHT

    def _draw_pin_label(self, painter, pin, base: QPointF):
        painter.setPen(Qt.black)
        f = QFont("Arial", 7); painter.setFont(f)
        painter.drawText(base + QPointF(4, -4), pin.name)

    # ---- Sync model on move ----
    def itemChange(self, change, value):
        if change == QGraphicsItem.ItemPositionChange:
            # Snap to GRID_SCHEMATIC_MAJOR (50-mil grid).
            from .units import GRID_SCHEMATIC_MAJOR
```

```
            x = round(value.x() / GRID_SCHEMATIC_MAJOR) * GRID_SCHEMATIC_MAJOR
            y = round(value.y() / GRID_SCHEMATIC_MAJOR) * GRID_SCHEMATIC_MAJOR
            return QPointF(x, y)
        if change == QGraphicsItem.ItemPositionHasChanged:
            self.instance.position = scene_to_nm(value.x(), value.y())
        return super().itemChange(change, value)
```

```
```

Why subclass QGraphicsItem and not QGraphicsItemGroup? Group items add their children's bounding rects naively, which overpaints during transforms and produces flicker. A custom QGraphicsItem renders in one paint() call, is faster, and gives full control over the selection visuals.

7.8 WireItem and the wire-drawing tool

Wires are polylines that snap to the grid. While drawing, the user is in “wire mode”: each click adds a vertex; ESC or double-click finishes. Auto-90° elbows when the cursor moves diagonally between vertices, the same as KiCad and Eagle.

```
# ui/graphics/wire_item.py
```

```
from PySide6.QtCore import QPointF, QRectF, Qt
from PySide6.QtGui import QPainterPath, QPainterPathStroker
from PySide6.QtWidgets import QGraphicsItem
from pcbsmith.core.schematic import Wire
from .units import nm_to_scene, scene_to_nm
from .styles import PEN_WIRE, PEN_SELECT


class WireItem(QGraphicsItem):
    """Polyline visual for a Wire."""
    def __init__(self, wire: Wire, parent=None):
        super().__init__(parent)
        self.wire = wire
        self.setFlag(QGraphicsItem.ItemIsSelectable, True)

    def points(self) -> list[QPointF]:
        return [QPointF(*nm_to_scene(p)) for p in self.wire.points]

    def boundingRect(self) -> QRectF:
        pts = self.points()
        if not pts:
            return QRectF()
        xs = [p.x() for p in pts]; ys = [p.y() for p in pts]
        margin = 8
        return QRectF(min(xs) - margin, min(ys) - margin,
                      max(xs) - min(xs) + 2 * margin,
                      max(ys) - min(ys) + 2 * margin)

    def shape(self) -> QPainterPath:
        """Use a stroked path so clicks near the line still hit the item."""
        path = QPainterPath()
        pts = self.points()
        if not pts: return path
        path.moveTo(pts[0])
        for p in pts[1:]:
            path.lineTo(p)
        stroker = QPainterPathStroker()
        stroker.setWidth(12)   # 12 scene units = 12 mil click tolerance
        return stroker.createStroke(path)

    def paint(self, painter, option, widget=None):
        painter.setRenderHint(painter.Antialiasing)
        painter.setPen(PEN_SELECT if self.isSelected() else PEN_WIRE)
        pts = self.points()
        for a, c in zip(pts, pts[1:]):
            painter.drawLine(a, c)
```

7.9 The canvas and tool dispatch

SchematicCanvas is the QGraphicsView. It owns a SchematicScene (subclass of QGraphicsScene), a QUndoStack, and a current tool. Tools are small classes that intercept mouse events. This is a clean alternative to a giant if/elif chain in the view's mousePressEvent.

```
# ui/schematic_canvas.py (selected pieces)
```

```
from __future__ import annotations
from PySide6.QtCore import Qt, QPointF
from PySide6.QtGui import QPainter, QUndoStack, QUndoCommand
from PySide6.QtWidgets import QGraphicsScene, QGraphicsView
from pcbsmith.core.schematic import Schematic, Wire, SymbolInstance
from pcbsmith.core.geom import Point
from pcbsmith.core.ids import new_id
from .graphics.symbol_item import SymbolItem
from .graphics.wire_item import WireItem
from .graphics.grid_background import draw_grid
from .graphics.units import GRID_SCHEMATIC_MAJOR, scene_to_nm


class SchematicScene(QGraphicsScene):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setSceneRect(-5000, -3500, 10000, 7000)  # generous A3-ish in mils

    def drawBackground(self, painter, rect):
        view_scale = 1.0
        if self.views():
            view_scale = self.views()[0].transform().m11()
        draw_grid(painter, rect, view_scale)


class SchematicCanvas(QGraphicsView):
    def __init__(self, main_window):
        super().__init__()
        self.main_window = main_window
        self.scene_obj = SchematicScene(self)
        self.setScene(self.scene_obj)
        self.setRenderHint(QPainter.Antialiasing)
        self.setDragMode(QGraphicsView.RubberBandDrag)
        self.setTransformationAnchor(QGraphicsView.AnchorUnderMouse)
        self.scale(1.0, -1.0)   # flip Y so up is up

        self.undo_stack = QUndoStack(self)
        self.undo_stack.setUndoLimit(200)

        self.tool: BaseTool = SelectTool(self)
        self.schematic: Schematic | None = None

    # ---- API ----
    def set_tool(self, name: str, **kwargs):
        if name == "select":   self.tool = SelectTool(self)
        elif name == "place":  self.tool = PlaceTool(self, kwargs["symbol_id"])
        elif name == "wire":   self.tool = WireTool(self)
        elif name == "label":  self.tool = LabelTool(self)
```

```
        elif name == "junction": self.tool = JunctionTool(self)

    def load_schematic(self, sch: Schematic):
        self.schematic = sch
        self.scene_obj.clear()
        from pcbsmith.services.builtin_library import _SYMBOLS
        for inst in sch.symbols:
            self.scene_obj.addItem(SymbolItem(_SYMBOLS[inst.symbol_id], inst))
        for w in sch.wires:
            self.scene_obj.addItem(WireItem(w))

    def commit_to_model(self):
        """Pull current item positions back into the SymbolInstance / Wire
models.

        SymbolItem.itemChange already keeps SymbolInstance.position in sync.
        For Wires: WireItem holds the model object, so updates flow through too.
        """

    # ---- Forward to current tool ----
    def mousePressEvent(self, e):
        if not self.tool.mouse_press(e): super().mousePressEvent(e)
    def mouseMoveEvent(self, e):
        if not self.tool.mouse_move(e): super().mouseMoveEvent(e)
    def mouseReleaseEvent(self, e):
        if not self.tool.mouse_release(e): super().mouseReleaseEvent(e)
    def keyPressEvent(self, e):
        if not self.tool.key_press(e): super().keyPressEvent(e)
    def wheelEvent(self, e):
        # Ctrl + wheel = zoom; otherwise scroll.
        if e.modifiers() & Qt.ControlModifier:
            factor = 1.15 if e.angleDelta().y() > 0 else 1 / 1.15
            self.scale(factor, factor)
        else:
            super().wheelEvent(e)
```

```
```

7.9.1 Tool base class and Place tool

```
class BaseTool:
```

```
    def __init__(self, canvas): self.canvas = canvas
    def mouse_press(self, e):   return False
    def mouse_move(self, e):    return False
    def mouse_release(self, e): return False
    def key_press(self, e):     return False


class SelectTool(BaseTool):
    """Default tool: rubber-band select, drag to move."""


class PlaceTool(BaseTool):
    def __init__(self, canvas, symbol_id: str):
        super().__init__(canvas)
        from pcbsmith.services.builtin_library import _SYMBOLS
        self.symbol = _SYMBOLS[symbol_id]
        self.preview: SymbolItem | None = None

    def mouse_move(self, e):
        scene_pos = self.canvas.mapToScene(e.position().toPoint())
        x = round(scene_pos.x() / GRID_SCHEMATIC_MAJOR) * GRID_SCHEMATIC_MAJOR
        y = round(scene_pos.y() / GRID_SCHEMATIC_MAJOR) * GRID_SCHEMATIC_MAJOR
        if self.preview is None:
            inst = self._make_instance(scene_to_nm(x, y))
            self.preview = SymbolItem(self.symbol, inst)
            self.preview.setOpacity(0.5)
            self.canvas.scene_obj.addItem(self.preview)
        else:
            self.preview.setPos(x, y)
            self.preview.instance.position = scene_to_nm(x, y)
        return True

    def mouse_press(self, e):
        if e.button() != Qt.LeftButton or self.preview is None: return False
        # Commit. Lift opacity. Push undo command.
        self.preview.setOpacity(1.0)
        cmd = AddSymbolCommand(self.canvas.schematic, self.preview.instance,
                               self.preview, self.canvas.scene_obj)
        self.canvas.undo_stack.push(cmd)
        # Start a fresh preview at the same spot for chained placement.
        self.preview = None
        self.mouse_move(e)
        return True

    def key_press(self, e):
        if e.key() == Qt.Key_Escape:
```

```
            if self.preview:
                self.canvas.scene_obj.removeItem(self.preview)
                self.preview = None
            self.canvas.set_tool("select")
            return True
        if e.key() == Qt.Key_R and self.preview:
            self.preview.instance.rotation_deg =
(self.preview.instance.rotation_deg + 90) % 360
            self.preview.setRotation(self.preview.instance.rotation_deg)
            return True
        return False

    def _make_instance(self, position: Point) -> SymbolInstance:
        sch = self.canvas.schematic
        prefix = self.symbol.reference_prefix
        n = 1 + sum(1 for s in sch.symbols if s.reference.startswith(prefix))
        return SymbolInstance(
            id=new_id(),
            symbol_id=self.symbol.id,
            reference=f"{prefix}{n}",
            value=self.symbol.name,
            position=position,
        )
```

```
```

7.9.2 Wire tool

```
class WireTool(BaseTool):
```

```
    """Click to start, click for corners, double-click or ESC to finish."""
    def __init__(self, canvas):
        super().__init__(canvas)
        self.points_in_progress: list[Point] = []
        self.preview_item: WireItem | None = None

    def mouse_press(self, e):
        if e.button() != Qt.LeftButton: return False
        sp = self.canvas.mapToScene(e.position().toPoint())
        x = round(sp.x() / GRID_SCHEMATIC_MAJOR) * GRID_SCHEMATIC_MAJOR
        y = round(sp.y() / GRID_SCHEMATIC_MAJOR) * GRID_SCHEMATIC_MAJOR
        self.points_in_progress.append(scene_to_nm(x, y))
        if e.type().name == "MouseButtonDblClick":
            self._commit(); return True
        return True

    def mouse_move(self, e):
        if not self.points_in_progress: return False
        sp = self.canvas.mapToScene(e.position().toPoint())
        x = round(sp.x() / GRID_SCHEMATIC_MAJOR) * GRID_SCHEMATIC_MAJOR
        y = round(sp.y() / GRID_SCHEMATIC_MAJOR) * GRID_SCHEMATIC_MAJOR
        live = self.points_in_progress + [scene_to_nm(x, y)]
        if self.preview_item is None:
            self.preview_item = WireItem(Wire(id=new_id(), points=live))
            self.canvas.scene_obj.addItem(self.preview_item)
        else:
            self.preview_item.wire.points = live
            self.preview_item.update()
        return True

    def key_press(self, e):
        if e.key() == Qt.Key_Escape:
            if len(self.points_in_progress) >= 2: self._commit()
            else: self._abort()
            return True
        return False

    def _commit(self):
        if self.preview_item:
            self.canvas.scene_obj.removeItem(self.preview_item)
        wire = Wire(id=new_id(), points=list(self.points_in_progress))
        cmd = AddWireCommand(self.canvas.schematic, wire, self.canvas.scene_obj)
        self.canvas.undo_stack.push(cmd)
        self._reset()

    def _abort(self):
```

```
        if self.preview_item:
            self.canvas.scene_obj.removeItem(self.preview_item)
        self._reset()

    def _reset(self):
        self.points_in_progress = []
        self.preview_item = None
```

```
```

7.10 Undo / redo

Use QUndoStack from the start. Every model-mutating action is a QUndoCommand subclass. It is dramatically harder to bolt this on later than to design for it now. The pattern: redo() applies the change to both the model and the scene; undo() reverses it.

```
# Inside ui/schematic_canvas.py
```

```
class AddSymbolCommand(QUndoCommand):
    def __init__(self, schematic, instance, item, scene):
        super().__init__(f"Add {instance.reference}")
        self.schematic = schematic
        self.instance = instance
        self.item = item
        self.scene = scene

    def redo(self):
        if self.instance not in self.schematic.symbols:
            self.schematic.symbols.append(self.instance)
        if self.item.scene() is None:
            self.scene.addItem(self.item)

    def undo(self):
        if self.instance in self.schematic.symbols:
            self.schematic.symbols.remove(self.instance)
        if self.item.scene() is not None:
            self.scene.removeItem(self.item)


class AddWireCommand(QUndoCommand):
    def __init__(self, schematic, wire, scene):
        super().__init__("Draw wire")
        self.schematic = schematic
        self.wire = wire
        self.scene = scene
        self.item: WireItem | None = None

    def redo(self):
        if self.wire not in self.schematic.wires:
            self.schematic.wires.append(self.wire)
        if self.item is None:
            self.item = WireItem(self.wire)
            self.scene.addItem(self.item)
        elif self.item.scene() is None:
            self.scene.addItem(self.item)

    def undo(self):
        if self.wire in self.schematic.wires:
            self.schematic.wires.remove(self.wire)
        if self.item is not None and self.item.scene() is not None:
            self.scene.removeItem(self.item)
```

```
```

7.11 Library palette

Left dock listing the 25 built-in parts. Filter box at top, two-column list of name + ref prefix below. Selecting a part puts the canvas into PlaceTool with that symbol.

```
# ui/library_palette.py
```

```
from PySide6.QtCore import Qt
from PySide6.QtWidgets import (
    QDockWidget, QWidget, QVBoxLayout, QLineEdit, QListWidget, QListWidgetItem,
)
from pcbsmith.services.builtin_library import _SYMBOLS


class LibraryPalette(QDockWidget):
    def __init__(self, main_window):
        super().__init__("Library", main_window)
        self.main_window = main_window
        w = QWidget(); v = QVBoxLayout(w); v.setContentsMargins(6, 6, 6, 6)
        self.search = QLineEdit(); self.search.setPlaceholderText("Search…")
        self.list = QListWidget()
        v.addWidget(self.search); v.addWidget(self.list)
        self.setWidget(w)

        self._populate()
        self.search.textChanged.connect(self._filter)
        self.list.itemDoubleClicked.connect(self._on_pick)

    def _populate(self):
        self.list.clear()
        for sym in sorted(_SYMBOLS.values(), key=lambda s: s.name):
            item = QListWidgetItem(f"{sym.name}   ({sym.reference_prefix})")
            item.setData(Qt.UserRole, sym.id)
            self.list.addItem(item)

    def _filter(self, text):
        text = text.lower()
        for i in range(self.list.count()):
            item = self.list.item(i)
            sid = item.data(Qt.UserRole)
            sym = _SYMBOLS[sid]
            visible = (text in sym.name.lower() or
                       any(text in k for k in sym.keywords))
            item.setHidden(not visible)

    def _on_pick(self, item):
        sid = item.data(Qt.UserRole)
        self.main_window.schematic_canvas.set_tool("place", symbol_id=sid)
```

```
```

7.12 Inspector dock

Right dock that shows the properties of the selected item. Resistor selected? Show Reference, Value, Footprint, Position. Wire selected? Show net name (computed), point list. Edits commit through QUndoCommand wrappers so they participate in undo.

```
# ui/inspector.py (sketch)
```

```
from PySide6.QtCore import Slot
from PySide6.QtWidgets import (
    QDockWidget, QWidget, QFormLayout, QLineEdit, QLabel,
)
from .graphics.symbol_item import SymbolItem
from .graphics.wire_item import WireItem


class Inspector(QDockWidget):
    def __init__(self, main_window):
        super().__init__("Inspector", main_window)
        self.main_window = main_window
        self._w = QWidget(); self._lay = QFormLayout(self._w)
        self.setWidget(self._w)

        canvas = main_window.schematic_canvas
        canvas.scene_obj.selectionChanged.connect(self._refresh)

    @Slot()
    def _refresh(self):
        # Clear existing rows
        while self._lay.rowCount(): self._lay.removeRow(0)

        sel = self.main_window.schematic_canvas.scene_obj.selectedItems()
        if len(sel) != 1:
            self._lay.addRow(QLabel("—"))
            return
        item = sel[0]
        if isinstance(item, SymbolItem):
            inst = item.instance
            ref_edit = QLineEdit(inst.reference)
            val_edit = QLineEdit(inst.value)
            self._lay.addRow("Reference", ref_edit)
            self._lay.addRow("Value",     val_edit)
            self._lay.addRow("Symbol",    QLabel(item.symbol.id))
            self._lay.addRow("Position",  QLabel(f"{inst.position.x/1e6:.2f},
{inst.position.y/1e6:.2f} mm"))
            ref_edit.editingFinished.connect(
                lambda: self._set_ref(inst, item, ref_edit.text()))
            val_edit.editingFinished.connect(
                lambda: self._set_value(inst, item, val_edit.text()))
        elif isinstance(item, WireItem):
            self._lay.addRow("Type", QLabel("Wire"))
            self._lay.addRow("Points", QLabel(str(len(item.wire.points))))

    def _set_ref(self, inst, item, text):
        # In production, push a SetFieldCommand onto the undo stack.
        inst.reference = text
```

```
        item.update()

    def _set_value(self, inst, item, text):
        inst.value = text
        item.update()
```

```
```

7.13 Console dock and ERC integration

Bottom dock with three tabs: ERC, LLM history, log. ERC tab is a list of issues; double-clicking an issue should pan / zoom the canvas to the offending item.

```
# ui/console.py (ERC tab)
```

```
from PySide6.QtCore import Qt
from PySide6.QtWidgets import (
    QDockWidget, QTabWidget, QListWidget, QListWidgetItem, QPlainTextEdit,
)
from pcbsmith.services.erc import ErcIssue, Severity


_SEV_ICON = {Severity.INFO: "ℹ️", Severity.WARNING: "⚠", Severity.ERROR: "✖"}


class Console(QDockWidget):
    def __init__(self, main_window):
        super().__init__("Console", main_window)
        self.main_window = main_window
        self.tabs = QTabWidget()
        self.erc = QListWidget()
        self.history = QPlainTextEdit(); self.history.setReadOnly(True)
        self.log = QPlainTextEdit(); self.log.setReadOnly(True)
        self.tabs.addTab(self.erc, "ERC")
        self.tabs.addTab(self.history, "LLM")
        self.tabs.addTab(self.log, "Log")
        self.setWidget(self.tabs)
        self.erc.itemDoubleClicked.connect(self._on_erc_dblclick)

    def show_erc(self, issues: list[ErcIssue]):
        self.erc.clear()
        for iss in issues:
            text = f"{_SEV_ICON.get(iss.severity, '?')}  {iss.code}  {iss.where}:
{iss.message}"
            item = QListWidgetItem(text)
            item.setData(Qt.UserRole, iss)
            self.erc.addItem(item)
        self.tabs.setCurrentWidget(self.erc)

    def _on_erc_dblclick(self, item: QListWidgetItem):
        iss: ErcIssue = item.data(Qt.UserRole)
        # Pan the canvas to the offending instance/net. For "R3.2" extract "R3".
        ref = iss.where.split(".")[0] if "." in iss.where else None
        if not ref: return
        canvas = self.main_window.schematic_canvas
        for it in canvas.scene_obj.items():
            from .graphics.symbol_item import SymbolItem
            if isinstance(it, SymbolItem) and it.instance.reference == ref:
                canvas.centerOn(it)
                it.setSelected(True)
                break
```

7.14 Acceptance test for Phase 1

pytest-qt drives the application headlessly. The test below replicates the milestone from the chapter intro: place two resistors, draw a wire, save, reopen, verify.

```
# tests/integration/test_phase1_acceptance.py
```

```
import pytest
from pathlib import Path
from PySide6.QtCore import Qt, QPointF
from pcbsmith.ui.main_window import MainWindow
from pcbsmith.services.project_io import load


@pytest.mark.qt
def test_voltage_divider_via_ui(qtbot, tmp_path):
    w = MainWindow(); qtbot.addWidget(w)
    w.show()

    # Create a project at tmp_path/divider.
    project_path = tmp_path / "divider"
    project_path.mkdir()
    from pcbsmith.services.project_io import new_project
    w.project = new_project(project_path, name="divider")
    w.project_path = project_path
    w._refresh()
    sch = w.project.schematics[0] = type(w.project.schematics[0] if
w.project.schematics else None)(  # placeholder
        id="x", name="main")  # in production new_project would create one

    # Place two resistors via the API the place tool uses.
    canvas = w.schematic_canvas
    canvas.set_tool("place", symbol_id="stdlib:R")
    qtbot.mouseMove(canvas.viewport(), pos=QPointF(0, 0))
    qtbot.mouseClick(canvas.viewport(), Qt.LeftButton, pos=QPointF(0, 0))
    qtbot.mouseClick(canvas.viewport(), Qt.LeftButton, pos=QPointF(0, 200))   #
200 mil down

    # Wire them.
    canvas.set_tool("wire")
    qtbot.mouseClick(canvas.viewport(), Qt.LeftButton, pos=QPointF(0, 100))
    qtbot.mouseClick(canvas.viewport(), Qt.LeftButton, pos=QPointF(0, 200))
    qtbot.keyPress(canvas, Qt.Key_Escape)

    w.on_save()

    # Re-open from disk.
    fresh = load(project_path)
    assert len(fresh.schematics[0].symbols) == 2
    assert len(fresh.schematics[0].wires) >= 1
```

```
```

pytest-qt note. Headless GUI tests are slow and flaky compared to model tests. Cover the UI broadly but not deeply: one happy-path acceptance test per phase, plus a handful of smoke tests for regression-prone widgets. Drive the model directly for everything else.

7.15 Phase 1 gotchas

- **Coordinate flipping.** Qt's scene Y points down by default; schematic users expect Y up. Apply a scale(1, -1) on the view, not on the scene, so item bounding rects stay sane.
- **Drag-mode and rubber band.** Set RubberBandDrag, but disable it while a non-select tool is active or you'll see weird selection rectangles every time the user clicks.
- **Item bounding rect must be stable.** If boundingRect() returns different rects after a setRotation(), the scene's BSP tree breaks. Compute boundingRect in local coords and let Qt apply the transform.
- **Antialiasing on every paint.** Paint is called constantly during pan/zoom. setRenderHint at the view level once, not per-item.
- **Net names update lazily.** Recomputing the netlist on every keystroke makes the UI feel sluggish on large schematics. Recompute on a timer (250 ms debounce) or on demand from ERC.

8. Phase 3 — LLM Pipeline

This is the feature that distinguishes PCBSmith from KiCad. The user types a circuit description in plain English; the application converts it into a real schematic with real components, real nets, and a layout that opens in the schematic editor.

The user is not allowed near a finished schematic the LLM produced directly. The LLM produces a structured intermediate representation (IR) that the application validates and then materialises into Schematic objects. There are three reasons this matters.

- **Validation.** If the LLM emits a non-existent part, an unknown pin number, or a malformed value, we want the application to refuse the change and explain why — not silently produce a broken schematic that the user wonders about an hour later when DRC fails.
- **Determinism on retry.** The IR is cheap to validate, cheap to diff against the existing schematic, and cheap to apply or reject. If we let the LLM emit native schematic objects, we'd get random UUIDs everywhere and no two retries would diff cleanly.
- **Substitutability.** Today the IR is filled by Claude. Tomorrow it could be filled by a local model, or by a different vendor, or by a programmatic recipe. The IR is the contract; the LLM is an implementation detail.

Phase 3 milestone. The user types “ATtiny85 with an LED on PB0 and a 3xAA battery” into the prompt panel. The application produces an IR (visible in the side pane), validates it (“unknown part stdlib:ATtiny85” — the built-in library doesn't have it; after Phase 4 the KiCad library does), shows a diff (“Adds 3 components, 4 nets, 5 wires”), and on Apply, the schematic editor populates with the components.

8.1 The IR schema

The IR is a pydantic model. Keep it small. It does not need to express every nuance of the underlying Schematic model; the materialiser fills in defaults (positions, pin tips, junction inference). It only needs to express what the LLM is supposed to know: what parts to use, what their values are, how they connect, and what the user-visible labels are.

```
# services/llm/ir_schema.py
```

```
from __future__ import annotations
from typing import Literal
from pydantic import BaseModel, Field, ConfigDict


class IRComponent(BaseModel):
    """One component the user wants on the board."""
    model_config = ConfigDict(extra="forbid")
    reference: str             # "R1", "U1", "D2". Must be unique in this IR.
    symbol_id: str             # library entry: "stdlib:R", "kicad:Device:LED"
    value: str                 # "10k", "STM32F411CEU6", "red"
    footprint_id: str | None = None     # optional; default footprint used if
None
    fields: dict[str, str] = Field(default_factory=dict)   # MPN, Manufacturer,
Datasheet


class IRConnection(BaseModel):
    """One pin participating in a net."""
    model_config = ConfigDict(extra="forbid")
    reference: str             # must match an IRComponent.reference
    pin: str                   # the pin NUMBER ("1", "3"), not the pin name


class IRNet(BaseModel):
    """A named net and the pins on it."""
    model_config = ConfigDict(extra="forbid")
    name: str                  # "VCC", "GND", "SDA", or "" for auto-named
    is_power: bool = False     # true for VCC/GND/+3V3 etc.
    pins: list[IRConnection] = Field(min_length=2)   # at least two pins to be a
net


class IR(BaseModel):
    """The whole intermediate representation.

    A complete IR fully describes a schematic at the logical level: every
    component, every net, with explicit pin numbers. It does NOT describe
    geometry; the materialiser handles placement. It does NOT describe
    footprint geometry; that comes from the library.
    """
    model_config = ConfigDict(extra="forbid")
    schema_version: Literal[1] = 1
    rationale: str = ""          # one paragraph, why these choices
    components: list[IRComponent] = Field(default_factory=list)
    nets:       list[IRNet]       = Field(default_factory=list)
    notes: list[str] = Field(default_factory=list)   # questions/warnings to
surface to the user
```

Why pin numbers, not pin names? Pin names like “VCC”, “SDA”, “IN+” are not unique on a symbol — a quad op-amp has four pins called IN+, four called IN-, four called OUT. Pin numbers (1, 2, 3, …) are guaranteed unique. The library tells the LLM which number maps to which name in the prompt; the model gives back numbers; we resolve them during validation.

8.2 Validating the IR

The pydantic schema gets you syntactic validation for free (extra="forbid", required fields, types). Semantic validation is your responsibility, and it is the more important half. Validation must catch:

- Unknown symbol_id (not in any loaded library).
- Unknown footprint_id.
- Reference designator collisions (two R1s).
- Connection to a pin number the symbol does not have.
- A net with zero or one pin.
- A net containing two POWER_OUT pins (drives in conflict).
- A power net not connected to any source.
- Components mentioned only in connections but not declared.

```
# services/llm/validator.py
```

```
from __future__ import annotations
from dataclasses import dataclass
from collections import Counter
from .ir_schema import IR
from pcbsmith.core.library import Symbol, PinElectricalType
from pcbsmith.services.builtin_library import _SYMBOLS, _FOOTPRINTS


@dataclass(frozen=True)
class IRError:
    code: str        # "V001" .. "V015"
    message: str
    where: str = ""


@dataclass
class ValidationResult:
    errors:   list[IRError]
    warnings: list[IRError]
    @property
    def ok(self) -> bool: return not self.errors


def validate(ir: IR,
             symbols: dict[str, Symbol] = _SYMBOLS,
             footprints: dict[str, object] = _FOOTPRINTS) -> ValidationResult:
    errs:  list[IRError] = []
    warns: list[IRError] = []

    # 1. Reference designator uniqueness
    counts = Counter(c.reference for c in ir.components)
    for ref, n in counts.items():
        if n > 1:
            errs.append(IRError("V001",
                f"Duplicate reference designator '{ref}' (declared {n} times)",
ref))

    # 2. Symbol existence
    refs_to_symbols: dict[str, Symbol] = {}
    for c in ir.components:
        sym = symbols.get(c.symbol_id)
        if sym is None:
            errs.append(IRError("V002",
                f"Unknown symbol_id '{c.symbol_id}' for {c.reference}",
c.reference))
            continue
        refs_to_symbols[c.reference] = sym
```

```
    # 3. Footprint existence
    for c in ir.components:
        if c.footprint_id is None: continue
        if c.footprint_id not in footprints:
            errs.append(IRError("V003",
                f"Unknown footprint_id '{c.footprint_id}' for {c.reference}",
c.reference))

    # 4. Connection sanity: every reference exists, every pin exists
    declared = {c.reference for c in ir.components}
    for net in ir.nets:
        if len(net.pins) < 2:
            errs.append(IRError("V004",
                f"Net '{net.name}' has fewer than 2 pins", f"Net {net.name}"))
        for conn in net.pins:
            if conn.reference not in declared:
                errs.append(IRError("V005",
                    f"Net '{net.name}' refers to undeclared component
'{conn.reference}'",
                    f"Net {net.name}"))
                continue
            sym = refs_to_symbols.get(conn.reference)
            if sym is None: continue   # already errored on V002
            if not any(p.number == conn.pin for p in sym.pins):
                errs.append(IRError("V006",
                    f"Pin '{conn.pin}' not found on {conn.reference} "
                    f"({sym.name}); has pins {[p.number for p in sym.pins]}",
                    f"{conn.reference}.{conn.pin}"))

    # 5. Output conflicts (two POWER_OUT or two OUTPUT on one net)
    for net in ir.nets:
        outputs = []
        for conn in net.pins:
            sym = refs_to_symbols.get(conn.reference)
            if sym is None: continue
            pin = next((p for p in sym.pins if p.number == conn.pin), None)
            if pin and pin.electrical_type in (PinElectricalType.OUTPUT,
                                               PinElectricalType.POWER_OUT):
                outputs.append(f"{conn.reference}.{conn.pin}")
        if len(outputs) > 1:
            errs.append(IRError("V007",
                f"Net '{net.name}': multiple outputs ({', '.join(outputs)}) drive
same net",
                f"Net {net.name}"))

    # 6. Power nets unsourced
    for net in ir.nets:
        if not net.is_power: continue
        sources = []
        for conn in net.pins:
```

```
            sym = refs_to_symbols.get(conn.reference)
            if sym is None: continue
            pin = next((p for p in sym.pins if p.number == conn.pin), None)
            if pin and pin.electrical_type == PinElectricalType.POWER_OUT:
                sources.append(f"{conn.reference}.{conn.pin}")
        if not sources:
            warns.append(IRError("V008",
                f"Power net '{net.name}' has no POWER_OUT source", f"Net
{net.name}"))

    return ValidationResult(errors=errs, warnings=warns)
```

8.3 Materialising the IR into a Schematic

Once an IR validates, the materialiser converts it into actual SymbolInstance / Wire / NetLabel objects with concrete positions. This is the simplest piece to write but the most consequential to get right. Two design decisions:

- **Auto-place by gridded force-direction.** First pass: lay components out on a coarse grid, group by the nets they share. Run a few iterations of a tiny force-directed simulation (springs along nets, repulsion between symbols) snapped to the schematic grid. The result is rough, and that's fine: the user will rearrange.
- **Wires implied, not drawn.** Initial materialisation does not draw wires; it places net labels at every pin tip involved in a multi-pin net. KiCad-style “global label” behaviour. Two pins with the same label → same net. The user can promote labels to wires when they want neat orthogonal routes.

```
# services/llm/materialise.py
```

```
from __future__ import annotations
from .ir_schema import IR, IRComponent
from pcbsmith.core.geom import Point
from pcbsmith.core.schematic import Schematic, SymbolInstance, NetLabel
from pcbsmith.core.library import Symbol
from pcbsmith.core.ids import new_id


GRID_NM = 2_540_000   # 100 mil

def materialise(ir: IR, symbols: dict[str, Symbol]) -> Schematic:
    """Convert a validated IR into a fresh Schematic.

    Caller is responsible for either replacing an existing schematic or merging.
    See §8.5 for the diff/merge approach used by the application.
    """
    sch = Schematic(id=new_id(), name="main")

    # 1. Lay components on a coarse grid: 8 per row, spaced 1500 mil apart.
    SPAN = 15 * GRID_NM
    for i, c in enumerate(ir.components):
        sym = symbols[c.symbol_id]
        col, row = i % 8, i // 8
        sch.symbols.append(SymbolInstance(
            id=new_id(),
            symbol_id=c.symbol_id,
            reference=c.reference,
            value=c.value,
            position=Point(col * SPAN, -row * SPAN),
            footprint_id=c.footprint_id or sym.default_footprint,
            fields=c.fields.copy(),
        ))

    # 2. For every net, place a NetLabel at each pin tip.
    for net in ir.nets:
        for conn in net.pins:
            inst = next(s for s in sch.symbols if s.reference == conn.reference)
            sym = symbols[inst.symbol_id]
            pin = next(p for p in sym.pins if p.number == conn.pin)
            tip = Point(inst.position.x + pin.position.x,
                        inst.position.y + pin.position.y)
            sch.labels.append(NetLabel(
                id=new_id(),
                position=tip,
                text=net.name or _autoname(net),
                is_power=net.is_power,
            ))
    return sch
```

```
def _autoname(net):
    if not net.pins: return "net"
    first = net.pins[0]
    return f"Net-({first.reference}-{first.pin})"
```

8.4 Prompting the model

The system prompt teaches the model what the IR is, what library entries it can use, and what the constraints are. The user prompt is the user's natural-language description. The model emits an IR via structured outputs.

```
# services/llm/prompt.py
```

```
from __future__ import annotations
from pcbsmith.core.library import Symbol


SYSTEM_PREAMBLE = """You are an electrical-engineering assistant inside a PCB
design tool.
Your job is to convert a user's natural-language description of a circuit into
a structured IR (intermediate representation) that the tool will validate and
turn into a real schematic.

You produce only the IR; never natural-language explanations of components,
never code, never suggested layouts, never diagrams. The tool handles all of
those. Your one and only output is a valid IR per the schema.

You may set the `rationale` field of the IR to a short paragraph explaining
your major design choices (e.g. "chose 0.1µF bypass on each VCC pin per
datasheet recommendation"). You may set `notes` for things the user should
review (e.g. "datasheet for U2 is unclear on pin 7 — confirm before
fabrication").

Rules:
1. Only use components from the library list below. If the user asks for a
   part not in the library, prefer a near-equivalent that IS in the library
   and add a note explaining the substitution. Never invent a symbol_id.
2. Every component you list must be connected to at least one net. A floating
   component is an error.
3. Every connection must use a pin NUMBER (1, 2, A6) from the library data
   below, not a pin name. Get this right; the tool rejects mismatches.
4. Power nets (VCC, GND, +3V3, +5V) must have is_power=true and must include
   a power source pin (POWER_OUT type).
5. Reference designators follow EE convention: R for resistors, C for
   capacitors, U for ICs, D for diodes, Q for transistors, J for connectors,
   Y for crystals, SW for switches, BT for batteries, TP for test points.
   Number them sequentially (R1, R2, ...).
6. If the user's request is ambiguous, make a sensible choice and add a note.
   Do not ask follow-up questions; produce a complete IR even if some details
   are inferred.
"""

def build_system_prompt(library_subset: list[Symbol]) -> str:
    parts = [SYSTEM_PREAMBLE, "\nLIBRARY (only these symbols are available):\n"]
    for sym in library_subset:
        pin_table = ", ".join(f"{p.number}={p.name}({p.electrical_type.value})"
                              for p in sym.pins)
        parts.append(
            f"  {sym.id}  ({sym.reference_prefix}*)  — {sym.name}: pins
[{pin_table}]"
        )
```

```
    return "\n".join(parts)
```

```
```

Don't dump 20,000 KiCad symbols into the system prompt. When the full KiCad library is loaded (Phase 4), pre-filter the library before prompting. Use the user's request as a query — keyword-match against symbol names/keywords — and include the top 50–100 candidates plus the always-available passives (R, C, L, D, Q). Re-prompt-on-miss if the model says the part it wants is not in the subset.

8.5 Diff and confirm

Never apply an IR to the user's project without showing them what will change. Compute a diff between the existing Schematic and the materialised IR, present it in a confirmation dialog, and only mutate the project on Apply.

```
# services/llm/diff.py
```

```
from __future__ import annotations
from dataclasses import dataclass, field
from pcbsmith.core.schematic import Schematic


@dataclass
class SchematicDiff:
    added_components:    list[str] = field(default_factory=list)
    removed_components:  list[str] = field(default_factory=list)
    modified_components: list[tuple[str, str]] = field(default_factory=list)  #
(ref, what changed)
    added_nets:          list[str] = field(default_factory=list)
    removed_nets:        list[str] = field(default_factory=list)
    @property
    def is_empty(self) -> bool:
        return not (self.added_components or self.removed_components
                 or self.modified_components or self.added_nets or
self.removed_nets)


def diff_schematics(old: Schematic, new: Schematic) -> SchematicDiff:
    d = SchematicDiff()
    old_by_ref = {s.reference: s for s in old.symbols}
    new_by_ref = {s.reference: s for s in new.symbols}
    for ref in new_by_ref:
        if ref not in old_by_ref: d.added_components.append(ref)
        else:
            o, n = old_by_ref[ref], new_by_ref[ref]
            if o.symbol_id != n.symbol_id:
                d.modified_components.append((ref, f"{o.symbol_id} →
{n.symbol_id}"))
            elif o.value != n.value:
                d.modified_components.append((ref, f"{o.value!r} → {n.value!r}"))
    for ref in old_by_ref:
        if ref not in new_by_ref: d.removed_components.append(ref)

    old_label_names = {l.text for l in old.labels}
    new_label_names = {l.text for l in new.labels}
    d.added_nets = sorted(new_label_names - old_label_names)
    d.removed_nets = sorted(old_label_names - new_label_names)
    return d
```

```

```

```
# ui/dialogs/ir_apply.py (sketch)
```

```
from PySide6.QtWidgets import (
    QDialog, QVBoxLayout, QLabel, QPlainTextEdit, QDialogButtonBox, QSplitter,
)
from PySide6.QtCore import Qt
from pcbsmith.services.llm.ir_schema import IR
from pcbsmith.services.llm.diff import SchematicDiff

class IRApplyDialog(QDialog):
    def __init__(self, ir: IR, diff: SchematicDiff, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Apply LLM proposal?")
        self.resize(900, 600)
        v = QVBoxLayout(self)

        v.addWidget(QLabel(f"<b>Rationale</b><br>{ir.rationale or '—'}"))

        # diff summary
        v.addWidget(QLabel("<b>Summary of changes</b>"))
        summary = []
        if diff.added_components:
            summary.append(f"➕ Add {len(diff.added_components)} components: {',
'.join(diff.added_components)}")
        if diff.removed_components:
            summary.append(f"➖ Remove {len(diff.removed_components)} components:
{', '.join(diff.removed_components)}")
        if diff.modified_components:
            summary.append(f"✏ Modify {len(diff.modified_components)}: "
                           + "; ".join(f"{r}: {what}" for r, what in
diff.modified_components))
        if diff.added_nets:   summary.append(f"➕ Add nets: {',
'.join(diff.added_nets)}")
        if diff.removed_nets: summary.append(f"➖ Remove nets: {',
'.join(diff.removed_nets)}")
        v.addWidget(QPlainTextEdit("\n".join(summary) or "No changes."))

        # Full IR for inspection
        v.addWidget(QLabel("<b>Full IR</b>"))
        ir_view = QPlainTextEdit(ir.model_dump_json(indent=2));
ir_view.setReadOnly(True)
        v.addWidget(ir_view)

        if ir.notes:
            v.addWidget(QLabel("<b>Notes from the model</b>"))
            v.addWidget(QPlainTextEdit("\n".join("• " + n for n in ir.notes)))

        bb = QDialogButtonBox(QDialogButtonBox.Apply | QDialogButtonBox.Cancel)
        bb.button(QDialogButtonBox.Apply).clicked.connect(self.accept)
        bb.button(QDialogButtonBox.Cancel).clicked.connect(self.reject)
```

```
        v.addWidget(bb)
```

```
```

8.6 Local-model fallback (optional)

Anthropic's structured outputs guarantee a valid IR; local models do not. The OllamaBackend implements parse() with a simple JSON-mode prompt and a retry loop on validation failure. Cap retries at three; if the local model still produces malformed JSON, surface the parse error to the user.

```
# services/llm/client.py (continued)
```

```
import json, requests
from pydantic import BaseModel, ValidationError

class OllamaBackend:
    def __init__(self, model="qwen2.5-coder:14b", url="http://localhost:11434"):
        self.model, self.url = model, url

    def parse(self, system: str, user: str, schema, max_tokens: int = 4096):
        schema_json = schema.model_json_schema()
        instruction = (
            "\n\nRespond with a single valid JSON object that matches THIS
schema. "
            "Do not include any prose, markdown, or commentary.\n\n"
            f"SCHEMA:\n{json.dumps(schema_json, indent=2)}"
        )
        last_err = None
        prompt = system + instruction
        for attempt in range(3):
            r = requests.post(f"{self.url}/api/chat", json={
                "model": self.model, "format": "json", "stream": False,
                "messages": [
                    {"role": "system", "content": prompt},
                    {"role": "user",   "content": user},
                ],
                "options": {"num_predict": max_tokens, "temperature": 0.2},
            }, timeout=180).json()
            content = r.get("message", {}).get("content", "")
            try:
                return schema.model_validate_json(content)
            except (json.JSONDecodeError, ValidationError) as e:
                last_err = e
                # Add the failure to the prompt for the next try.
                prompt += f"\n\nPrevious attempt failed validation: {e}. Try
again."
        raise last_err
```

```
```

8.7 The prompt panel

Right dock or detachable window with a multi-line input, a Submit button, the IR preview, and a history of past prompts. Submitting fires the LLM, runs validation, computes the diff, and shows the apply dialog. Append every prompt and IR to prompts/history.jsonl so the user can replay or share them.

```
# ui/prompt_panel.py (skeleton)
```

```
from PySide6.QtCore import QThread, Signal
from PySide6.QtWidgets import (
    QDockWidget, QWidget, QVBoxLayout, QPlainTextEdit, QPushButton,
    QLabel, QMessageBox,
)
from pcbsmith.services.llm.client import LLMClient
from pcbsmith.services.llm.ir_schema import IR
from pcbsmith.services.llm.prompt import build_system_prompt
from pcbsmith.services.llm.validator import validate
from pcbsmith.services.llm.materialise import materialise
from pcbsmith.services.llm.diff import diff_schematics
from pcbsmith.services.builtin_library import _SYMBOLS


class LLMWorker(QThread):
    result = Signal(object)   # IR
    error  = Signal(str)
    def __init__(self, system, user):
        super().__init__()
        self.system, self.user = system, user
    def run(self):
        try:
            client = LLMClient()
            ir = client.parse(self.system, self.user, IR, max_tokens=8000)
            self.result.emit(ir)
        except Exception as e:
            self.error.emit(str(e))


class PromptPanel(QDockWidget):
    def __init__(self, main_window):
        super().__init__("Prompt", main_window)
        self.main_window = main_window
        w = QWidget(); v = QVBoxLayout(w)
        self.input = QPlainTextEdit()
        self.input.setPlaceholderText("Describe your circuit…")
        self.submit = QPushButton("Submit")
        self.status = QLabel("")
        v.addWidget(self.input); v.addWidget(self.submit);
v.addWidget(self.status)
        self.setWidget(w)
        self.submit.clicked.connect(self._submit)
        self.worker = None

    def _submit(self):
        if self.worker and self.worker.isRunning(): return
        text = self.input.toPlainText().strip()
        if not text: return
```

```
        self.status.setText("Calling Claude…")
        self.submit.setEnabled(False)

        # Pre-filter library by simple keyword match for now.
        lib = list(_SYMBOLS.values())
        sys_prompt = build_system_prompt(lib)
        self.worker = LLMWorker(sys_prompt, text)
        self.worker.result.connect(self._on_result)
        self.worker.error.connect(self._on_error)
        self.worker.start()

    def _on_result(self, ir: IR):
        self.submit.setEnabled(True)
        result = validate(ir)
        if not result.ok:
            details = "\n".join(f"{e.code} {e.where}: {e.message}" for e in
result.errors)
            QMessageBox.critical(self, "IR validation failed", details)
            return
        # Build a candidate Schematic, diff, show dialog.
        new_sch = materialise(ir, _SYMBOLS)
        existing = self.main_window.project.schematics[0]
        d = diff_schematics(existing, new_sch)
        from .dialogs.ir_apply import IRApplyDialog
        dlg = IRApplyDialog(ir, d, self.main_window)
        if dlg.exec():
            # Apply: replace the schematic. Phase 3 keeps it simple; Phase 4 adds
merge.
            self.main_window.project.schematics[0] = new_sch
            self.main_window.schematic_canvas.load_schematic(new_sch)
            self._append_history(self.input.toPlainText(), ir)

    def _on_error(self, msg: str):
        self.submit.setEnabled(True)
        self.status.setText("")
        QMessageBox.critical(self, "LLM error", msg)

    def _append_history(self, prompt_text: str, ir: IR):
        import json
        from pathlib import Path
        from datetime import datetime, timezone
        if not self.main_window.project_path: return
        log = Path(self.main_window.project_path) / "prompts" / "history.jsonl"
        with log.open("a", encoding="utf-8") as fh:
            fh.write(json.dumps({
                "ts": datetime.now(timezone.utc).isoformat(),
                "prompt": prompt_text,
                "ir": ir.model_dump(),
            }) + "\n")
```

8.8 Few-shot examples in the prompt

The model gets dramatically better at IR production with two or three worked examples in the system prompt. Add them at the bottom of build_system_prompt. Keep them tiny (≤ 5 components each) and pick examples that demonstrate the most error-prone patterns: power nets, ICs with many pins, and IR notes.

```
FEW_SHOT_EXAMPLES = [
```

```
    {
        "user": "A 1k pull-up resistor on a button input to a microcontroller
GPIO.",
        "ir": {
            "rationale": "Pull-up biases the input high; button shorts to GND
when pressed.",
            "components": [
                {"reference": "R1", "symbol_id": "stdlib:R", "value": "1k"},
                {"reference": "SW1", "symbol_id": "stdlib:SW_Push", "value":
"button"},
            ],
            "nets": [
                {"name": "VCC", "is_power": True,
                 "pins": [{"reference": "R1", "pin": "1"}]},
                {"name": "GPIO", "pins": [
                    {"reference": "R1", "pin": "2"},
                    {"reference": "SW1", "pin": "1"},
                ]},
                {"name": "GND", "is_power": True,
                 "pins": [{"reference": "SW1", "pin": "2"}]},
            ],
        },
    },
    # add 1–2 more covering: bypass cap on IC, voltage divider, LED with current-
limit R
]
```

8.9 Things to watch out for

- **Latency.** A long IR (50+ components) can take 30 seconds. Always use a worker thread; never block the GUI on the API call. Show a spinner.
- **Token limits.** max_tokens of 8000 is enough for ~30 components. For larger circuits, set higher; for very large, decompose the prompt ("design the power supply" → "now add the MCU and its supporting parts").
- **Cost awareness.** Show the token count and approximate cost in the prompt panel. Users get anxious without it. Cache the system prompt.
- **Re-prompts cost real money.** If validation fails, do not silently re-prompt. Show the user the IR, the errors, and let them decide whether to retry, edit the prompt, or abandon. Hidden retry loops are how a $5 model run becomes a $50 one.
- **Prompt injection.** If the user pastes content from the web (a forum thread, a datasheet snippet), treat that text as untrusted. Never let the model emit shell commands or take actions based on the content. Fortunately the IR itself is a constraint: an IR can only describe a circuit, not run code.

9. Phase 2 — PCB Editor MVP

Phase 2 takes the schematic from Phase 1 and lets the user lay out a physical PCB. The user assigns footprints, places them on a board outline, sees the ratsnest, draws traces by hand, places vias, runs DRC, and exports Gerbers. Auto-routing, push-and-shove, and zone fills come later (Phase 5).

> **Phase 2 milestone.** From the voltage divider built in Phase 1, click Tools → Generate Board. The PCB canvas opens with two footprints and a ratsnest line connecting their pads. Draw a trace from one to the other, draw a board outline, save, click Export Gerbers; verify the resulting files open without errors in gerbv. The board you can hold in your hand.

9.1 Layer stackup

Start with the universal 2-layer stackup. Add a 4-layer option in Phase 5; any deeper is best left to Phase 6+ once routing has been proven. The stackup determines which Gerber files you produce.

```
# services/board_setup.py
```

```
from pcbsmith.core.board import Board, Layer, LayerType, DesignRules
from pcbsmith.core.ids import new_id

DEFAULT_2_LAYER = [
    Layer(ordinal=0,  name="F.Cu",      type=LayerType.SIGNAL),
    Layer(ordinal=31, name="B.Cu",      type=LayerType.SIGNAL),
    Layer(ordinal=32, name="F.Adhes",   type=LayerType.ADHES,  visible=False),
    Layer(ordinal=33, name="B.Adhes",   type=LayerType.ADHES,  visible=False),
    Layer(ordinal=34, name="F.Paste",   type=LayerType.PASTE),
    Layer(ordinal=35, name="B.Paste",   type=LayerType.PASTE),
    Layer(ordinal=36, name="F.SilkS",   type=LayerType.SILK),
    Layer(ordinal=37, name="B.SilkS",   type=LayerType.SILK),
    Layer(ordinal=38, name="F.Mask",    type=LayerType.MASK),
    Layer(ordinal=39, name="B.Mask",    type=LayerType.MASK),
    Layer(ordinal=40, name="Edge.Cuts", type=LayerType.EDGE),
    Layer(ordinal=44, name="F.Fab",     type=LayerType.FAB,    visible=False),
    Layer(ordinal=45, name="B.Fab",     type=LayerType.FAB,    visible=False),
]


def new_board(name: str = "main") -> Board:
    return Board(id=new_id(), name=name, layers=DEFAULT_2_LAYER,
                 design_rules=DesignRules())
```

```
```

KiCad layer names matter. Use the canonical KiCad layer names (F.Cu, B.Cu, F.SilkS, etc.). When you export to KiCad in Phase 4, no remapping is needed. Standard manufacturer Gerber filename conventions also expect these.

9.2 Tasks for Phase 2, in order

26. Add new_board() and the layer stackup constants. Add a 'Generate board' menu item.
27. Implement footprint-assignment dialog: for each unassigned SymbolInstance, pick a Footprint.
28. Implement FootprintItem (QGraphicsItem subclass) with per-layer rendering.
29. Implement ratsnest computation (networkx MST per net) and RatsnestItem rendering.
30. Implement TraceItem and the routing tool: 45° corners, click-to-segment, snap-to-pad.
31. Implement ViaItem and a via-drop tool (V key while routing).
32. Implement board-outline drawing on Edge.Cuts (rect / polygon / circle).
33. Wire up DRC: minimum width, clearance, courtyard overlap. Issues into the console dock.
34. Implement Gerber export per layer using gerbonara.
35. Implement Excellon drill export (PTH and NPTH separated).
36. Acceptance test: scripted board with a routed trace, exported, parsed by gerbv via subprocess.

9.3 The footprint-assignment step

The schematic doesn't carry footprints by default. Each SymbolInstance has an optional footprint_id that may be filled by (a) the symbol's default_footprint, (b) the LLM IR, or (c) the user via the assignment dialog. Before opening the PCB editor, every instance whose ref designator the user wants on the board must have a footprint.

```
# ui/dialogs/footprint_assignment.py
```

```
from PySide6.QtWidgets import (
    QDialog, QVBoxLayout, QTableWidget, QTableWidgetItem, QComboBox,
    QDialogButtonBox, QLabel,
)
from pcbsmith.services.builtin_library import _SYMBOLS, _FOOTPRINTS


class FootprintAssignmentDialog(QDialog):
    def __init__(self, project, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Assign footprints")
        self.resize(800, 500)
        self.project = project
        v = QVBoxLayout(self)
        v.addWidget(QLabel("Pick a footprint for every component on the
schematic:"))
        sch = project.schematics[0]
        self.table = QTableWidget(len(sch.symbols), 4)
        self.table.setHorizontalHeaderLabels(["Ref", "Symbol", "Value",
"Footprint"])
        self.combos: dict[str, QComboBox] = {}

        for row, inst in enumerate(sch.symbols):
            sym = _SYMBOLS[inst.symbol_id]
            self.table.setItem(row, 0, QTableWidgetItem(inst.reference))
            self.table.setItem(row, 1, QTableWidgetItem(sym.name))
            self.table.setItem(row, 2, QTableWidgetItem(inst.value))
            combo = QComboBox()
            for fid, fp in sorted(_FOOTPRINTS.items()):
                combo.addItem(f"{fp.name}  ({fid})", fid)
            current = inst.footprint_id or sym.default_footprint
            if current:
                idx = combo.findData(current)
                if idx >= 0: combo.setCurrentIndex(idx)
            self.table.setCellWidget(row, 3, combo)
            self.combos[inst.id] = combo

        v.addWidget(self.table)
        bb = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
        bb.accepted.connect(self.accept); bb.rejected.connect(self.reject)
        v.addWidget(bb)

    def apply_to_project(self):
        sch = self.project.schematics[0]
        for inst in sch.symbols:
            inst.footprint_id = self.combos[inst.id].currentData()
```

9.4 FootprintItem rendering

A footprint draws on multiple layers simultaneously. Pads draw on F.Cu and F.Mask. Silkscreen polylines draw on F.SilkS. Courtyard draws on F.CrtYd. The painter has to respect the layer panel's visibility settings: invisible layers don't render.

```
# ui/graphics/footprint_item.py
```

```
from PySide6.QtCore import QRectF, Qt
from PySide6.QtGui import QPainter, QPen, QBrush, QColor
from PySide6.QtWidgets import QGraphicsItem
from pcbsmith.core.library import Footprint, PadShape, PadType
from pcbsmith.core.board import FootprintInstance
from .units import nm_to_scene, scene_to_nm

LAYER_COLORS = {
    "F.Cu":      QColor("#C83232"),
    "B.Cu":      QColor("#4D7FC4"),
    "F.SilkS":   QColor("#E8DC95"),
    "B.SilkS":   QColor("#AAAAAA"),
    "F.Mask":    QColor("#777777"),
    "F.Paste":   QColor("#888888"),
    "F.CrtYd":   QColor("#7F7F7F"),
    "Edge.Cuts": QColor("#FFFFFF"),
}


class FootprintItem(QGraphicsItem):
    def __init__(self, footprint: Footprint, instance: FootprintInstance,
                 layer_visibility: dict[str, bool], parent=None):
        super().__init__(parent)
        self.footprint = footprint
        self.instance = instance
        self.layer_visibility = layer_visibility
        self.setFlag(QGraphicsItem.ItemIsSelectable, True)
        self.setFlag(QGraphicsItem.ItemIsMovable, not instance.locked)
        sx, sy = nm_to_scene(instance.position)
        self.setPos(sx, sy)
        self.setRotation(instance.rotation_deg)
        self._b = self._compute_bounding()

    def _compute_bounding(self) -> QRectF:
        xs, ys = [0.0], [0.0]
        for pad in self.footprint.pads:
            sx, sy = nm_to_scene(pad.position)
            half_w = (pad.size_x_nm / 25_400) / 2
            half_h = (pad.size_y_nm / 25_400) / 2
            xs += [sx - half_w, sx + half_w]; ys += [sy - half_h, sy + half_h]
        for poly in self.footprint.silkscreen_polylines +
self.footprint.courtyard_polylines:
            for pt in poly:
                sx, sy = nm_to_scene(pt); xs.append(sx); ys.append(sy)
        m = 30
        return QRectF(min(xs) - m, min(ys) - m,
                      max(xs) - min(xs) + 2*m, max(ys) - min(ys) + 2*m)
```

```
    def boundingRect(self): return self._b

    def paint(self, painter: QPainter, option, widget=None):
        painter.setRenderHint(QPainter.Antialiasing)
        # Pads
        for pad in self.footprint.pads:
            self._paint_pad(painter, pad)
        # Silkscreen
        if self.layer_visibility.get("F.SilkS", True):
            painter.setPen(QPen(LAYER_COLORS["F.SilkS"], 4))
            for poly in self.footprint.silkscreen_polylines:
                for a, c in zip(poly, poly[1:]):
                    painter.drawLine(*nm_to_scene(a), *nm_to_scene(c))
        # Courtyard (dotted)
        if self.layer_visibility.get("F.CrtYd", True):
            pen = QPen(LAYER_COLORS["F.CrtYd"], 1, Qt.DotLine)
            painter.setPen(pen)
            for poly in self.footprint.courtyard_polylines:
                for a, c in zip(poly, poly[1:]):
                    painter.drawLine(*nm_to_scene(a), *nm_to_scene(c))

        # Reference designator
        painter.setPen(QPen(LAYER_COLORS["F.SilkS"], 1))
        painter.drawText(0, 0, self.instance.reference)

    def _paint_pad(self, painter: QPainter, pad):
        sx, sy = nm_to_scene(pad.position)
        w = pad.size_x_nm / 25_400
        h = pad.size_y_nm / 25_400
        painter.setBrush(QBrush(LAYER_COLORS["F.Cu"]))
        painter.setPen(Qt.NoPen)
        if pad.shape == PadShape.RECT:
            painter.drawRect(QRectF(sx - w/2, sy - h/2, w, h))
        elif pad.shape == PadShape.OVAL:
            painter.drawEllipse(QRectF(sx - w/2, sy - h/2, w, h))
        elif pad.shape == PadShape.CIRCLE:
            painter.drawEllipse(QRectF(sx - w/2, sy - w/2, w, w))
        # Drill (PTH)
        if pad.drill_diameter_nm > 0:
            d = pad.drill_diameter_nm / 25_400
            painter.setBrush(QBrush(QColor("#202020")))
            painter.drawEllipse(QRectF(sx - d/2, sy - d/2, d, d))
```

9.5 Ratsnest

The ratsnest is a set of straight lines, one per net, showing the not-yet-routed connections the user still needs to make. It is computed as a Minimum Spanning Tree (MST) over the pad positions of each net, using Euclidean distance as edge weight. MSTs minimise total connection length, which is what the user wants to see.

```
# services/ratsnest.py
```

```
from __future__ import annotations
from dataclasses import dataclass
import networkx as nx
from pcbsmith.core.board import Board
from pcbsmith.core.geom import Point


@dataclass(frozen=True)
class RatsnestEdge:
    net_name: str
    a: Point        # pad center 1 (world coords)
    b: Point        # pad center 2 (world coords)
    is_routed: bool # True if at least one trace already covers this edge


def compute_ratsnest(board: Board, net_lookup: dict[str, list[Point]]) ->
list[RatsnestEdge]:
    """
    `net_lookup`: maps net_name → list of pad world-positions on that net.
    The caller assembles this from FootprintInstance positions and pad-to-net
    bindings (see §9.6).
    """
    edges: list[RatsnestEdge] = []
    for net_name, points in net_lookup.items():
        if len(points) < 2: continue
        g = nx.Graph()
        for i, p in enumerate(points): g.add_node(i, pt=p)
        for i in range(len(points)):
            for j in range(i + 1, len(points)):
                pa, pb = points[i], points[j]
                d2 = (pa.x - pb.x) ** 2 + (pa.y - pb.y) ** 2
                g.add_edge(i, j, weight=d2)
        mst = nx.minimum_spanning_tree(g)
        routed = _routed_edges(board, net_name, points)
        for i, j in mst.edges:
            edges.append(RatsnestEdge(
                net_name=net_name,
                a=points[i], b=points[j],
                is_routed=(i, j) in routed or (j, i) in routed,
            ))
    return edges


def _routed_edges(board, net_name, points) -> set[tuple[int, int]]:
    """For each pair of pads on the net, check whether traces already span them.

    The simple version just sees if any trace touches both pad locations.
    A more accurate version walks the trace graph and runs connectivity per net.
```

```
    For Phase 2 the simple version is fine; it understates how much is routed
    but never overstates, and that's the safe direction.
    """
    pos_set = {(p.x, p.y) for p in points}
    routed = set()
    # ... see Chapter 11 for the full implementation
    return routed
```

```

```

```
# ui/graphics/ratsnest_item.py
```

```
from PySide6.QtCore import QRectF, Qt
from PySide6.QtGui import QPen, QColor
from PySide6.QtWidgets import QGraphicsItem
from pcbsmith.services.ratsnest import RatsnestEdge
from .units import nm_to_scene


class RatsnestItem(QGraphicsItem):
    """Renders all ratsnest edges as thin dotted lines."""
    def __init__(self, edges: list[RatsnestEdge]):
        super().__init__()
        self.edges = edges
        self.setZValue(-100)   # under everything
        self._b = self._compute_bounding()

    def _compute_bounding(self):
        xs, ys = [0.0], [0.0]
        for e in self.edges:
            ax, ay = nm_to_scene(e.a); bx, by = nm_to_scene(e.b)
            xs += [ax, bx]; ys += [ay, by]
        return QRectF(min(xs), min(ys), max(xs)-min(xs), max(ys)-min(ys))

    def boundingRect(self): return self._b

    def paint(self, painter, option, widget=None):
        pen = QPen(QColor("#808080"), 0, Qt.DashLine)
        painter.setPen(pen)
        for e in self.edges:
            if e.is_routed: continue
            painter.drawLine(*nm_to_scene(e.a), *nm_to_scene(e.b))
```

```

```

9.6 Mapping pads to nets

Pads on the board need to know which net they are on. The mapping flows from the schematic: a SymbolInstance on the schematic and a FootprintInstance on the board share a reference designator (R1, U1). The schematic's netlist tells you R1 pin 1 is on net VCC. The library's R_0603 footprint says pad number 1 is at position (-0.825, 0). Therefore the pad at R1's pin 1 is on VCC.

```
# services/board_netmap.py
```

```
from __future__ import annotations
from pcbsmith.core.geom import Point
from pcbsmith.core.board import Board
from pcbsmith.core.schematic import Schematic
from pcbsmith.core.library import Symbol, Footprint
from pcbsmith.core.netops import derive_netlist
from math import cos, sin, radians


def pad_world_position(fp_inst, pad) -> Point:
    """Rotate the pad's footprint-local position into world coords."""
    theta = radians(fp_inst.rotation_deg)
    cx, sy = cos(theta), sin(theta)
    lx, ly = pad.position.x, pad.position.y
    rx = int(round(lx * cx - ly * sy))
    ry = int(round(lx * sy + ly * cx))
    if fp_inst.side == "bottom":
        rx = -rx   # mirror across X axis
    return Point(fp_inst.position.x + rx, fp_inst.position.y + ry)


def build_pad_to_net(
    board: Board,
    schematic: Schematic,
    symbols: dict[str, Symbol],
    footprints: dict[str, Footprint],
) -> dict[str, list[Point]]:
    """Return {net_name: [pad world positions on that net]}."""
    nets = derive_netlist(schematic, symbols)
    sym_inst_by_ref = {s.reference: s for s in schematic.symbols}
    fp_inst_by_ref  = {f.reference: f for f in board.footprints}

    out: dict[str, list[Point]] = {n.name: [] for n in nets}
    for net in nets:
        for pc in net.pins:
            sinst = next(s for s in schematic.symbols if s.id ==
pc.symbol_instance_id)
            ref = sinst.reference
            finst = fp_inst_by_ref.get(ref)
            if finst is None: continue   # not placed on board yet
            fp = footprints[finst.footprint_id]
            pad = next((p for p in fp.pads if p.number == pc.pin_number), None)
            if pad is None: continue
            out[net.name].append(pad_world_position(finst, pad))
    return out
```

```
```

9.7 Manual routing tool

Click on a pad to start a trace. As the user moves the mouse, the application draws a preview from the start to the cursor with 45° corners (the standard PCB convention for hand-routed boards). Click to commit the segment and continue from the new endpoint. Click on a destination pad to end. Press V to drop a via and switch layers. Press ESC to cancel.

```
# ui/graphics/route_tool.py (sketch)
```

```
from PySide6.QtCore import Qt, QPointF
from pcbsmith.core.board import Trace
from pcbsmith.core.geom import Point
from pcbsmith.core.ids import new_id
from .units import scene_to_nm


def route_45(start: QPointF, end: QPointF) -> list[QPointF]:
    """Two-segment 45/90 route from start to end. The first segment is the
    longer of dx/dy at 0/90; the diagonal segment closes the gap."""
    dx = end.x() - start.x(); dy = end.y() - start.y()
    if abs(dx) >= abs(dy):
        elbow = QPointF(end.x() - (abs(dy) * (1 if dx >= 0 else -1)), start.y())
    else:
        elbow = QPointF(start.x(), end.y() - (abs(dx) * (1 if dy >= 0 else -1)))
    return [start, elbow, end]


class RouteTool:
    def __init__(self, canvas, layer: str = "F.Cu", width_nm: int = 200_000):
        self.canvas = canvas
        self.layer = layer
        self.width_nm = width_nm
        self.start: QPointF | None = None
        self.preview = None

    def mouse_press(self, e):
        sp = self.canvas.mapToScene(e.position().toPoint())
        if self.start is None:
            self.start = sp; return True
        # Commit a Trace from self.start through 45-elbow to sp
        pts = route_45(self.start, sp)
        trace = Trace(
            id=new_id(),
            layer=self.layer,
            width_nm=self.width_nm,
            points=[scene_to_nm(p.x(), p.y()) for p in pts],
        )
        self.canvas.board.traces.append(trace)
        # Continue from sp.
        self.start = sp
        self.canvas.refresh_all()
        return True

    def key_press(self, e):
        if e.key() == Qt.Key_Escape:
            self.start = None; return True
        if e.key() == Qt.Key_V:
```

```
            self._drop_via(); return True
        return False

    def _drop_via(self):
        """Drop a via at self.start, flip the active layer F.Cu ↔ B.Cu."""
        from pcbsmith.core.board import Via, ViaType
        if self.start is None: return
        nm = scene_to_nm(self.start.x(), self.start.y())
        via = Via(
            id=new_id(), type=ViaType.THROUGH, position=nm,
            drill_nm=300_000, diameter_nm=600_000,
            layer_from="F.Cu", layer_to="B.Cu",
        )
        self.canvas.board.vias.append(via)
        self.layer = "B.Cu" if self.layer == "F.Cu" else "F.Cu"
        self.canvas.refresh_all()
```

```
```

Snapping is the half of routing that matters. When the cursor is within ~10 mil of a pad center, snap the trace endpoint exactly to the pad. Without this, your traces look fine on screen but DRC reports thousands of near-miss connections, and Gerbers fail at the fab. Implement pad-snap from day one.

9.8 Board outline

The board outline lives on the Edge.Cuts layer as a closed polygon. Provide three ways to draw it: rectangle (drag two corners), polygon (click vertices, double-click to close), and circle (drag center to radius). Internally all three produce a list of Point closing back to the first.

```
# services/board_outline.py
```

```
from pcbsmith.core.geom import Point


def rect_outline(x0_nm: int, y0_nm: int, x1_nm: int, y1_nm: int) -> list[Point]:
    return [
        Point(x0_nm, y0_nm), Point(x1_nm, y0_nm),
        Point(x1_nm, y1_nm), Point(x0_nm, y1_nm),
        Point(x0_nm, y0_nm),
    ]

def circle_outline(cx_nm: int, cy_nm: int, r_nm: int, n: int = 64) ->
list[Point]:
    """Approximate a circle with `n` segments. Closed."""
    import math
    pts = []
    for i in range(n):
        a = 2 * math.pi * i / n
        pts.append(Point(cx_nm + int(r_nm * math.cos(a)),
                         cy_nm + int(r_nm * math.sin(a))))
    pts.append(pts[0])
    return pts
```

```
```

9.9 Gerber export with gerbonara

Gerbonara abstracts the format details. You construct GerberFile objects per layer, add Line / Arc / Region graphics for the copper, traces, mask, paste, silk, and outline, then save. Use X2/X3 attributes so the manufacturer knows which file is which copper layer, what nets the pads belong to, and which components are which.

```
# services/exporters/gerber.py
```

```
from __future__ import annotations
from pathlib import Path
from gerbonara.graphic_objects import Line, Region, Flash
from gerbonara.apertures import CircleAperture, RectangleAperture
from gerbonara import GerberFile
from gerbonara.utils import MM
from pcbsmith.core.board import Board
from pcbsmith.core.library import Footprint, PadShape


# Per-layer filename suffix expected by most fabs.
LAYER_FILENAMES = {
    "F.Cu":      "-F_Cu.gbr",
    "B.Cu":      "-B_Cu.gbr",
    "F.SilkS":   "-F_Silkscreen.gbr",
    "B.SilkS":   "-B_Silkscreen.gbr",
    "F.Mask":    "-F_Mask.gbr",
    "B.Mask":    "-B_Mask.gbr",
    "F.Paste":   "-F_Paste.gbr",
    "B.Paste":   "-B_Paste.gbr",
    "Edge.Cuts": "-Edge_Cuts.gbr",
}


def export_gerbers(board: Board,
                   footprints: dict[str, Footprint],
                   project_name: str,
                   out_dir: Path) -> list[Path]:
    out_dir.mkdir(parents=True, exist_ok=True)
    written: list[Path] = []

    for layer in board.layers:
        if layer.name not in LAYER_FILENAMES: continue
        gbr = GerberFile()
        gbr.import_settings = gbr.original_unit = MM   # Gerbonara API; enforce
mm

        # Write objects appropriate to this layer.
        if layer.type.value == "signal":
            _emit_signal_layer(gbr, board, layer.name, footprints)
        elif layer.type.value == "silkscreen":
            _emit_silk_layer(gbr, board, layer.name, footprints)
        elif layer.type.value == "solder_mask":
            _emit_mask_layer(gbr, board, layer.name, footprints)
        elif layer.type.value == "solder_paste":
            _emit_paste_layer(gbr, board, layer.name, footprints)
        elif layer.type.value == "edge_cuts":
            _emit_edge_cuts(gbr, board)
```

```
        path = out_dir / f"{project_name}{LAYER_FILENAMES[layer.name]}"
        gbr.save(str(path))
        written.append(path)
    return written


def _emit_signal_layer(gbr, board, layer_name, footprints):
    # Traces
    for trace in board.traces:
        if trace.layer != layer_name: continue
        ap = CircleAperture(trace.width_nm / 1e6, unit=MM)
        for a, c in zip(trace.points, trace.points[1:]):
            gbr.objects.append(Line(
                x1=a.x / 1e6, y1=a.y / 1e6,
                x2=c.x / 1e6, y2=c.y / 1e6,
                aperture=ap, polarity_dark=True, unit=MM,
            ))

    # Pads (flashed)
    for fpi in board.footprints:
        fp = footprints[fpi.footprint_id]
        for pad in fp.pads:
            if layer_name not in pad.layers: continue
            from .gerber_helpers import pad_aperture, pad_world_position
            ap = pad_aperture(pad)
            pos = pad_world_position(fpi, pad)
            gbr.objects.append(Flash(
                x=pos.x / 1e6, y=pos.y / 1e6,
                aperture=ap, polarity_dark=True, unit=MM,
            ))

    # Vias (flashed copper rings; the drill is in the Excellon file)
    from gerbonara.apertures import CircleAperture
    for via in board.vias:
        if layer_name not in (via.layer_from, via.layer_to): continue
        ap = CircleAperture(via.diameter_nm / 1e6, unit=MM)
        gbr.objects.append(Flash(
            x=via.position.x / 1e6, y=via.position.y / 1e6,
            aperture=ap, polarity_dark=True, unit=MM,
        ))


def _emit_edge_cuts(gbr, board):
    if not board.outline: return
    ap = CircleAperture(0.1, unit=MM)
    for a, c in zip(board.outline, board.outline[1:]):
        gbr.objects.append(Line(
            x1=a.x / 1e6, y1=a.y / 1e6,
            x2=c.x / 1e6, y2=c.y / 1e6,
```

```
            aperture=ap, polarity_dark=True, unit=MM,
        ))


def _emit_silk_layer(gbr, board, layer_name, footprints):
    ...   # walk silkscreen polylines on each footprint, emit Line objects

def _emit_mask_layer(gbr, board, layer_name, footprints):
    ...   # mask is the inverse of pads with mask_margin offset; use Flash on
each pad

def _emit_paste_layer(gbr, board, layer_name, footprints):
    ...   # paste is a copy of SMT pads on F.Paste / B.Paste with paste_margin
offset
```

```
```

Always run a Gerber viewer test in CI. Install gerbv on the CI runner and run gerbv --no-gui --export=png on every fixture board. If gerbv refuses to load a file, the Gerber is malformed; fail the build.

9.10 Excellon drill export

Excellon is the older NC drill format. Two files: one for plated through holes (PTH), one for non-plated holes (NPTH). Modern fabs accept the XNC subset (a strict, well-specified slice of Excellon defined jointly by Ucamco, KiCad, and others); use XNC unless a specific fab needs vendor Excellon. Gerbonara writes XNC.

```
# services/exporters/excellon.py
```

```
from pathlib import Path
from gerbonara.excellon import ExcellonFile
from gerbonara.aperture_macros.parse import GenericMacros  # tools defined inline
from pcbsmith.core.board import Board
from pcbsmith.core.library import Footprint, PadType


def export_drills(board: Board,
                  footprints: dict[str, Footprint],
                  project_name: str,
                  out_dir: Path) -> dict[str, Path]:
    out_dir.mkdir(parents=True, exist_ok=True)
    pth = ExcellonFile(); pth.is_plated = True
    npth = ExcellonFile(); npth.is_plated = False

    # Pads with drills
    for fpi in board.footprints:
        fp = footprints[fpi.footprint_id]
        for pad in fp.pads:
            if pad.drill_diameter_nm <= 0: continue
            from .gerber_helpers import pad_world_position
            pos = pad_world_position(fpi, pad)
            target = npth if pad.type == PadType.NPTH else pth
            target.drill(pos.x / 1e6, pos.y / 1e6,
                         diameter=pad.drill_diameter_nm / 1e6, unit="mm")

    # Vias → PTH only
    for via in board.vias:
        pth.drill(via.position.x / 1e6, via.position.y / 1e6,
                  diameter=via.drill_nm / 1e6, unit="mm")

    out_pth  = out_dir / f"{project_name}-PTH.drl"
    out_npth = out_dir / f"{project_name}-NPTH.drl"
    pth.save(str(out_pth)); npth.save(str(out_npth))
    return {"pth": out_pth, "npth": out_npth}
```

```
```

9.11 Putting it together: the export-all action

The user clicks File → Export Manufacturing. The application produces a complete release package the user can zip and email to a fab.

```
# services/exporters/manufacturing.py
```

```
from pathlib import Path
import shutil, zipfile
from .gerber import export_gerbers
from .excellon import export_drills
from .bom import export_bom
from .pnp import export_pnp


def export_manufacturing(board, project, out_dir: Path) -> Path:
    out_dir.mkdir(parents=True, exist_ok=True)
    fps = {fp.id: fp for fp in _all_footprints(project)}
    name = project.name

    files = []
    files.extend(export_gerbers(board, fps, name, out_dir))
    drills = export_drills(board, fps, name, out_dir)
    files.extend(drills.values())
    files.append(export_bom(project, out_dir / f"{name}-BOM.csv"))
    files.append(export_pnp(board, fps, out_dir / f"{name}-Position.csv"))

    zip_path = out_dir / f"{name}-manufacturing.zip"
    with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as z:
        for f in files: z.write(f, arcname=f.name)
    return zip_path


def _all_footprints(project):
    """Resolve every Footprint referenced by the project. Walks built-in and
KiCad libs."""
    ...
```

```

```

9.12 Phase 2 acceptance test

```
# tests/integration/test_phase2_acceptance.py
```

```
import subprocess, zipfile, tempfile
from pathlib import Path
from pcbsmith.core.geom import Point
from pcbsmith.core.board import Trace, FootprintInstance
from pcbsmith.core.ids import new_id
from pcbsmith.services.board_setup import new_board
from pcbsmith.services.board_outline import rect_outline
from pcbsmith.services.exporters.manufacturing import export_manufacturing
from pcbsmith.services.builtin_library import _FOOTPRINTS
import pytest


@pytest.mark.skipif(shutil_which("gerbv") is None, reason="gerbv not installed")
def test_two_pad_board_exports_and_renders(tmp_path):
    project = make_simple_two_pad_project()      # in tests/conftest.py
    board = new_board("smoke")
    board.outline = rect_outline(0, 0, 20_000_000, 10_000_000)   # 20×10 mm
    board.footprints = [
        FootprintInstance(id=new_id(), footprint_id="stdlib:R_0603",
                          reference="R1", value="10k",
                          position=Point.from_mm(5, 5)),
        FootprintInstance(id=new_id(), footprint_id="stdlib:R_0603",
                          reference="R2", value="10k",
                          position=Point.from_mm(15, 5)),
    ]
    board.traces = [
        Trace(id=new_id(), layer="F.Cu", width_nm=200_000,
              points=[Point.from_mm(5.825, 5), Point.from_mm(14.175, 5)])
    ]
    project.boards = [board]

    out = tmp_path / "export"
    zip_path = export_manufacturing(board, project, out)
    assert zip_path.exists()

    # Render with gerbv to confirm validity.
    front = next(out.glob("*-F_Cu.gbr"))
    r = subprocess.run(["gerbv", "--no-gui", "--export=png",
                        "--output=" + str(out / "front.png"), str(front)],
                       capture_output=True, text=True)
    assert r.returncode == 0, r.stderr


def shutil_which(name):
    import shutil; return shutil.which(name)
```

9.13 Phase 2 gotchas

- **Forgetting the Edge.Cuts file.** Without an Edge.Cuts Gerber, the fab does not know where to cut your board. They will reject the order or, worse, guess.
- **Mask layers are inverse.** Solder mask is the COVERING; pads are openings in it. Your F.Mask Gerber renders pad-shaped holes that the fab subtracts from the mask. Easy to invert by accident.
- **Bottom-side mirroring.** Footprints on the bottom side render mirrored on B.Cu, B.SilkS, etc. Build the mirror into the gerber emitter, not the data model.
- **Drill files in inches by default.** Excellon historical default is inches with 2.4 fixed-point. Most modern fabs prefer mm, but always include the explicit METRIC header. Gerbonara handles this if you pass unit="mm".
- **Trace endpoints not snapped to pad centers.** Routes that visually look connected may have a 0.001 mm gap. The fab's CAM tool will report a DRC error and slow the order. Snap on commit.

11. ERC and DRC Engines

Phase 0 ships a minimal ERC. This chapter covers the production engine you build alongside Phase 2 (DRC) and refine in Phase 4 (ERC). Both engines are pure functions that take a model and return a list of issues. They live in services/erc.py and services/drc.py.

11.1 The full ERC issue list

An ERC issue is one of about a dozen fixed codes. Use codes (E001 … E020) so the user can search documentation and disable individual rules. Severities are advisory — the user can downgrade WARNING to INFO if they really mean to do something unusual.

Code	Severity	Rule
E001	ERROR	Duplicate reference designator (two R1s, two U2s, etc.).
E002	WARNING	Pin not connected to a net and no NoConnect marker placed.
E003	ERROR	Two driving outputs on the same net.
E004	WARNING	POWER_IN pin on a net with no POWER_OUT and no power label.
E005	WARNING	Wire endpoint coincident with a pin tip but pin is not on the wire's net (likely a near-miss connection).
E006	ERROR	NetLabel attached to no wire (orphan label).
E007	INFO	Net containing exactly one pin (“stub” net).
E008	WARNING	Net name reused for two electrically distinct nets in different sheets.
E009	ERROR	Component placed but no symbol_id resolves (broken library reference).
E010	WARNING	Output and POWER_IN on the same net (often valid, e.g. enable pulled to VCC, but worth flagging).
E011	WARNING	Bidirectional pins driving each

		other with no clear direction (review).
E012	WARNING	Reference designator does not match the symbol's reference_prefix (R for resistor, etc.).
E013	INFO	Symbol has fields containing TBD or ?, ?, suggesting incomplete entry.
E014	WARNING	Pull-up / pull-down on a net with no driver (likely intentional but worth flagging).
E015	ERROR	Two NoConnect markers on the same point.

11.2 ERC implementation pattern

Each rule is a function that takes the schematic and returns a list of ErcIssue. The main run_erc loops through registered rules. Make rule registration data-driven so users can disable individual rules in project settings.

```
# services/erc.py (production version)
```

```
from __future__ import annotations
from typing import Callable
from .erc_rules import (
    rule_duplicate_refs, rule_unconnected_pins, rule_output_conflicts,
    rule_power_unsourced, rule_orphan_label, rule_stub_net,
    rule_unresolved_symbol, rule_ref_prefix_mismatch,
)

ALL_RULES: dict[str, Callable] = {
    "E001": rule_duplicate_refs,
    "E002": rule_unconnected_pins,
    "E003": rule_output_conflicts,
    "E004": rule_power_unsourced,
    "E006": rule_orphan_label,
    "E007": rule_stub_net,
    "E009": rule_unresolved_symbol,
    "E012": rule_ref_prefix_mismatch,
    # ...
}


def run_erc(schematic, symbols, *, disabled: set[str] | None = None):
    disabled = disabled or set()
    issues = []
    for code, fn in ALL_RULES.items():
        if code in disabled: continue
        try:
            issues.extend(fn(schematic, symbols))
        except Exception as exc:
            # A buggy rule should not poison the rest.
            issues.append(_internal_error(code, exc))
    return issues
```

```
```

11.3 The DRC issue list

DRC operates on the Board. It is heavier than ERC because most checks are geometric. Use pyclipper / shapely for the polygon work and an R-tree (rtree) for spatial lookups; full O(n²) loops over traces will be unusable on a board with a few hundred segments.

Code	Severity	Rule
D001	ERROR	Trace width below

		DesignRules.min_trace_width_nm.
D002	ERROR	Trace-to-trace clearance below DesignRules.min_clearance_nm (different nets).
D003	ERROR	Trace-to-pad clearance below min_clearance_nm (different nets).
D004	ERROR	Pad-to-pad clearance below min_clearance_nm (different nets).
D005	ERROR	Trace short circuit (two nets touching).
D006	ERROR	Drill smaller than min_drill_nm.
D007	ERROR	Annular ring smaller than min_annular_ring_nm.
D008	WARNING	Silkscreen overlapping a pad (silk gets trimmed at the fab; warn before they trim it).
D009	WARNING	Courtyard overlap between two footprints (not a fatal error but a placement smell).
D010	ERROR	Footprint outside the board outline.
D011	ERROR	Trace outside the board outline.
D012	ERROR	Via diameter smaller than min_via_diameter_nm.
D013	WARNING	Unrouted ratsnest edge (electrical connection still missing).
D014	ERROR	Trace endpoints not on grid (not strictly an error in 2026 fabs but signals sloppy editing).
D015	WARNING	Acute angles in trace (< 90°). Some fabs reject; most just warn.

11.4 Spatial indexing

Build an R-tree once at the start of DRC, query it for every check. R-tree lookups are O(log n) per query, so even an O(n) outer loop with R-tree queries inside is fine for tens of thousands of objects.

```
# services/drc_index.py
```

```
from __future__ import annotations
from rtree import index
from pcbsmith.core.board import Board, Trace, Via


def build_index(board: Board):
    """Returns an rtree index keyed by an integer id, plus a lookup dict."""
    idx = index.Index()
    items: dict[int, object] = {}
    counter = 0
    for trace in board.traces:
        for a, c in zip(trace.points, trace.points[1:]):
            x0, x1 = sorted((a.x, c.x)); y0, y1 = sorted((a.y, c.y))
            margin = trace.width_nm
            idx.insert(counter, (x0 - margin, y0 - margin, x1 + margin, y1 +
margin))
            items[counter] = ("trace_seg", trace, a, c)
            counter += 1
    for via in board.vias:
        r = via.diameter_nm
        idx.insert(counter,
                   (via.position.x - r, via.position.y - r,
                    via.position.x + r, via.position.y + r))
        items[counter] = ("via", via)
        counter += 1
    return idx, items
```

```
```

11.5 Clearance check using polygon offset

The robust way to check clearance: inflate every conducting object by half the minimum clearance (pyclipper's polygon offset), then test for any pair of inflated polygons that intersect AND have different net IDs. If two inflated polygons intersect, the originals are closer than the clearance allows.

```
# services/drc_clearance.py
```

```
from __future__ import annotations
import pyclipper
from pcbsmith.core.board import Board
from .drc_issue import DrcIssue, Severity

CLIPPER_SCALE = 1000   # pyclipper requires integer paths; nm → fm-resolution


def trace_segment_polygon(p1_nm, p2_nm, width_nm) -> list[tuple[int, int]]:
    """A trace is a fat line; turn it into a polygon (capsule) for offsetting."""
    import math
    dx, dy = p2_nm.x - p1_nm.x, p2_nm.y - p1_nm.y
    length = math.hypot(dx, dy)
    if length == 0: return []
    nx, ny = -dy / length, dx / length
    half = width_nm / 2
    return [
        (int(p1_nm.x + nx * half), int(p1_nm.y + ny * half)),
        (int(p2_nm.x + nx * half), int(p2_nm.y + ny * half)),
        (int(p2_nm.x - nx * half), int(p2_nm.y - ny * half)),
        (int(p1_nm.x - nx * half), int(p1_nm.y - ny * half)),
    ]


def inflate(poly: list[tuple[int, int]], delta_nm: int) -> list[list[tuple[int,
int]]]:
    pco = pyclipper.PyclipperOffset()
    pco.AddPath([(x * CLIPPER_SCALE, y * CLIPPER_SCALE) for (x, y) in poly],
                pyclipper.JT_ROUND, pyclipper.ET_CLOSEDPOLYGON)
    return [[(x // CLIPPER_SCALE, y // CLIPPER_SCALE) for (x, y) in p]
            for p in pco.Execute(delta_nm * CLIPPER_SCALE)]


def check_clearance(board: Board, net_lookup) -> list[DrcIssue]:
    """
    `net_lookup`: function (object) -> net_name | None.
    Returns DRC violations where two objects on different nets are too close.
    """
    min_cl = board.design_rules.min_clearance_nm
    objects = list(_iter_conductive(board))
    issues = []
    for i in range(len(objects)):
        for j in range(i + 1, len(objects)):
            net_i = net_lookup(objects[i]); net_j = net_lookup(objects[j])
            if net_i is not None and net_i == net_j:
                continue   # same net; clearance does not apply
            if _too_close(objects[i], objects[j], min_cl):
                issues.append(DrcIssue("D002", Severity.ERROR,
```

```
                    f"Clearance violation between {objects[i]} and {objects[j]}",
                    where=""))
    return issues
```

```
```

In practice, use the rtree. The pseudo-code above is O(n²). For a real board, replace the outer loops with an rtree query: for each object, query the index for objects whose inflated bounding boxes overlap, then run the precise polygon test only on those candidates. This makes DRC feasible on boards with thousands of segments.

11.6 Connectivity check

DRC's most user-visible job: "did I forget to route something?" Walk the trace graph per net using union-find on segment endpoints; for each net, every pad must end up in the same equivalence class. Anything else means an unrouted pin and produces a D013 issue with the net name.

```
# services/drc_connectivity.py
```

```
from pcbsmith.core.netops import UnionFind


def check_connectivity(board, pads_by_net, traces_by_net, vias_by_net):
    issues = []
    for net_name, pad_positions in pads_by_net.items():
        if len(pad_positions) < 2: continue
        uf = UnionFind()
        for p in pad_positions: uf.add((p.x, p.y))
        for trace in traces_by_net.get(net_name, []):
            for a, c in zip(trace.points, trace.points[1:]):
                uf.add((a.x, a.y)); uf.add((c.x, c.y))
                uf.union((a.x, a.y), (c.x, c.y))
        for via in vias_by_net.get(net_name, []):
            # treat via positions as connection points
            uf.add((via.position.x, via.position.y))
        # Now: every pad on this net must share a union-find root.
        roots = {uf.find((p.x, p.y)) for p in pad_positions}
        if len(roots) > 1:
            issues.append((net_name, "unrouted"))
    return issues
```

```
```

11.7 Live DRC vs batch DRC

Two modes. Batch DRC runs on demand (Tools → Run DRC), checks everything, takes as long as it takes. Live DRC runs continuously while the user routes, highlights the active trace red when it would create a violation, and gates the commit. Live DRC must be fast (sub-100 ms per cursor move) so it only checks the trace currently being drawn, not the whole board.

- Phase 2: ship batch DRC only. Performance budget: 5 seconds for a 200-component board.
- Phase 5: add live DRC. Performance budget: 50 ms per cursor move.

11.8 Reporting issues

The DRC and ERC engines emit a uniform issue type. The console dock shows them in a list with double-click-to-locate. Add a Markers layer on the canvas that overlays issue indicators (red circles) on the offending objects so the user sees them in context, not just in the dock.

```
# core/issues.py
```

```
from dataclasses import dataclass
from enum import Enum

class Severity(str, Enum):
    INFO = "info"; WARNING = "warning"; ERROR = "error"


@dataclass(frozen=True)
class Issue:
    code: str
    severity: Severity
    message: str
    where: str
    location_nm: tuple[int, int] | None = None   # for spatial markers
```

```

```

12. Exporters

Every form of output the application produces lives under services/exporters/. This chapter is the reference for each one. Most were sketched in Chapter 9; this is the complete picture, including the formats Phase 9 ships only as stubs and Phase 4 fills in (KiCad, BOM, PnP).

12.1 Gerber RS-274X with X2/X3 attributes

The current Gerber specification is revision 2024.05 from Ucamco (the format owner). Three relevant variants:

- **RS-274X (Extended Gerber).** Pre-2014. The minimum modern fabs accept. Embedded apertures, no metadata.
- **X2.** Adds attributes (%TF, %TA, %TO) describing layer function, pad function, net names. Backwards-compatible: a fab tool that does not understand the attributes ignores them.
- **X3.** Adds component placement attributes so the same Gerber set carries pick-and-place information. Use it; the fab will appreciate it.

Gerbonara emits X2 by default and X3 if you include component attributes. Always include attributes. They turn a Gerber set from “here are some pictures” into “here is the entire board, machine-readable.”

```
# services/exporters/gerber_attributes.py
```

```
"""Common X2/X3 attribute keys we set on every layer."""

LAYER_FUNCTION = {
    "F.Cu":      "Copper,L1,Top",
    "B.Cu":      "Copper,L2,Bot",
    "F.SilkS":   "Legend,Top",
    "B.SilkS":   "Legend,Bot",
    "F.Mask":    "Soldermask,Top",
    "B.Mask":    "Soldermask,Bot",
    "F.Paste":   "Paste,Top",
    "B.Paste":   "Paste,Bot",
    "Edge.Cuts": "Profile,NP",   # NP = non-plated
}

# Apply per layer at file scope:
#   %TF.FileFunction,Copper,L1,Top*%
#   %TF.FilePolarity,Positive*%
#   %TF.GenerationSoftware,PCBSmith,0.x.x*%

# Apply per pad:
#   %TO.N,GND*%        (net name)
#   %TO.P,U1,3*%       (pad on component U1 pin 3)
#   %TA.AperFunction,SMDPad,Copper*%
```

```
```

12.2 Excellon / XNC

Two drill files: PTH (plated) and NPTH (non-plated). XNC is the strict subset most fabs prefer in 2026; gerbonara writes XNC by default. Conventions to follow:

- Header: M48, METRIC, semicolon comments only, no FMAT.
- Tools: T01, T02, … with diameters in mm. Define every tool in the header before use.
- Body: G05 followed by tool selects and X/Y coordinates.
- Footer: M30.
- File extensions: .drl is universal; .xnc is more correct but rarer.

12.3 SVG documentation

An SVG of the schematic for blog posts, lab notes, or homework. Produce one SVG per schematic sheet, sized to fit the sheet block, with a CSS theme block at the top so users can re-skin without editing the geometry. Do the same for the board on a per-layer basis and combine into a multi-layer assembly view.

```
# services/exporters/svg.py (sketch)
```

```
def export_schematic_svg(sch, symbols, out_path):
    parts = [
        '<svg xmlns="http://www.w3.org/2000/svg" version="1.1" \n'
        '     viewBox="-100 -100 1000 1000">',
        '<style>',
        '  .symbol { stroke: #992A1A; fill: none; stroke-width: 4; }',
        '  .pin    { stroke: #992A1A; stroke-width: 4; }',
        '  .wire   { stroke: #0F771A; stroke-width: 4; }',
        '  .label  { font-family: Arial; font-size: 10px; fill: black; }',
        '  .ref    { font-family: Arial; font-size: 8px; fill: black; }',
        '</style>',
    ]
    # ... emit paths from each symbol body, pin, wire, label
    parts.append('</svg>')
    out_path.write_text("\n".join(parts), encoding="utf-8")
```

```
```

Don't use SVG to round-trip data. Schematic and board SVGs are write-only. They are not a format we round-trip through. If a user wants their schematic back, they open the .pcbsmith project. If they have an SVG and want to import it, they need to redraw it in PCBSmith — SVGs do not carry nets, pin numbers, or component identity.

12.4 PDF documentation

A multi-page PDF of the schematic + board renders. Use reportlab or matplotlib's PDF backend; for full-fidelity, render the QGraphicsScene to a QPainter writing into a QPdfWriter. The Qt route gives consistency with the on-screen render.

```
# services/exporters/pdf.py
```

```
from PySide6.QtCore import QSizeF
from PySide6.QtGui  import QPainter, QPdfWriter, QPageSize


def export_schematic_pdf(canvas, out_path):
    writer = QPdfWriter(str(out_path))
    writer.setPageSize(QPageSize(QPageSize.A3))
    writer.setResolution(600)
    painter = QPainter(writer)
    canvas.scene_obj.render(painter)
    painter.end()
```

```
```

12.5 Bill of Materials (BOM)

CSV with one row per distinct part. Columns: Reference designators (comma-joined), Quantity, Symbol, Value, Footprint, Manufacturer, MPN, Datasheet. Match what JLCPCB / PCBWay's import dialog expects so the user can paste the file straight in.

```
# services/exporters/bom.py
```

```
import csv
from collections import defaultdict
from pathlib import Path


def export_bom(project, out_path: Path):
    sch = project.schematics[0]
    groups: dict[tuple, list] = defaultdict(list)
    for inst in sch.symbols:
        if inst.reference.startswith("#"): continue   # power ports etc.
        key = (inst.symbol_id, inst.value, inst.footprint_id or "",
               inst.fields.get("Manufacturer", ""), inst.fields.get("MPN", ""))
        groups[key].append(inst.reference)

    with out_path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["Reference", "Qty", "Symbol", "Value", "Footprint",
                    "Manufacturer", "MPN", "Datasheet"])
        for (sym, value, fp, mfr, mpn), refs in groups.items():
            inst = next(i for i in sch.symbols if i.reference == refs[0])
            w.writerow([", ".join(sorted(refs)), len(refs),
                        sym, value, fp, mfr, mpn, inst.fields.get("Datasheet",
""")])
    return out_path
```

```
```

12.6 Pick-and-place (Position file)

CSV with one row per FootprintInstance. Columns: Designator, Val, Package, Mid X, Mid Y, Rotation, Layer (top/bottom). Format aligns with JLCPCB's expectations; other fabs may want tweaks. Coordinates in mm with 4 decimals.

```
# services/exporters/pnp.py
```

```
import csv
from pathlib import Path


def export_pnp(board, footprints, out_path: Path):
    with out_path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["Designator", "Val", "Package", "Mid X", "Mid Y", "Rotation",
"Layer"])
        for fpi in board.footprints:
            fp = footprints[fpi.footprint_id]
            w.writerow([fpi.reference, fpi.value, fp.name,
                        f"{fpi.position.x / 1e6:.4f}",
                        f"{fpi.position.y / 1e6:.4f}",
                        f"{fpi.rotation_deg:.2f}",
                        "top" if fpi.side == "top" else "bottom"])
    return out_path
```

```

```

12.7 KiCad export

Export the project as a KiCad project (.kicad_pro + .kicad_sch + .kicad_pcb). KiCad's format is S-expressions; use sexpdata to construct trees. The mapping table from Chapter 4.7 lists every entity correspondence. Two practical notes:

- **Library references must already be KiCad-named.** If the user used “stdlib:R”, the export wraps it as “device:R” (KiCad's name) plus a project-specific symbol library file containing the definition. Phase 4's library import / export goes both ways.
- **Coordinates are in mm in KiCad files, with millimetre-resolution rounding.** Convert nm → mm at the boundary; KiCad rejects positions with too many decimal places.

```
# services/exporters/kicad.py (sketch)
```

```
from sexpdata import dumps, Symbol as Sym


def export_board_to_kicad(board, project, out_path):
    expr = [Sym("kicad_pcb"),
            [Sym("version"), 20240108],
            [Sym("generator"), "pcbsmith"],
            _general_block(board),
            _layers_block(board),
            _setup_block(board),
            *_footprints_blocks(board),
            *_traces_blocks(board),
            *_vias_blocks(board),
            *_zones_blocks(board),
            *_outline_blocks(board)]
    out_path.write_text(dumps(expr, pretty_print=True), encoding="utf-8")


def _layers_block(board):
    return [Sym("layers"), *([
        l.ordinal, l.name, Sym(l.type.value), l.name
    ] for l in board.layers)]
```

```

```

14. Phase 4 — KiCad Library Import and Polish

Phase 4 turns PCBSmith from “interesting prototype” into “useable for real boards.” The hard-coded 25-part library is replaced with the full KiCad 9 library (about 20,000 symbols and 15,000 footprints). The user gains a library browser, a custom symbol/footprint editor, BOM/PnP integration, KiCad-format export, and a long list of small UX improvements.

Phase 4 milestone. Open a project, click in the library palette, type “STM32F411”, see thirty candidate symbols. Pick the one with the right package. Place it. The footprint comes too. Export to KiCad and open the result in KiCad 9 — it should be a valid KiCad project.

14.1 KiCad library file formats

KiCad uses S-expressions for all its files. The library files you need to read in Phase 4 are:

Extension	Contains	Notes
.kicad_sym	Symbol library file	One file holds many symbols. Replaced .lib in v6+.
.kicad_mod	Single footprint	One file per footprint. Lives in a *.pretty/ directory.
fp-lib-table	List of footprint library directories	Maps a nickname to a path. KiCad uses these to find footprints.
sym-lib-table	List of symbol library files	Same idea for symbols.

14.2 Reading .kicad_sym

Use sexpdata. The structure is straightforward; the trick is that the same parser must tolerate small format differences across KiCad point releases. Test against fixtures from at least two KiCad versions.

```
# services/kicad_import.py (symbol reader)
```

```
from __future__ import annotations
from pathlib import Path
import sexpdata
from pcbsmith.core.geom import Point
from pcbsmith.core.library import (
    Symbol, Pin, PinDirection, PinElectricalType,
)


def read_kicad_symbol_library(path: Path) -> list[Symbol]:
    """Parse a .kicad_sym file and return all symbols inside."""
    with path.open(encoding="utf-8") as fh:
        tree = sexpdata.load(fh)
    if not isinstance(tree, list) or _car(tree) != "kicad_symbol_lib":
        raise ValueError(f"{path} is not a kicad_symbol_lib")
    out = []
    for child in tree[1:]:
        if not isinstance(child, list): continue
        if _car(child) == "symbol":
            out.append(_parse_symbol(child, library_name=path.stem))
    return out


def _parse_symbol(sym_expr, library_name: str) -> Symbol:
    name = _str(sym_expr[1])
    description = _find_property(sym_expr, "Description") or ""
    keywords    = (_find_property(sym_expr, "ki_keywords") or "").split()
    ref_prefix = (_find_property(sym_expr, "Reference") or "U").rstrip("?")

    pins: list[Pin] = []
    for child in sym_expr:
        if isinstance(child, list) and _car(child) == "symbol":
            # Sub-symbol holding the actual graphical data and pins.
            for sub in child:
                if isinstance(sub, list) and _car(sub) == "pin":
                    pins.append(_parse_pin(sub))
    body_polylines = _collect_polylines(sym_expr)

    return Symbol(
        id=f"kicad:{library_name}:{name}",
        name=name, description=description, keywords=keywords,
        reference_prefix=ref_prefix, pins=pins,
        body_polylines=body_polylines,
    )


def _parse_pin(p) -> Pin:
    elec = _str(p[1]).lower()
```

```
    style = _str(p[2]).lower()    # not used; line/inverted/clock
    at_expr = _find_child(p, "at")
    x, y, angle = float(at_expr[1]), float(at_expr[2]), int(at_expr[3])
    length_expr = _find_child(p, "length")
    length_mm = float(length_expr[1])
    name = _str(_find_child(p, "name")[1])
    number = _str(_find_child(p, "number")[1])

    direction = {0: PinDirection.RIGHT, 90: PinDirection.UP,
                 180: PinDirection.LEFT, 270: PinDirection.DOWN}[angle]
    elec_map = {
        "input": PinElectricalType.INPUT,
        "output": PinElectricalType.OUTPUT,
        "bidirectional": PinElectricalType.BIDI,
        "tri_state": PinElectricalType.TRISTATE,
        "passive": PinElectricalType.PASSIVE,
        "power_in": PinElectricalType.POWER_IN,
        "power_out": PinElectricalType.POWER_OUT,
        "open_collector": PinElectricalType.OPEN_COLL,
        "open_emitter": PinElectricalType.OPEN_EMIT,
        "unconnected": PinElectricalType.NC,
        "unspecified": PinElectricalType.UNSPECIFIED,
    }
    return Pin(
        number=number, name=name,
        position=Point.from_mm(x, y),
        direction=direction,
        length_nm=int(length_mm * 1_000_000),
        electrical_type=elec_map.get(elec, PinElectricalType.UNSPECIFIED),
    )


def _car(expr): return _str(expr[0]) if expr else ""
def _str(token): return token.value() if isinstance(token, sexpdata.Symbol) else
str(token)
def _find_child(expr, tag):
    for c in expr:
        if isinstance(c, list) and _car(c) == tag: return c
    raise KeyError(tag)
def _find_property(expr, name) -> str | None:
    for c in expr:
        if isinstance(c, list) and _car(c) == "property" and _str(c[1]) == name:
            return _str(c[2])
    return None
def _collect_polylines(expr): return []   # walk "polyline" children of sub-
symbols
```

14.3 Reading .kicad_mod

Footprints follow the same shape: an outer (footprint NAME) form with (pad), (fp_line), (fp_circle), (fp_arc), and (model) children. Pads have shapes (rect, oval, circle, roundrect, custom), an at position, layer list, optional drill, optional primitives. Map them to our Pad model.

```
# services/kicad_import.py (footprint reader)
```

```
from pcbsmith.core.library import Footprint, Pad, PadShape, PadType


def read_kicad_footprint(path: Path) -> Footprint:
    with path.open(encoding="utf-8") as fh:
        tree = sexpdata.load(fh)
    if _car(tree) != "footprint":
        raise ValueError(f"{path} is not a footprint")
    name = _str(tree[1])
    pads: list[Pad] = []
    silk_polys, courtyard_polys, fab_polys = [], [], []
    for child in tree[2:]:
        if not isinstance(child, list): continue
        head = _car(child)
        if head == "pad":
            pads.append(_parse_pad(child))
        elif head == "fp_line":
            layer = _str(_find_child(child, "layer")[1])
            seg = [Point.from_mm(*[float(x) for x in _find_child(child, "start")
[1:3]]),
                   Point.from_mm(*[float(x) for x in _find_child(child, "end")
[1:3]])]
            (silk_polys if "SilkS" in layer
             else courtyard_polys if "CrtYd" in layer
             else fab_polys).append(seg)
    return Footprint(
        id=f"kicad:{path.parent.stem}:{name}",
        name=name,
        pads=pads,
        silkscreen_polylines=silk_polys,
        courtyard_polylines=courtyard_polys,
        fab_polylines=fab_polys,
    )


def _parse_pad(p) -> Pad:
    number = _str(p[1])
    type_str = _str(p[2])      # smd, thru_hole, np_thru_hole, connect
    shape_str = _str(p[3])     # rect, circle, oval, roundrect, custom
    at = _find_child(p, "at")
    x, y = float(at[1]), float(at[2])
    rotation = float(at[3]) if len(at) > 3 else 0.0
    sz = _find_child(p, "size")
    sx, sy = float(sz[1]), float(sz[2])
    layers_expr = _find_child(p, "layers")
    layers = [_str(t) for t in layers_expr[1:]]
    drill_diameter_nm = 0
    try:
```

```
        drl = _find_child(p, "drill")
        drill_diameter_nm = int(round(float(drl[1]) * 1_000_000))
    except KeyError: pass

    type_map = {"smd": PadType.SMT_TOP,
                "thru_hole": PadType.THT,
                "np_thru_hole": PadType.NPTH,
                "connect": PadType.CONNECT}
    shape_map = {"rect": PadShape.RECT, "circle": PadShape.CIRCLE,
                 "oval": PadShape.OVAL, "roundrect": PadShape.RRECT,
                 "custom": PadShape.CUSTOM}
    return Pad(
        number=number, type=type_map[type_str], shape=shape_map[shape_str],
        position=Point.from_mm(x, y), rotation_deg=rotation,
        size_x_nm=int(sx * 1_000_000), size_y_nm=int(sy * 1_000_000),
        drill_diameter_nm=drill_diameter_nm, layers=layers,
    )
```

14.4 Library cache and lazy loading

Loading every KiCad symbol on startup is slow and wastes memory — a typical user uses fewer than 100 distinct parts in a project. Build a metadata cache: scan every .kicad_sym file once, extract just (id, name, keywords, ref_prefix), persist to a single JSON cache file. Load full Symbol objects on demand.

```
# services/library_cache.py
```

```
import json
from pathlib import Path
from xdg_base_dirs import xdg_cache_home


CACHE_DIR = xdg_cache_home() / "pcbsmith"


def build_symbol_index(library_root: Path) -> dict:
    index: dict = {}
    for kicad_sym in library_root.rglob("*.kicad_sym"):
        for sym in read_kicad_symbol_library(kicad_sym):
            index[sym.id] = {
                "name": sym.name, "keywords": sym.keywords,
                "file": str(kicad_sym), "ref_prefix": sym.reference_prefix,
            }
    CACHE_DIR.mkdir(parents=True, exist_ok=True)
    (CACHE_DIR / "symbol_index.json").write_text(
        json.dumps(index, indent=2), encoding="utf-8")
    return index


def search(query: str, index: dict) -> list[str]:
    q = query.lower()
    hits = []
    for sid, meta in index.items():
        if q in meta["name"].lower() or any(q in k.lower() for k in
meta["keywords"]):
            hits.append(sid)
    return hits[:200]    # cap so the UI does not have to render thousands
```

```
```

14.5 Library browser UI

Replace the simple library palette with a two-pane browser. Left pane: collapsible list of libraries (“Device”, “MCU_ST_STM32F4”, “Connector_USB”, …). Right pane: filtered symbol list with a thumbnail. Above both: a global search box that searches across all libraries by name, keywords, and category. Double-click activates the place tool with that symbol.

14.6 Custom symbol and footprint editor

Sometimes the user has a part that is not in any library. Provide a focused editor for creating one. Two tabs: Symbol (place pins, draw body) and Footprint (place pads, draw silk/courtyard). Save to the project's library/ folder. Treat editor changes as first-class undoable operations.

14.7 Footprint compatibility check

Once a symbol has more than one candidate footprint (an LM358 op-amp comes in SOIC-8, DIP-8, MSOP-8, …), validate that the user's choice has matching pin numbers. If the symbol's pins are 1–8 and the footprint's pads are 1–8, you're fine; if the footprint has different pad numbers (e.g. uses A1…D2 for a BGA), warn or refuse the assignment until the user maps explicitly.

Chapter 15 — Phase 5: Auto-routing via FreeRouting

Phase 5 turns the schematic-driven ratsnest into actual copper. Custom auto-routing is a multi-decade research problem — the algorithms that ship in commercial tools (Cadence, Altium, Zuken) represent person-centuries of work and are still imperfect. We will not write our own. We will integrate FreeRouting, the de-facto open-source Specctra-compatible auto-router, and treat it as an external service.

The contract is simple: PCBSmith exports the current Board to a Specctra DSN file, invokes FreeRouting, and reads the resulting SES file back into the Board as a set of Trace and Via objects. From the user's perspective the round-trip is hidden behind a single "Auto-route" button with a progress dialog and a Cancel control.

Why FreeRouting, specifically. It speaks Specctra DSN/SES, the industry-standard hand-off format, which means the integration is a file-format problem rather than an algorithmic one. It is actively maintained (current 2.2.x), available as both a standalone JAR and a Docker image (ghcr.io/freerouting/freerouting), and supports headless operation via --gui.enabled=false. KiCad, EasyEDA, and Eagle all use it the same way.

15.1 Integration Architecture

There are four pieces of code involved in Phase 5, each in its own module so that they can be tested independently:

Module	Responsibility
services/dsn_exporter.py	Serialise a Board (with a Schematic-derived netlist) to a DSN string.
services/ses_importer.py	Parse an SES file and produce a list of Trace/Via additions to apply to the Board.
services/freerouting_runner.py	Subprocess wrapper. Locates the JAR or Docker image, builds the command line, captures stdout/stderr, parses the progress log, exposes a cancellable async interface.
ui/route_panel.py	Dock panel with passes/threads/clearance overrides, Run/Cancel buttons, progress bar, log view, and a Diff-and-Apply confirmation dialog.

Single-direction data flow. DSN export reads from Board, never writes. SES import produces a BoardDiff (list of Trace/Via additions and removals) which the UI applies via a single AutoRouteCommand on the QUndoStack. This means the user can Undo an auto-route in one keystroke — an enormously valuable property when the result is unsatisfactory, which it often is.

15.2 Prerequisites and Discovery

FreeRouting is a Java application, not a Python package. PCBSmith therefore needs either (a) a Java Runtime Environment 21+ on the user's machine plus the freerouting JAR, or (b) Docker with the published image. We support both, with a discovery routine that prefers Docker (more reproducible, no Java version drift) and falls back to a local JAR.

```
# services/freerouting_runner.py — discovery
```

```
from __future__ import annotations
import shutil
import subprocess
from dataclasses import dataclass
from pathlib import Path
from typing import Literal

FR_DOCKER_IMAGE = "ghcr.io/freerouting/freerouting:2.2.2"
FR_JAR_ENV = "PCBSMITH_FREEROUTING_JAR"  # explicit override
FR_JAR_BUNDLED = Path(__file__).parents[2] / "vendor" / "freerouting.jar"

Backend = Literal["docker", "jar", "missing"]

@dataclass(frozen=True)
class FrEnv:
    backend: Backend
    detail: str  # human-readable: image tag, jar path, or reason missing

def discover() -> FrEnv:
    # 1. Docker
    if shutil.which("docker"):
        try:
            r = subprocess.run(
                ["docker", "image", "inspect", FR_DOCKER_IMAGE],
                capture_output=True, timeout=5,
            )
            if r.returncode == 0:
                return FrEnv("docker", FR_DOCKER_IMAGE)
            # Image not pulled, but docker is present — we can pull on demand
            return FrEnv("docker", FR_DOCKER_IMAGE + " (will pull on first run)")
        except subprocess.TimeoutExpired:
            pass

    # 2. Local JAR (env override, then bundled, then PATH)
    import os
    candidates = []
    if env := os.environ.get(FR_JAR_ENV):
        candidates.append(Path(env))
    candidates.append(FR_JAR_BUNDLED)
    for jar in candidates:
        if jar.is_file():
            if not shutil.which("java"):
                return FrEnv("missing", "JAR found but no `java` on PATH (need
JRE 21+).")
            return FrEnv("jar", str(jar))
```

```
    return FrEnv("missing", "No Docker image and no FreeRouting JAR. See
settings.")
```

> **Versioning matters.** FreeRouting 2.x is not 100% wire-compatible with 1.x. SES files emitted by 1.x boards can fail to round-trip through 2.x in edge cases (custom padstacks, numeric pad names). Always pin the Docker image tag and the bundled JAR to a single tested major version. We use 2.2.2 throughout this spec.

15.3 DSN Exporter — Theory

Specctra DSN is an old (1990s) S-expression format originally from Cooper & Chyan Technology. It describes a board as a set of structures: layers, padstacks, components, and a network of nets-to-pin assignments. FreeRouting cares about a subset of the format. The minimal valid DSN we need to emit has six top-level sections:

- (parser …) — file-format metadata, units, resolution.
- (resolution um N) — geometric resolution (we use micrometres at 10× — i.e. 0.1 µm).
- (structure …) — board outline, layers, default rules (clearance, trace width).
- (placement …) — component instances with their position, side, and rotation.
- (library …) — image (footprint) and padstack definitions.
- (network …) — nets, with the list of pins (component-ref + pin-number) on each.

> **Coordinate system.** Specctra's Y axis points UP. PCBSmith's internal Board (like KiCad's) has Y pointing DOWN. The exporter must flip Y on every coordinate it emits, and the importer must flip Y back. Forgetting this gives you a board that auto-routes fine but comes back mirrored. Write a unit test for this on day one.

15.4 DSN Exporter — Implementation Sketch

The exporter is a pure function: Board + Schematic-derived Net list → str. We use a tiny S-expression writer rather than sexpdata.dumps because DSN is whitespace-and-comment-sensitive and we want full control of indentation.

```
# services/dsn_exporter.py
```

```
from __future__ import annotations
from io import StringIO
from typing import Iterable
from core.board import Board, FootprintInstance
from core.schematic import Net
from core.library import Footprint, Pad

# DSN uses 0.1 µm = 100 nm resolution. PCBSmith uses int64 nm internally,
# so divide by 100 (rounding) to get DSN units.
DSN_RES_NM = 100


def _q(s: str) -> str:
    """Quote an identifier if it contains whitespace or starts with a digit."""
    if not s:
        return '""'
    if s[0].isdigit() or any(c.isspace() for c in s):
        return f'"{s}"'
    return s


def _coord(nm: int) -> int:
    return (nm + DSN_RES_NM // 2) // DSN_RES_NM  # round half-up


class DsnWriter:
    def __init__(self) -> None:
        self.buf = StringIO()
        self.indent = 0

    def open(self, name: str, *args: str) -> None:
        self.buf.write("  " * self.indent)
        self.buf.write("(" + name)
        for a in args:
            self.buf.write(" " + a)
        self.buf.write("\n")
        self.indent += 1

    def close(self) -> None:
        self.indent -= 1
        self.buf.write("  " * self.indent)
        self.buf.write(")\n")

    def line(self, *parts: str) -> None:
        self.buf.write("  " * self.indent)
        self.buf.write("(" + " ".join(parts) + ")\n")

    def text(self) -> str:
```

```
        return self.buf.getvalue()
```

15.4.1 Emitting structure and rules

The structure block declares each signal layer FreeRouting may route on, the board outline (a closed polygon on Edge.Cuts in our model), and the default design rules. Rules can be overridden per-net-class but PCBSmith Phase 5 ships with a single default class.

```
def write_structure(w: DsnWriter, board: Board) -> None:
```

```
    w.open("structure")
    # Signal layers in stack order, top-side first
    for layer in board.signal_layers():
        w.open("layer", _q(layer.name))
        w.line("type", "signal")
        w.line("property", f"(index {layer.stack_index})")
        w.close()

    # Board outline as a single polygon on PCB layer
    outline = board.edge_cuts_polygon_nm()  # list[(x_nm, y_nm)]
    coords = " ".join(f"{_coord(x)} {_coord(-y)}" for x, y in outline)
    w.line("boundary", f"(polygon pcb 0 {coords})")

    # Default design rules
    rules = board.design_rules
    w.open("rule")
    w.line("width", str(_coord(rules.default_trace_width_nm)))
    w.line("clearance", str(_coord(rules.default_clearance_nm)))
    # FreeRouting needs separate clearance for trace-via and via-via;
    # use the same value unless DesignRules has overrides.
    w.close()
```

```
    w.close()
```

Why we negate Y here. _coord(-y) is the Y flip discussed above. PCBSmith stores y growing downward (screen convention). DSN expects y growing upward (math convention). The flip happens exactly once, on the boundary between the two coordinate systems.

15.4.2 Emitting placement and library

Each FootprintInstance becomes one component in the placement block and one image-reference. Each unique Footprint becomes one image and contributes its pads' padstacks to the library. We deduplicate padstacks across footprints — a 0603 pad is the same in every passive.

```
def write_library(w: DsnWriter, board: Board) -> dict[str, str]:
```

```
    """Returns a mapping {pad_geom_key: padstack_id} so placement can reference
them."""
    w.open("library")
    padstacks: dict[str, Pad] = {}  # key -> exemplar Pad

    # First pass: collect unique padstacks, write image (footprint) blocks
    seen_footprints: set[str] = set()
    for inst in board.footprint_instances:
        fp: Footprint = board.library.footprint(inst.footprint_id)
        if fp.id in seen_footprints:
            continue
        seen_footprints.add(fp.id)
        w.open("image", _q(fp.id))
        for pad in fp.pads:
            key = pad.padstack_key()  # shape+size+drill+layer-set
            padstacks.setdefault(key, pad)
            w.line("pin", _q(key), _q(pad.number),
                   str(_coord(pad.position.x_nm)),
                   str(_coord(-pad.position.y_nm)))
        w.close()

    # Second pass: emit padstacks
    for key, pad in padstacks.items():
        w.open("padstack", _q(key))
        for layer in pad.layer_names_for_dsn(board):
            shape_expr = pad.shape_dsn_expr(scale=DSN_RES_NM)
            w.line("shape", f"({shape_expr} {layer})")
        w.line("attach", "off")  # do not allow vias to be placed on pads
        w.close()
    w.close()
```

```
    return {k: k for k in padstacks}
```

15.4.3 Emitting the network

The network block is the heart of the DSN: it tells FreeRouting which pads are electrically the same. A Net contains a list of (component_ref, pin_number) pairs. Order does not matter inside a net but you must include every pad of every connected pin or FreeRouting will leave airwires.

```
def write_network(w: DsnWriter, board: Board, nets: list[Net]) -> None:
```

```
    w.open("network")
    for net in nets:
        if net.name == "" or net.is_no_connect():
            continue
        w.open("net", _q(net.name))
        pins = [
            f"{conn.component_ref}-{conn.pin_number}"
            for conn in net.pin_connections
        ]
        # Wrap pins onto multiple lines for readable diffs
        w.buf.write("  " * w.indent + "(pins")
        for pin in pins:
            w.buf.write(" " + _q(pin))
        w.buf.write(")\n")
        w.close()

    # Default class: every net not in another class
    w.open("class", "default")
    for net in nets:
        if net.name and not net.is_no_connect():
            w.buf.write("  " * w.indent + _q(net.name) + "\n")
    w.line("circuit", "(use_via via_default)")
    w.line("rule", f"(width
{_coord(board.design_rules.default_trace_width_nm)})")
    w.close()
```

```
    w.close()
```

15.4.4 Putting it together

```
def board_to_dsn(board: Board, nets: list[Net], design_name: str = "board") ->
str:
```

```
    w = DsnWriter()
    w.open("pcb", _q(design_name))
    w.line("parser",
           "(string_quote \")",
           "(space_in_quoted_tokens on)",
           "(host_cad PCBSmith)",
           "(host_version 0.1)")
    w.line("resolution", "um", "10")  # 0.1 um per unit
    w.line("unit", "um")
    write_structure(w, board)
    # placement before library is fine; readers do two passes
    write_placement(w, board)
    write_library(w, board)
    write_network(w, board, nets)
    # No (wiring) section — that's where SES will put the routed traces.
    w.close()
```

```
    return w.text()
```

Test this against KiCad. The most useful sanity check on the DSN exporter is to take the same simple board, export it via KiCad's File → Export → Specctra DSN and via PCBSmith's DsnWriter, and run both through FreeRouting. Compare SES outputs. If FreeRouting completes both within similar pass counts and produces similar routing, the exporter is correct enough. Diffing the DSN strings directly is rarely useful because there are many valid serialisations of the same board.

15.5 Running FreeRouting Headlessly

We run FreeRouting in a subprocess, write its log to a file, and parse the progress lines as they appear. The runner is async-friendly: it exposes a started/finished/progress/cancelled signal pattern that the UI consumes via Qt signals (we wrap it in a QThread in the UI module).

```
# services/freerouting_runner.py — runner
```

```
from __future__ import annotations
import re
import subprocess
import threading
from dataclasses import dataclass
from pathlib import Path
from typing import Callable, Optional

@dataclass
class RunOptions:
    max_passes: int = 30
    threads: int = 1   # see callout: multi-threaded optimiser is broken in 2.2.x
    ignore_nets: tuple[str, ...] = ()    # e.g. ("GND",) for hand-routed power
    extra_args: tuple[str, ...] = ()


_PASS_RE = re.compile(r"pass\s+(\d+)\s+of\s+(\d+)", re.IGNORECASE)
_DONE_RE = re.compile(r"(routing finished|completed)", re.IGNORECASE)


def build_command(env: FrEnv, dsn: Path, ses: Path,
                  opt: RunOptions) -> list[str]:
    args = [
        "-de", str(dsn),
        "-do", str(ses),
        "-mp", str(opt.max_passes),
        "-mt", str(opt.threads),
        "--gui.enabled=false",
        "-dct", "0",
    ]
    if opt.ignore_nets:
        args += ["-inc", ",".join(opt.ignore_nets)]
    args += list(opt.extra_args)

    if env.backend == "docker":
        # Mount the working dir so the JAR sees the dsn/ses files
        wd = dsn.parent.resolve()
        return [
            "docker", "run", "--rm", "-i",
            "-v", f"{wd}:/work", "-w", "/work",
            FR_DOCKER_IMAGE,
            "-de", dsn.name, "-do", ses.name,
            "-mp", str(opt.max_passes), "-mt", str(opt.threads),
            "--gui.enabled=false", "-dct", "0",
            *(["-inc", ",".join(opt.ignore_nets)] if opt.ignore_nets else []),
            *opt.extra_args,
        ]
    if env.backend == "jar":
```

```
        return ["java", "-jar", env.detail, *args]
    raise RuntimeError(f"FreeRouting not available: {env.detail}")
```

Threads default to 1. Upstream issue #445 documents that FreeRouting's multi-threaded post-route optimiser produces clearance violations. Default to -mt 1 and only let the user override it from an Advanced section of the dialog with a clear warning. The honesty here matters — silently using a known-broken setting because it is faster will cost the user real money in fab spins.

15.5.1 Streaming progress

FreeRouting writes pass-progress lines to stdout. The runner reads them line-by-line on a background thread and emits a progress callback. Cancellation is implemented by terminating the subprocess; FreeRouting does not respond to SIGINT cleanly, so we send SIGTERM and, after a 2-second grace, SIGKILL.

```
class FrRunner:
```

```
    def __init__(self, env: FrEnv) -> None:
        self.env = env
        self._proc: Optional[subprocess.Popen] = None
        self._cancel = threading.Event()

    def run(self,
            dsn: Path, ses: Path, opt: RunOptions,
            on_progress: Callable[[int, int], None],
            on_log: Callable[[str], None]) -> int:
        cmd = build_command(self.env, dsn, ses, opt)
        on_log(f"$ {' '.join(cmd)}")
        self._proc = subprocess.Popen(
            cmd, stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
            text=True, bufsize=1,
        )
        for line in self._proc.stdout:
            on_log(line.rstrip())
            if self._cancel.is_set():
                self._terminate()
                return -1
            m = _PASS_RE.search(line)
            if m:
                on_progress(int(m.group(1)), int(m.group(2)))
        rc = self._proc.wait()
        return rc

    def cancel(self) -> None:
        self._cancel.set()

    def _terminate(self) -> None:
        if self._proc and self._proc.poll() is None:
            self._proc.terminate()
            try:
                self._proc.wait(timeout=2)
            except subprocess.TimeoutExpired:
```

```
                self._proc.kill()
```

15.6 SES Importer

The Specctra SES (Session) format is the inverse of DSN: a record of routing decisions, primarily a (wiring) block listing every wire segment and via that FreeRouting placed. PCBSmith reads this, converts each wire into a Trace and each via into a Via in our internal model.

SES is also S-expression-based, so we use sexpdata to parse it (read-only consumption is the easy direction — DSN serialisation is the hard one).

```
# services/ses_importer.py
```

```
from __future__ import annotations
import sexpdata
from dataclasses import dataclass
from core.board import Trace, Via, ViaType
from core.geom import Point

@dataclass
class RouteResult:
    new_traces: list[Trace]
    new_vias: list[Via]
    unrouted_nets: list[str]   # nets FreeRouting could not complete


SES_RES_NM = 100  # must match what we wrote in DSN: 0.1 µm per unit


def _sym(s: object) -> str:
    return s.value() if isinstance(s, sexpdata.Symbol) else str(s)


def parse_ses(text: str, layer_lookup: dict[str, str]) -> RouteResult:
    """layer_lookup maps DSN layer names back to PCBSmith Layer ids."""
    tree = sexpdata.loads(text)
    # Top form: (session NAME (base_design ...) (placement ...) (was_is ...)
(routes ...))
    routes = _find(tree, "routes")
    network_out = _find(routes, "network_out")
    new_traces: list[Trace] = []
    new_vias: list[Via] = []
    unrouted: list[str] = []

    for net_form in _find_all(network_out, "net"):
        net_name = _sym(net_form[1])
        for wire in _find_all(net_form, "wire"):
            path = _find(wire, "path")
            # (path LAYER WIDTH x1 y1 x2 y2 ... xn yn)
            layer_dsn = _sym(path[1])
            width_nm = int(path[2]) * SES_RES_NM
            coords = [int(c) * SES_RES_NM for c in path[3:]]
            layer_id = layer_lookup[layer_dsn]
            for i in range(0, len(coords) - 2, 2):
                x1, y1 = coords[i], -coords[i + 1]   # un-flip Y
                x2, y2 = coords[i + 2], -coords[i + 3]
                new_traces.append(Trace(
                    net_name=net_name,
                    layer_id=layer_id,
                    start=Point(x1, y1),
                    end=Point(x2, y2),
```

```
                    width_nm=width_nm,
                ))
        for via_form in _find_all(net_form, "via"):
            padstack = _sym(via_form[1])
            x = int(via_form[2]) * SES_RES_NM
            y = -int(via_form[3]) * SES_RES_NM
            new_vias.append(Via(
                net_name=net_name,
                position=Point(x, y),
                padstack_id=padstack,
                via_type=ViaType.THROUGH,
            ))

    # Unrouted nets are reported in the (network_out (net X (rule ...)) ) form
    # without any (wire) children — detect those.
    for net_form in _find_all(network_out, "net"):
        if not _find_all(net_form, "wire"):
            unrouted.append(_sym(net_form[1]))
```

```
    return RouteResult(new_traces, new_vias, unrouted)
```

Why we don't trust SES blindly. FreeRouting can produce SES output that fails our own DRC: zero-length wires, wires that overshoot pads by sub-resolution amounts, vias placed on top of pads on the same net (legal but ugly). Always run a clean-up pass on the RouteResult before applying it: drop zero-length traces, snap endpoints to pad centres within a tolerance, and run the DRC on the merged board to surface anything else.

15.7 Route Panel and Diff Confirmation

The UI piece exposes three things to the user: a setup form, a live progress view, and a final confirmation dialog. The confirmation step is critical — auto-router output is sometimes worse than manual routing on simple boards, and the user must be able to reject a result without polluting their undo history.

- RoutePanel (QDockWidget) — Max passes (1–500), Threads (1–8 with warning), Net classes to skip, Run/Cancel buttons.
- Progress view — Pass N of M with bar, scrolling log (last 200 lines), elapsed time.
- Diff dialog — "Auto-route added 247 segments and 12 vias across 18 nets, left 2 nets unrouted." Buttons: Apply, Apply and Edit, Discard.

```
# ui/route_panel.py — sketch
```

```
from PySide6.QtCore import QObject, QThread, Signal, Slot
from PySide6.QtWidgets import QDockWidget, QFormLayout, QSpinBox, QPushButton,
QPlainTextEdit, QProgressBar, QWidget, QVBoxLayout
from pathlib import Path
import tempfile
from services import freerouting_runner as fr
from services.dsn_exporter import board_to_dsn
from services.ses_importer import parse_ses


class RouteWorker(QObject):
    progress = Signal(int, int)        # pass_n, pass_total
    log = Signal(str)
    finished = Signal(object)          # RouteResult or None

    def __init__(self, board, nets, opt: fr.RunOptions):
        super().__init__()
        self.board, self.nets, self.opt = board, nets, opt
        self.runner: fr.FrRunner | None = None

    @Slot()
    def run(self):
        env = fr.discover()
        if env.backend == "missing":
            self.log.emit(env.detail)
            self.finished.emit(None)
            return
        with tempfile.TemporaryDirectory() as td:
            dsn = Path(td) / "board.dsn"
            ses = Path(td) / "board.ses"
            dsn.write_text(board_to_dsn(self.board, self.nets))
            self.runner = fr.FrRunner(env)
            rc = self.runner.run(dsn, ses, self.opt,
                                 on_progress=self.progress.emit,
                                 on_log=self.log.emit)
            if rc == 0 and ses.exists():
                layer_lookup = {l.name: l.id for l in self.board.signal_layers()}
                result = parse_ses(ses.read_text(), layer_lookup)
                self.finished.emit(result)
            else:
                self.finished.emit(None)

    @Slot()
    def cancel(self):
        if self.runner:
```

```
            self.runner.cancel()
```

15.8 AutoRouteCommand and Undo Integration

Once the user clicks Apply on the diff dialog, the result is wrapped in a single QUndoCommand so that one Ctrl+Z reverses the entire auto-route. This is more important than it sounds: an auto-route that adds 250 segments would otherwise fill the undo history and push out earlier work.

```
# ui/commands.py — extension
```

```
from PySide6.QtGui import QUndoCommand

class AutoRouteCommand(QUndoCommand):
    def __init__(self, board, result):
        super().__init__("Auto-route")
        self.board = board
        self.added_traces = list(result.new_traces)
        self.added_vias = list(result.new_vias)

    def redo(self):
        self.board.traces.extend(self.added_traces)
        self.board.vias.extend(self.added_vias)
        self.board.bump_revision()

    def undo(self):
        # Remove by identity; safe because we kept references
        for t in self.added_traces:
            self.board.traces.remove(t)
        for v in self.added_vias:
            self.board.vias.remove(v)
```

```
        self.board.bump_revision()
```

15.9 Phase 5 Acceptance Test

The end-to-end test exercises the full pipeline: open the voltage divider from Phase 0, place footprints, run FreeRouting, expect 100% routed, run DRC, expect zero violations. The test skips itself gracefully if FreeRouting is not available in the CI environment.

```
# tests/acceptance/test_phase5_autoroute.py
```

```
import pytest
from services import freerouting_runner as fr
from services.dsn_exporter import board_to_dsn
from services.ses_importer import parse_ses
from cli import open_project
from drc.runner import run_drc


@pytest.mark.acceptance
def test_voltage_divider_autoroute(tmp_path):
    env = fr.discover()
    if env.backend == "missing":
        pytest.skip(env.detail)

    proj = open_project("tests/fixtures/voltage_divider.pcbsmith")
    board, nets = proj.board, proj.derive_netlist()

    dsn = tmp_path / "vd.dsn"
    ses = tmp_path / "vd.ses"
    dsn.write_text(board_to_dsn(board, nets))

    runner = fr.FrRunner(env)
    rc = runner.run(dsn, ses,
                    fr.RunOptions(max_passes=20, threads=1),
                    on_progress=lambda *_: None,
                    on_log=lambda _: None)
    assert rc == 0

    layer_lookup = {l.name: l.id for l in board.signal_layers()}
    result = parse_ses(ses.read_text(), layer_lookup)
    assert result.unrouted_nets == []
    assert len(result.new_traces) >= 3

    board.traces.extend(result.new_traces)
    board.vias.extend(result.new_vias)
    issues = run_drc(board)
```

```
    assert [i for i in issues if i.severity == "error"] == []
```

15.10 Phase 5 Gotchas

- Padstack name collisions. FreeRouting requires unique padstack names. If two footprints define a 0603-shape pad with slightly different sizes, our deduplication key must include the dimensions, not just the shape family. Check pad.padstack_key() carefully.

- Numeric pin numbers. Pin number "1" must be quoted as "1" in DSN — names that start with a digit are syntactically ambiguous. The _q helper handles this; do not bypass it.
- Y-axis flip. Done once on export, once on import. Test by routing a simple two-component net and verifying the trace runs in the visually correct direction.
- Zero-area board outline. If the user has not drawn an Edge.Cuts polygon, board.edge_cuts_polygon_nm() raises. The route panel must check this before enabling the Run button — surfacing a friendly message rather than letting FreeRouting fail with a cryptic stack trace.
- Net name escaping. Power nets often have names like "+3V3" or "GND/AGND". Wrap them in quotes in the DSN; do not rename them silently or the SES import will fail to map them back.
- Long-running calls block the UI. Always run inside a QThread. Always show a Cancel button. Always make the button work (the runner.cancel path above is non-optional).
- Docker first run is slow. Pulling the FreeRouting image takes 30–60 seconds. The progress dialog must distinguish "pulling image" from "routing" so the user does not assume it has hung. Pre-pull on application install if Docker is detected.

Phase 5 acceptance criteria. (a) Voltage-divider acceptance test passes locally and in CI when FreeRouting is available, skips cleanly when it is not. (b) Cancelling mid-route leaves the board untouched and frees the subprocess within 2 seconds. (c) An auto-route with 250+ segments is undone by a single Ctrl+Z. (d) The diff dialog accurately previews the count of additions and the list of unrouted nets before any change is committed.

Chapter 17 — Testing Strategy

PCBSmith is a graphical, file-format-heavy, externally-integrated application with non-trivial geometry. None of those words are good news for testing. The strategy in this chapter is opinionated and prescriptive: the tests we describe are the ones to write first, in this order. Anything beyond is a bonus.

What we are optimising for. Catching regressions in geometry, netlist, and file I/O before they reach the user, while keeping the suite fast enough to run on every commit (under 60 seconds for the unit tier on a laptop). Acceptance tests are slower, run in CI and on demand locally, and exist primarily to keep the manufacturing exports honest.

17.1 The Test Pyramid for PCBSmith

Five tiers, from cheapest to most expensive, with rough budgets:

Tier	Tooling	Budget	What goes here
1. Unit	pytest	<10s	Pure functions: geometry, units, S-expression parsing, validators.
2. Property	Hypothesis	<20s	Algebraic invariants on geom, netlist derivation, IR validators.
3. Snapshot	syrupy	<10s	Stable serialisations: project files, DSN export, BOM/PnP CSVs.
4. UI	pytest-qt	<30s	QGraphicsScene interactions, QUndoStack round-trips, dialogs.
5. Acceptance	pytest + subprocess	<5 min	End-to-end pipelines: voltage-divider netlist→DRC→Gerber→gerbv render.

Tiers 1–4 run on every commit. Tier 5 runs on push to main, on pull-request merge, and nightly. Anything that needs FreeRouting or KiCad CLI tools is marked @pytest.mark.acceptance and skipped automatically when the binary is not on the PATH — see Chapter 15 for the discovery pattern.

17.2 Unit Tests — The Boring Foundations

Unit tests cover the parts of PCBSmith that have no Qt, no subprocess, no file system, and no LLM. That is more code than you might think: the entire core/ package, the netlist derivation, the IR validator, the DSN-S-expression writer, and the unit conversion helpers.

17.2.1 Geometry test patterns

```
# tests/unit/test_geom.py
```

```
from core.geom import Point, Box, snap, mm_to_nm, nm_to_mm

def test_mm_round_trip_exact():
    # Any mm value with up to 4 decimal places must round-trip exactly
    for v_mm in [0, 0.1, 1.27, 2.54, 25.4, -1.5, 0.0001]:
        assert nm_to_mm(mm_to_nm(v_mm)) == v_mm

def test_snap_to_grid_nearest():
    grid_nm = mm_to_nm(0.1)  # 0.1 mm = 100_000 nm
    assert snap(120_000, grid_nm) == 100_000
    assert snap(160_000, grid_nm) == 200_000
    assert snap(150_000, grid_nm) in (100_000, 200_000)  # half-up or half-even

def test_box_contains_corners_inclusive():
    box = Box(Point(0, 0), Point(100, 100))
    assert box.contains(Point(0, 0))
    assert box.contains(Point(100, 100))
```

```
    assert not box.contains(Point(101, 50))
```

17.2.2 Netlist derivation tests

Netlist derivation is the heart of the schematic stage. Bug here corrupts every downstream stage. Test the union-find directly with hand-rolled fixtures, then test the derive_netlist function with a small set of canonical schematics.

```
# tests/unit/test_netlist.py
```

```
from core.netlist import derive_netlist
from tests.fixtures.schematics import voltage_divider, three_resistor_star

def test_voltage_divider_three_nets():
    sch = voltage_divider()
    nets = derive_netlist(sch)
    names = sorted(n.name for n in nets)
    assert names == ["GND", "VIN", "VOUT"]

def test_three_resistor_star_single_centre_net():
    sch = three_resistor_star()
    nets = derive_netlist(sch)
    centre = next(n for n in nets if n.name == "CENTRE")
    assert {(c.component_ref, c.pin_number) for c in centre.pin_connections} == {
        ("R1", "2"), ("R2", "1"), ("R3", "1"),
    }

def test_dangling_wire_creates_unnamed_net():
    sch = voltage_divider()
    sch = sch.with_extra_dangling_wire_at(Point(50_000_000, 50_000_000))
    nets = derive_netlist(sch)
    unnamed = [n for n in nets if not n.name]
```

```
    assert len(unnamed) == 1
```

17.3 Property Tests with Hypothesis

Hypothesis lets us state algebraic properties — things that should be true for every input — and have it generate hundreds of inputs to try to break them. The two highest-value places to use it are geometry primitives and netlist derivation, because both have invariants that are easy to state and hard to enumerate by hand.

17.3.1 Geometry invariants

```
# tests/property/test_geom_props.py
```

```
from hypothesis import given, strategies as st, settings
from core.geom import mm_to_nm, nm_to_mm, snap

finite_mm = st.floats(
    min_value=-1e4, max_value=1e4,
    allow_nan=False, allow_infinity=False, allow_subnormal=False,
)

@settings(max_examples=500)
@given(finite_mm)
def test_mm_to_nm_monotone(v):
    assert mm_to_nm(v) == mm_to_nm(v)            # determinism
    if v >= 0:
        assert mm_to_nm(v) >= 0

@given(st.integers(min_value=-10**12, max_value=10**12),
       st.integers(min_value=1, max_value=10**8))
def test_snap_idempotent(v_nm, grid_nm):
    once = snap(v_nm, grid_nm)
    twice = snap(once, grid_nm)
    assert once == twice

@given(st.integers(min_value=-10**12, max_value=10**12),
       st.integers(min_value=1, max_value=10**8))
def test_snap_within_half_grid(v_nm, grid_nm):
```

```
    assert abs(snap(v_nm, grid_nm) - v_nm) <= grid_nm  # within one grid step
```

What makes a good property test. Idempotence (f(f(x)) == f(x)), inverses (decode(encode(x)) == x), monotonicity (a < b implies f(a) <= f(b)), and bounded error (|f(x) - x| <= ε). If you cannot phrase your invariant in one of these shapes, the function is probably too complex to property-test directly — break it down first.

17.3.2 Netlist derivation invariants

Generate small random schematics — N components, M wires connecting random pairs of pins — and assert structural properties of the resulting netlist.

```
# tests/property/test_netlist_props.py
```

```
from hypothesis import given, strategies as st
from tests.fixtures.gen import schematic_strategy
from core.netlist import derive_netlist

@given(schematic_strategy(max_components=8, max_wires=20))
def test_every_pin_appears_in_exactly_one_net(sch):
    nets = derive_netlist(sch)
    pin_count: dict[tuple, int] = {}
    for net in nets:
        for c in net.pin_connections:
            key = (c.component_ref, c.pin_number)
            pin_count[key] = pin_count.get(key, 0) + 1
    # Every pin in the schematic must be in exactly one net
    for inst in sch.symbol_instances:
        for pin_num in inst.symbol_pin_numbers():
            assert pin_count.get((inst.ref, pin_num), 0) == 1

@given(schematic_strategy(max_components=6, max_wires=15))
def test_renaming_unconnected_label_does_not_change_topology(sch):
    n1 = derive_netlist(sch)
    sch2 = sch.with_relabelled_unconnected_label("X", "Y")
    n2 = derive_netlist(sch2)
    # Topologies must match modulo the renamed net
    canon = lambda nets: sorted(
        sorted((c.component_ref, c.pin_number) for c in n.pin_connections)
        for n in nets
    )
```

```
    assert canon(n1) == canon(n2)
```

Beware Hypothesis flakiness. Property tests find real bugs but they also find rare bugs. If a Hypothesis test fails intermittently, do not @flaky it away. Capture the falsifying example with @example, fix the underlying bug, and only then close the case. An intermittent failure in CI usually means a real concurrency or ordering bug — silencing it is technical debt that compounds.

17.4 Snapshot Tests for Serialisation

Anything that emits a stable text format — the .pcbsmith JSON, DSN, BOM CSV, PnP CSV, KiCad export — should have a snapshot test. The pattern is: build a small canonical Project, serialise it, compare against a checked-in golden file. When intentionally changing the format, regenerate snapshots in one deliberate commit.

```
# tests/snapshot/test_serialisation.py
```

```
from tests.fixtures.projects import voltage_divider_project
from services.project_io import save_project_to_string
from services.dsn_exporter import board_to_dsn
from services.bom import board_to_bom_csv

def test_pcbsmith_json_snapshot(snapshot):
    proj = voltage_divider_project()
    s = save_project_to_string(proj)
    assert s == snapshot(name="voltage_divider.pcbsmith")

def test_dsn_snapshot(snapshot):
    proj = voltage_divider_project()
    s = board_to_dsn(proj.board, proj.derive_netlist())
    assert s == snapshot(name="voltage_divider.dsn")

def test_bom_snapshot(snapshot):
    proj = voltage_divider_project()
    csv = board_to_bom_csv(proj.board)
```

```
    assert csv == snapshot(name="voltage_divider_bom.csv")
```

Determinism is the prerequisite. Snapshot tests demand byte-stable output. That means: sort dict keys, sort lists where order is not semantic, format floats with a fixed pattern, write CRLF or LF consistently, and do not embed timestamps. Adopt these rules in the serialisers — not in the snapshot test setup. A serialiser that emits different bytes on different runs is broken even without snapshot testing.

17.5 UI Tests with pytest-qt

pytest-qt provides a qtbot fixture that owns a QApplication and exposes click/key/wait helpers. The objective of UI tests is not to verify pixels — that way lies misery — but to verify that user gestures produce the expected model changes via the documented chain (tool → command → undo stack → scene refresh).

```
# tests/ui/test_schematic_editor.py
```

```
import pytest
from PySide6.QtCore import QPointF, Qt
from ui.main_window import MainWindow

@pytest.fixture
def main_window(qtbot):
    w = MainWindow()
    qtbot.addWidget(w)
    w.show()
    return w

def test_place_resistor_via_palette(main_window, qtbot):
    w = main_window
    w.library_palette.select_symbol("R")
    w.scene.activate_tool("place")
    pos = QPointF(100, 100)
    w.scene.simulate_click_at_scene_pos(pos)
    sch = w.current_schematic()
    assert len(sch.symbol_instances) == 1
    assert sch.symbol_instances[0].symbol_id == "R"

def test_undo_after_place(main_window, qtbot):
    w = main_window
    w.library_palette.select_symbol("R")
    w.scene.activate_tool("place")
    w.scene.simulate_click_at_scene_pos(QPointF(0, 0))
    qtbot.keyClicks(w, "z", modifier=Qt.ControlModifier)
```

```
    assert len(w.current_schematic().symbol_instances) == 0
```

Helper APIs over event simulation. Notice simulate_click_at_scene_pos rather than qtbot.mouseClick(scene, ...). Mouse-event coordinate transforms inside QGraphicsView are a long-standing source of fragile tests. Expose deterministic helpers on the scene that skip the event loop and call the tool directly. The thing under test is the tool→command path, not Qt's event delivery — keep the test surface as small as possible.

17.6 Acceptance Tests — End-to-End Pipelines

Acceptance tests treat PCBSmith as a black box. They open a project file, run a CLI subcommand or drive the UI through a script, and assert on the external artefacts (Gerber files, DRC report, exported netlist). They are slow, environment-sensitive, and disproportionately valuable: each one covers a thousand lines of code in one go.

17.6.1 The voltage-divider canonical

The voltage divider — VIN → R1 → VOUT, R2 → GND, decoupling cap optional — is our canonical fixture. It exists in three forms: a pre-built .pcbsmith project file, a Python builder function, and an LLM IR JSON snippet. Every phase chapter has an acceptance test that exercises this fixture end-to-end.

```
# tests/acceptance/test_voltage_divider_pipeline.py
```

```
import subprocess, pytest, shutil
from pathlib import Path

@pytest.mark.acceptance
def test_full_pipeline(tmp_path):
    proj = tmp_path / "vd.pcbsmith"
    shutil.copy("tests/fixtures/voltage_divider.pcbsmith", proj)

    # 1. Phase 0: netlist + ERC
    r = subprocess.run(["pcbsmith", "netlist", str(proj)], capture_output=True,
text=True)
    assert r.returncode == 0
    assert "GND" in r.stdout and "VIN" in r.stdout and "VOUT" in r.stdout

    r = subprocess.run(["pcbsmith", "erc", str(proj)], capture_output=True,
text=True)
    assert r.returncode == 0  # zero errors

    # 2. Phase 2: place footprints, run DRC
    r = subprocess.run(["pcbsmith", "auto-place", str(proj)],
capture_output=True)
    assert r.returncode == 0

    # 3. Phase 5: auto-route (skip if FreeRouting absent)
    if shutil.which("docker") or shutil.which("java"):
        r = subprocess.run(["pcbsmith", "auto-route", str(proj),
                            "--passes", "20"], capture_output=True)
        assert r.returncode == 0

    # 4. Phase 7: export Gerbers
    out = tmp_path / "gerbers"
    r = subprocess.run(["pcbsmith", "export-gerbers", str(proj),
                        "-o", str(out)], capture_output=True)
    assert r.returncode == 0
    assert (out / "vd-F_Cu.gbr").exists()
```

```
    assert (out / "vd.drl").exists()
```

17.6.2 Gerber round-trip via gerbv

The single most important acceptance test in the suite. We render PCBSmith's Gerber output via gerbv (open-source Gerber viewer, command-line capable) and compare it against a checked-in golden PNG within a small pixel tolerance. If something breaks the Gerber writer — pad shape changes, layer naming bugs, polarity errors — the diff catches it visually before any user does.

```
# tests/acceptance/test_gerber_render.py
```

```
import subprocess, shutil, pytest
from pathlib import Path
from PIL import Image, ImageChops
from services.gerber_export import write_gerbers
from tests.fixtures.projects import voltage_divider_project

@pytest.mark.acceptance
def test_top_copper_gerber_matches_golden(tmp_path):
    if not shutil.which("gerbv"):
        pytest.skip("gerbv not installed")
    proj = voltage_divider_project()
    write_gerbers(proj.board, tmp_path)
    out_png = tmp_path / "top.png"
    subprocess.run([
        "gerbv", "-x", "png", "-D", "300",
        "-o", str(out_png),
        str(tmp_path / "vd-F_Cu.gbr"),
    ], check=True)

    actual = Image.open(out_png).convert("RGB")
    golden = Image.open("tests/golden/voltage_divider_top.png").convert("RGB")
    diff = ImageChops.difference(actual, golden)
    bbox = diff.getbbox()
    if bbox is None:
        return  # exact match
    # Allow up to 0.1% of pixels to differ (anti-alias jitter between gerbv runs)
    diff_pixels = sum(1 for p in diff.getdata() if p != (0, 0, 0))
    total = actual.width * actual.height
```

```
    assert diff_pixels / total < 0.001, f"{diff_pixels}/{total} pixels differ"
```

Golden images are version-locked to gerbv. A gerbv minor version bump can shift anti-aliasing and break a 0.001 tolerance. Pin gerbv in CI via the OS package manager or a Docker image, and check the version into a comment at the top of the golden image. When regenerating goldens, do it in a dedicated commit titled "chore: regenerate Gerber goldens for gerbv N.M" so the diff is reviewable.

17.7 Testing the LLM Pipeline

The LLM is the only non-deterministic component in PCBSmith. We test the parts around it deterministically (the IR validator, the materialiser, the diff view) and treat the model itself as a fixture: a small set of recorded API responses replayed by a stub backend. Live model calls happen only in a nightly job, not in the per-commit suite.

```
# tests/unit/test_ir_validator.py
```

```
from services.llm.ir import IR, IRComponent, IRConnection, IRNet
from services.llm.validate import validate, V_DUPLICATE_REF, V_UNKNOWN_SYMBOL
from tests.fixtures.library import minimal_lib

def test_duplicate_ref_caught():
    ir = IR(
        components=[
            IRComponent(ref="R1", symbol_id="R", footprint_id="0603",
value="10k"),
            IRComponent(ref="R1", symbol_id="R", footprint_id="0603",
value="20k"),
        ],
        nets=[], connections=[],
    )
    issues = validate(ir, minimal_lib())
    assert any(i.code == V_DUPLICATE_REF for i in issues)

def test_unknown_symbol_caught():
    ir = IR(
        components=[IRComponent(ref="U1", symbol_id="NOT_A_SYMBOL",
                                footprint_id="0603", value="?")],
        nets=[], connections=[],
    )
    issues = validate(ir, minimal_lib())
```

```
    assert any(i.code == V_UNKNOWN_SYMBOL for i in issues)
```

17.7.1 Recorded-response stub backend

```
# tests/fixtures/llm_stub.py
```

```
import json
from pathlib import Path
from services.llm.backend import LlmBackend, LlmResult

class RecordedBackend(LlmBackend):
    """Returns canned responses keyed by prompt hash. Records misses in a
manifest."""
    def __init__(self, fixture_dir: Path):
        self.dir = fixture_dir
        self.manifest = json.loads((fixture_dir / "manifest.json").read_text())

    def request_ir(self, system_prompt: str, user_prompt: str) -> LlmResult:
        import hashlib
        key = hashlib.sha256((system_prompt + "\x00" +
user_prompt).encode()).hexdigest()[:12]
        if key not in self.manifest:
            raise AssertionError(
                f"No recorded LLM response for key {key}. Run nightly recorder.")
        path = self.dir / self.manifest[key]
```

```
        return LlmResult(ir_json=path.read_text(), latency_ms=0)
```

Why not mock at the HTTP layer? We could intercept Anthropic API calls with vcrpy or responses. We don't, because changes to the SDK or to structured-output headers should not silently re-record responses. By stubbing at the LlmBackend interface we test our own contract, not Anthropic's wire format. Wire format compatibility is checked by a single live call in the nightly job.

17.8 Continuous Integration Layout

The CI workflow runs four jobs in parallel and a fifth nightly. The structure is documented in .github/workflows/ci.yml (Chapter 5) and summarised here:

Job	Trigger	Tiers run	Extra services
lint-type	every push	ruff, mypy, import-linter	—
unit	every push	1, 2	—
snapshot	every push	3	—
ui	every push	4	Xvfb on Linux
acceptance	PR merge + nightly	5	gerbv, FreeRouting, KiCad

			CLI
llm-live	nightly only	select 7.1 tests	ANTHROPIC_API_KEY secret

The acceptance job will be flaky if you let it. Pin every tool version (gerbv, FreeRouting JAR, Java JRE, KiCad CLI) and every Python dependency. Treat tool upgrades as deliberate, separate commits. When acceptance flakes, your first hypothesis should be "a transitive dependency drifted", not "the test is bad".

17.9 Test Coverage Targets and What Not to Measure

Coverage is a useful early signal and a misleading late one. Targets for PCBSmith:

- core/, services/ (excluding I/O glue): 90% line coverage. These are pure-logic modules and hitting the target is a matter of writing tests, not heroics.
- ui/: 60% line coverage. Higher targets force pixel-level tests that break with every visual tweak. Coverage in this layer is a smoke test, not a quality measure.
- services/llm/: 80% line coverage on validator and materialiser; live-call code paths are exempt because they are exercised in the nightly job.
- Total project: 80% as a soft floor. Drops below this fail CI; rises above 95% are not required and not rewarded.

Coverage is necessary, not sufficient. A 100%-covered geom module can still be wrong if every test is a tautology. Pair every coverage target with a property-test or snapshot-test budget so that you are not measuring lines-touched but behaviours-pinned. The healthiest signal is mutation testing — cosmic-ray or mutmut against core/, run nightly. If mutants survive, your line coverage is lying to you.

17.10 Bug Reproduction Discipline

Every closed bug must produce a regression test. The test goes in tests/regression/ with a filename that references the issue number. The test exists forever; it is the strongest signal that a class of bug has been retired.

```
# tests/regression/test_issue_0042_yflip_freerouting.py
```

```
"""
Issue #42: Auto-routed boards came back mirrored along Y axis.
Root cause: dsn_exporter omitted the Y negation on board outline coordinates,
so the outline was inverted relative to the placement.
"""
from services.dsn_exporter import board_to_dsn
from tests.fixtures.projects import voltage_divider_project

def test_outline_y_negated_in_dsn():
    proj = voltage_divider_project()
    proj.board.set_edge_cuts_rectangle(0, 0, 50_000_000, 30_000_000)  # 50x30 mm
    dsn = board_to_dsn(proj.board, proj.derive_netlist())
    # The lowest Y in the outline must be -30000_0 (in 0.1um units)
```

```
    assert "0 -300000" in dsn or "-300000 0" in dsn
```

Phase exit criteria. A phase is complete only when (a) its acceptance test passes locally and in CI, (b) the unit tier covers the new modules above the 80% floor, and (c) at least one property test exists for any new pure function that produces structured output. Any unmet criterion is documented as deferred with a tracking issue, not silently shipped.

Chapter 20 — Glossary

Terms used throughout this specification, with the precise PCBSmith meaning where the general PCB-CAD usage is ambiguous. When implementing, prefer these terms in code, comments, and UI strings. Cross-references in italics indicate other glossary entries.

A

Aperture

In the Gerber format, the shape and size used to draw a feature. PCBSmith emits standard apertures (circle, rectangle, oblong) and only emits a macro when no standard suffices. See Chapter 12.

Annular ring

The copper that surrounds a drilled hole on a through-hole pad. Minimum annular ring is a fab-house design rule (typical 0.15 mm); the DRC checks it. See also drill, pad.

Auto-place

Initial placement of footprints based on schematic structure. PCBSmith implements this naively (grid-pack by ref-prefix) in Phase 2; smarter placement is out of scope and explicitly deferred.

Auto-route

Automated trace placement. PCBSmith integrates FreeRouting via DSN/SES rather than implementing its own router. See Chapter 15.

B

BOM (Bill of Materials)

CSV listing every distinct part on the board with quantity, reference list, value, manufacturer part number, and footprint. Required for assembly. PCBSmith groups by (symbol, value, footprint, MPN) — see Chapter 12.

Board

PCBSmith data class representing the physical PCB: layer stack, footprint instances, traces, vias, zones, and design rules. Distinct from Schematic; the two are linked by reference (component ref) and net name only.

C

Clearance

Minimum distance between two copper features on the same layer that belong to different nets. Specified per net-class. The DRC enforces it via polygon offset (pyclipper). See Chapter 11.

Component

A part on the board, instantiated from a Symbol (schematic side) and a Footprint (board side), linked by a designator (e.g. R1, U2). In PCBSmith code, components do not exist as a single class — they are decomposed into SymbolInstance + FootprintInstance with a shared reference.

Copper pour

See zone.

D

Designator (ref)

Short identifier for a component, by convention a letter prefix plus a number: R1, C12, U3. Must be unique within a project. PCBSmith ERC checks uniqueness; the auto-renumber tool can compact gaps.

Design rules

Numeric thresholds that govern DRC: minimum trace width, minimum clearance, minimum drill, minimum annular ring, etc. Stored on Board.design_rules; may be overridden per net-class.

DRC (Design Rule Check)

Geometric verification of a Board against the design rules. PCBSmith runs live (cheap subset, <50 ms during interaction) and batch (full, <5 s for 100×100 mm boards) variants. See Chapter 11.

Drill

A hole through the board, either plated (PTH, electrically connects layers) or non-plated (NPTH, mechanical only). Drill files are emitted in Excellon format (.drl). See Chapter 12.

DSN (Specctra DSN)

Specctra Design — an S-expression-based file format for handing off a board to an auto-router. PCBSmith generates DSN to send to FreeRouting and reads SES back. See Chapter 15.

E

Edge.Cuts

The KiCad-canonical layer name for the board outline. PCBSmith adopts this name for consistency with imported libraries.

ERC (Electrical Rule Check)

Validation of a Schematic for electrical-rule violations: floating power pins, output–output collisions, undriven inputs, duplicate refs, etc. See Chapter 11 for the full code list (E001–E015).

Excellon

Drill-file format originating from Excellon Automation in the 1970s. PCBSmith emits a strict subset compatible with both classic Excellon and the modern XNC variant.

F

F.Cu / B.Cu

KiCad-canonical names for the front (top) and back (bottom) copper layers. PCBSmith uses these names everywhere for parity with KiCad libraries.

Footprint

The physical pad-and-silkscreen pattern that lands a part on the PCB. Distinct from Symbol (the schematic side). One Symbol may map to many footprints (e.g. an op-amp in SOIC-8 or DIP-8). See library.

FreeRouting

Open-source Specctra-compatible auto-router. PCBSmith integrates the current 2.x release via subprocess; see Chapter 15. Available as a platform-independent JAR or as a Docker image.

G

Gerber

Industry-standard file format for fabricating PCBs. PCBSmith emits Gerber X2 format with the 2024.05 revision attributes (%TF, %TO, %TA metadata) per layer. See Chapter 12.

Grid

The discrete spacing to which user input snaps. PCBSmith defaults to 0.1 mm on the schematic and 0.05 mm on the PCB; both are user-configurable. See geom and Chapter 4.

H

Hierarchical sheet

A schematic feature where one sheet is included as a black-box symbol on another. Out of scope for PCBSmith Phase 1; flat sheets only.

I

IR (Intermediate Representation)

PCBSmith's structured-JSON format that LLMs emit instead of free text. Validated against a Pydantic schema before being materialised into a Schematic. See Chapter 8.

J

Junction

An explicit dot in a schematic indicating that crossing wires are electrically connected. PCBSmith treats junctions as first-class objects rather than inferring them from geometry — see core/schematic.py.

L

Layer

A 2D plane of features in the board: a copper layer, a silkscreen layer, a mask layer, etc. The full default 2-layer stack is documented in Chapter 9. See also stackup.

Library

A collection of reusable Symbols and Footprints. PCBSmith ships a 25-part minimal library and an importer for KiCad libraries (CC-BY-SA 4.0). See Chapter 14.

M

Material (solder mask, paste, silkscreen)

Fabrication-applied layers. Solder mask is the green coating that prevents solder bridging; solder paste is the stencil opening for SMT assembly; silkscreen is the printed white text and outlines. PCBSmith uses these exact terms — never "UV resin" or "ink layer".

N

Net

An electrically common collection of pins. Derived from Schematic via union-find on wires, junctions, and net labels. PCBSmith never asks the user to construct nets directly — they emerge from connectivity.

Net class

A named group of nets sharing design rules: e.g. POWER (wider traces), DIFF_PAIR (matched lengths). Phase 5 ships with one default class; additional classes are deferred.

Net label

A user-applied name on a wire that forces nets of the same name to be merged across sheets and across disconnected wire segments. The mechanism by which power and ground propagate without explicit wires.

Netlist

The set of all Nets in a Schematic. The output of derive_netlist().

P

Pad

A copper feature on a Footprint that solders to a part lead. Each pad has a number (matching a Symbol pin), a shape (circle, rectangle, oblong), a size, optional drill, and a layer set.

Padstack

The cross-section of a pad through the layer stack — what shape it is on F.Cu, on F.Mask, on B.Cu, etc. In Specctra/FreeRouting parlance, padstacks are first-class library objects.

Pin (schematic)

A connection point on a Symbol with a number, a name, an electrical type (input/output/power_in/...), and a direction. Pin numbers must match the associated Footprint's pad numbers.

Project

PCBSmith's top-level data class. Holds one Schematic, one Board, library references, and metadata. Persisted as a single .pcbsmith JSON file.

R

Ratsnest

The visual indication of unrouted electrical connections, drawn as thin straight lines between pads on the same net. Computed via networkx MST per net. See Chapter 9.

Reference (ref)

See designator.

S

Schematic

PCBSmith data class representing the electrical design: Symbol instances, wires, junctions, net labels, no-connect markers. Distinct from Board. See Chapter 4.

Selection

The set of currently selected scene items. Drives the inspector panel and is the default target of edit operations. PCBSmith uses Qt's built-in QGraphicsItem selection.

SES (Specctra Session)

The output format of an auto-router: a list of routed wire segments and vias, keyed by net name. Read by ses_importer to apply auto-route results to the Board.

Silkscreen

Printed text and outlines on the assembled board, typically white on green. Layer names: F.SilkS, B.SilkS. Holds component reference labels (R1, C2…) and orientation markers.

S-expression

A Lisp-derived parenthesised list syntax used by KiCad files (.kicad_sym, .kicad_mod), Specctra DSN, and SES. PCBSmith parses with sexpdata and emits with hand-rolled writers where formatting matters.

Stackup

The ordered set of physical layers in a Board, top to bottom: F.Paste, F.SilkS, F.Mask, F.Cu, dielectric, B.Cu, B.Mask, B.SilkS, B.Paste. Default is two-layer; multilayer is deferred.

Symbol

The schematic representation of a part: pins, body geometry, and a default reference prefix (R, C, U, …). Distinct from Footprint.

T

Trace

A copper segment connecting two points on a single layer. PCBSmith stores traces as straight line segments with explicit endpoints; arcs are out of scope for Phase 2 and deferred.

U

Unit (length)

PCBSmith's internal unit is the int64 nanometre. The UI displays millimetres or mils based on a per-project setting; conversions live in ui/units.py and core/geom.py. See Chapter 4.

Undo stack

QUndoStack instance per project. Every model change is wrapped in a QUndoCommand. The stack is shared by schematic and board so that Ctrl+Z across stages behaves predictably.

V

Via

A drilled, plated hole that connects copper on different layers. Through-hole vias span the full stack; blind/buried vias span subsets and are deferred until multilayer is supported.

W

Wire (schematic)

A line segment in the schematic that represents an electrical connection. Distinct from Trace (board copper). PCBSmith uses Wire on the schematic side and Trace on the PCB side; never confuse the two.

X

XNC

A strict, well-specified subset of the Excellon drill format defined by IPC-NC-349. PCBSmith emits XNC-conformant drill files; this is what modern fab houses prefer.

Z

Zone (copper pour)

A filled copper region on a layer assigned to a net (typically GND), with automatic clearance around other-net features. PCBSmith Phase 2 ships rectangular zones with simple flood-fill; smarter pour algorithms (thermal reliefs on through-hole pads, hatched fills) are deferred.

End of Specification

This document is the canonical reference for PCBSmith. It is intended to be loaded into a code-generation model at the start of every implementation session, alongside the persistent prompt template in Chapter 1. Treat it as living: when implementation reveals that a section is wrong, fix the section before fixing the code, and let the spec lead.

Document conventions for revisions. Revisions to this document are dated and listed at the top of Chapter 1. When changing a chapter, update its acceptance criteria simultaneously — an implementation guide that drifts from its acceptance tests is worse than no guide at all. Pull-request reviewers should reject changes to this document that do not include a rationale and a tested code example.